RKNN3 Runtime C API 参考手册

文件标识：RK-YH-YF-425

发布版本：V1.2.0

日期：2026-10-22

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文是Rockchip RKNN3 Runtime C API 参考手册。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V0.1.0	HPC团队	2025-05-12	初始版本	熊伟
V0.2.0	HPC团队	2025-08-04	更新API接口说明	熊伟
V0.3.0b0	HPC团队	2025-09-10	新增rknn3_set_internal_mem接口描述；增加KV缓存存储方式与数据类型定义；完善LLM配置结构体内容	熊伟
V0.4.0b0	HPC团队	2025-11-03	更新rknn3_vocab_info和rknn3_llm_multimodal_tensor结构体和相关接口描述	熊伟
V0.5.0	HPC团队	2025-11-29	增加后处理相关查询命令与自定义算子插件机制;调整输出回调函数接口;增加YUV输入相关结构体和接口描述	熊伟
V1.0.0	HPC团队	2025-12-22	增加性能和内存分析接口	熊伟
V1.0.4	HPC团队	2026-05-09	新增流式权重加载接口；新增rknn3_create_mem_from_fd接口	熊伟
V1.1.0	HPC团队	2026-08-03	新增rknn3_set_weight_mem、rknn3_set_kvcache_group、rknn3_session_pause、rknn3_session_resume、rknn3_mem_sync_range等接口说明	熊伟
V1.2.0	HPC团队	2026-10-22	新增PagedAttention支持；新增结构化输出（grammar）支持；新增MatMul矩阵乘法接口	熊伟

目录

- RKNN3 Runtime C API 参考手册
  - 目录
  - 1. 参考手册说明
    - 1.1 适用SDK和Runtime版本
    - 1.2 支持平台
    - 1.3 头文件和运行时库
    - 1.4 API稳定性和兼容性
  - 2. 编译和链接
    - 2.1 Linux
    - 2.2 Android
    - 2.3 交叉编译
  - 3. C API公共约定
    - 3.1 Context、Model和Session生命周期
    - 3.2 内存所有权
    - 3.3 输入输出数据有效期
    - 3.4 同步和异步行为
    - 3.5 线程安全和Callback
    - 3.6 返回值和错误处理
    - 3.7 平台差异
    - 3.8 API废弃规则
  - 4. 调用流程
    - 4.1 通用模型推理
    - 4.2 LLM Session推理
    - 4.3 多模态大模型推理
    - 4.4 外部内存使用流程
      - 4.4.1 权重内存
      - 4.4.2 内部内存
      - 4.4.3 KV cache内存
      - 4.4.4 Page KV cache内存
    - 4.5 MatMul矩阵乘法
      - 4.5.1 多shape注册与适用情况
      - 4.5.2 多shape切换流程
      - 4.5.3 多shape的量化参数

5. 函数参考
5.1. 设备与上下文
rknn3_init
rknn3_destroy
rknn3_find_devices
rknn3_set_input_name_alias
5.2. 模型加载与初始化
rknn3_load_model_from_path
rknn3_load_model_from_data
rknn3_load_model_from_device
rknn3_model_init
rknn3_load_weight_chunk
5.3. 查询
rknn3_query
5.4. 推理执行
rknn3_run
rknn3_set_shape
5.5. 张量和内存
rknn3_create_mem_from_fd
rknn3_create_mem
rknn3_destroy_mem
rknn3_mem_sync
rknn3_mem_sync_range
rknn3_set_kvcache_pages
rknn3_set_kvcache_mem
rknn3_set_internal_mem
rknn3_set_weight_mem
rknn3_set_kvcache_group
5.6. 模型解密
rknn3_set_decrypt_key_from_path
rknn3_set_decrypt_key_from_data
5.7. LLM Session
rknn3_session_init
rknn3_session_destroy
rknn3_session_run
rknn3_session_run_async

rknn3_session_stop
rknn3_session_pause
rknn3_session_resume
rknn3_session_query_state
5.8. LLM配置与Callback
rknn3_session_set_callback
rknn3_session_set_chat_template
rknn3_session_set_llm_param
rknn3_session_set_function_tools
LLMResultCallback
LLMSamplingCallback
LLMGetEmbedCallback
LLMTokenizerCallback
LLMOutputCallback
LLMInputCallback
5.9. KVCache
rknn3_session_set_kvcache_policy
rknn3_session_clear_kvcache
rknn3_session_load_kvcache_from_path
rknn3_session_load_kvcache_from_data
rknn3_session_save_kvcache
5.10. LoRA
rknn3_lora_init
rknn3_lora_init_from_data
rknn3_lora_load
rknn3_lora_unload
rknn3_lora_enable
rknn3_lora_disable
rknn3_session_enable_lora
rknn3_session_disable_lora
5.11. 性能分析与调试
rknn3_dump_features
rknn3_profile_ops
rknn3_profile_mem
5.12. 自定义算子
rknn3_register_custom_ops_plugins

rknn3_register_custom_op_func
5.13. 图像处理
rknn3_im_mem_create
rknn3_im_mem_destroy
rknn3_im_cvt_color
5.14. 辅助函数
rknn3_get_type_string
rknn3_get_qnt_type_string
rknn3_get_layout_string
5.15. 结构化输出(grammar)
rknn3_init_grammar
rknn3_grammar_create
rknn3_grammar_destroy
rknn3_grammar_set
rknn3_grammar_clear
rknn3_grammar_apply_mask
rknn3_grammar_accept_token
rknn3_grammar_reset
rknn3_session_set_grammar
rknn3_session_clear_grammar
5.16. MatMul矩阵乘法
rknn3_matmul_init
rknn3_matmul_query_infos
rknn3_matmul_build
rknn3_matmul_set_shape
rknn3_matmul_transpose
rknn3_matmul_set_quant
rknn3_matmul_run
rknn3_matmul_destroy
使用示例
6. 类型参考
6.1. 常量
tensor 相关常量
设备相关常量
6.2. 状态码
6.3. 枚举

6.3.1. Tensor 与查询枚举

rknn3_query_cmd

rknn3_tensor_type

rknn3_tensor_qnt_type

rknn3_tensor_layout

6.3.2. 内存与核心枚举

rknn3_mem_alloc_flags

rknn3_mem_sync_mode

rknn3_mem_type

rknn3_core_mask

6.3.3. KV Cache 枚举

rknn3_kvcache_policy

rknn3_kvcache_clear_policy

rknn3_kvcache_dtype

rknn3_kvcache_store_method

rknn3_attention_type

rknn3_page_cache_type

6.3.4. LLM 枚举

rknn3_llm_input_type

LLMCallState

rknn3_llm_task_type

LLMOutputCallbackState

6.3.5. 图像处理枚举

rknn3_im_fmt

rknn3_im_proc_flag

6.3.6. 自定义算子枚举

rknn3_op_target_type

rknn3_op_plugin_type

6.3.7. 结构化输出枚举

rknn3_grammar_type

6.4. 通用结构体

Tensor 相关结构体

rknn3_input_output_num

rknn3_quant_info

rknn3_tensor_attr

rknn3_tensor_mem
rknn3_tensor
模型与配置结构体
rknn3_sdk_version
rknn3_core_mem_size
rknn3_custom_string
rknn3_config
rknn3_shape_info
rknn3_shape_config
rknn3_allocation_info
设备相关结构体
rknn3_init_extend
rknn3_node_mem_info
rknn3_dev_mem_info
rknn3_device
rknn3_devices
6.5. LLM和VLM结构体
rknn3_llm_config
rknn3_vocab_info
rknn3_llm_extend_param
rknn3_sampling_params
rknn3_llm_param
rknn3_lora
rknn3_llm_multimodal_tensor
rknn3_llm_tensor
rknn3_llm_input
rknn3_llm_infer_param
RKLLMResult
RKLLMRunState
6.6. KVCache结构体
rknn3_kvcache_policy_param
rknn3_attention_kvcache_lens
rknn3_kvcache_len_group_info
rknn3_attention_config
rknn3_cache_page_attr
rknn3_kvcache_page_info

rknn3_kvcache_page_binding
6.7. 自定义算子结构体
rknn3_custom_op_attr
rknn3_custom_op_context
rknn3_custom_op
6.8. 图像处理结构体
rknn3_im_rect
rknn3_im_metadata
rknn3_im_mem
6.9. Callback类型
LLMResultCallback
LLMSamplingCallback
LLMGetEmbedCallback
LLMTokenizerCallback
LLMOutputCallback
LLMInputCallback
RKLLMCallback
6.10. 结构化输出结构体
rknn3_grammar_config
rknn3_grammar
rknn3_grammar_set_options
6.11. MatMul类型
rknn3_matmul_context
rknn3_matmul_type
rknn3_matmul_desc
rknn3_matmul_config
rknn3_matmul_tensor_attr
rknn3_matmul_info
rknn3_matmul_quant_type_t
rknn3_matmul_quantization_t
scales与zero_points的存储约定
W4A16/W6A16/W8A16：B沿K方向分组
W8A8 per-channel：B每个输出通道一对参数
W8A8 per-layer：B每个批次一对参数
W16A16/W8A8O32：不提交scale/zp
7. API变更、废弃和迁移

v0.3.0 -> v0.4.0 Session API 主要改动 (2025-11-03)

v0.4.0b0 -> v0.5.0 主要改动 (2025-11-29)

v0.5.0 -> v1.0.0 主要改动 (2025-12-22)

v1.0.0 -> v1.0.4 主要改动 (2026-05-09)

v1.0.4 -> v1.1.0 主要改动 (2026-08-03)

v1.1.0 -> v1.2.0 主要改动 (2026-10-22)

1. 参考手册说明

RKNN3 C API是RKNN3 Runtime（运行时库）的C语言接口。开发者使用C/C++开发应用程序，通过RKNN3 C API部署模型推理。本文对RKNN3 C API的各个函数、数据结构以及状态码等定义进行说明。

1.1 适用SDK和Runtime版本

本文档对应RKNN3 Runtime发布版本 V1.2.0（见修订记录）。本文档描述的接口以 V1.2.0 版本为准。

1.2 支持平台

本文档适用如下硬件平台：

协处理器平台（NPU加速器，通过PCIe/USB连接主控SoC）：

- RK1820
- RK1828

通用SoC平台（集成NPU，应用程序直接在SoC上运行）：

- RK3572

1.3 头文件和运行时库

RKNN3 SDK支持两种部署架构：

协处理器模式： 应用程序的编译和运行在主控SoC上完成，协处理器（RK1820/RK1828）仅作为NPU加速单元。主控SoC通过PCIe/USB高速接口连接协处理器，例如RK3588连接到RK1820/RK1828。此模式下需要librknn3_api、rknn3_transfer_proxy与协处理器固件三者版本一致。

通用SoC模式： 应用程序直接在RK3572上编译和运行，NPU为SoC内部集成单元，无需协处理器通信。此模式下需要librknn3_api与rknn3_server两者版本一致。

两种模式下，开发者在编译RKNN3应用时均需包含 `rknn3_api.h` 头文件并链接 `librknn3_api.so` 运行时库。使用MatMul接口时，还需包含 `rknn3_matmul_api.h` 。

RKNN3 C API 定义于 `rknn3_api.h` 头文件，按功能分为用于通用深度学习模型部署的基础推理接口（如 `rknn3_init` 、 `rknn3_run` 等）以及专门为大型语言模型优化设计的 session 系列接口（如 `rknn3_session_init` 、 `rknn3_session_run` 等），后者提供 KV 缓存管理、LoRA 支持、采样策略配置等功能。

RKNN3 SDK在运行时涉及三个动态库，其关系如下：

库名	说明
librknn3_api.so	API入口层，应用程序链接此库。运行时根据部署模式自动加载对应的后端库
librknn3_api_rkcp.so	协处理器模式后端库。通过localsocket与rknn3_transfer_proxy通信，再经PCIe/USB转发至协处理器执行推理
librknn3_api_native.so	通用SoC模式后端库。直接访问SoC集成NPU，无需协处理器通信

协处理器模式调用链路：

```
APP -> librknn3_api.so -> librknn3_api_rkcp.so -> rknn3_transfer_proxy -> PCIe/USB
-> 协处理器(RK1820/RK1828)
```

通用SoC模式调用链路：

```
APP -> librknn3_api.so -> librknn3_api_native.so -> NPU(集成在SoC内)
```

应用程序只需链接 `librknn3_api.so`，运行时由该库自动检测部署模式并加载 `librknn3_api_rkcp.so` 或 `librknn3_api_native.so`。部署时需确保对应的后端库与 `librknn3_api.so` 位于同一目录或系统库搜索路径中。应用程序也可以根据实际场景，直接链接librknn3_api_rkcp.so或者librknn3_api_native.so。

获取运行时版本号有以下两种方式：

- 对相应的运行时库执行 `strings` 命令并过滤版本字符串，例如 `strings librknn3_api.so | grep "version:"`（协处理器模式对应 `librknn3_api_rkcp.so`，通用SoC模式对应 `librknn3_api_native.so`），即可从版本字符串中读出运行时版本；
- 在运行时通过 `RKNN3_QUERY_SDK_VERSION` 查询实际版本，结果写入 `rknn3_sdk_version` 结构体的 `api_version` 与 `drv_version` 字段（详见 5. 函数参考 中的 `rknn3_query` 接口）。

1.4 API稳定性和兼容性

- **版本一致性要求：** `librknn3_api`、`rknn3_transfer_proxy`（协处理器模式）与协处理器固件，或 `rknn3_server`（通用SoC模式）三者版本必须一致，版本不兼容时返回 `RKNN3_ERR_INCOMPATIBLE_VERSION`。
- **平台限制：** 部分接口仅在特定平台可用，例如 `rknn3_set_weight_mem` 当前版本仅支持RK3572。

2. 编译和链接

开发者在编译RKNN3应用时需根据目标平台的硬件架构和系统类型选择相应的交叉编译工具链，并确保正确链接对应平台的RKNN3 API库。RKNN3 C API的调用流程在协处理器模式和通用SoC模式下保持一致，区别仅在于底层通信方式（协处理器模式通过PCIe/USB与协处理器通信，通用SoC模式直接访问集成NPU）。

2.1 Linux

在Linux平台（含主控SoC上的Linux）上编译RKNN3应用时，需包含 `rknn3_api.h` 头文件并链接 `librknn3_api.so` 。编译示例：

```
aarch64-linux-gnu-gcc app.c -I/path/to/include -L/path/to/lib -lrknn3_api -o app
```

2.2 Android

在Android平台编译RKNN3应用时，使用Android NDK工具链，目标架构为arm64-v8a。编译时需包含 `rknn3_api.h` 并链接 `librknn3_api.so` ，编译示例：

```
aarch64-linux-android24-clang app.c -I/path/to/include -L/path/to/lib -lrknn3_api -
o app
```

2.3 交叉编译

协处理器模式下，应用程序的编译和运行在主控SoC上完成，需使用主控SoC对应的交叉编译工具链。主控SoC的交叉编译器的下载和安装方法可参考< `Rockchip_RK182X_Quick_Start_RKNN3_SDK` >文档。通用SoC模式下，应用程序直接在RK3572上编译和运行，使用RK3572对应的工具链。

无论哪种模式，编译时均需：

1. 在头文件搜索路径中包含 `rknn3_api.h` 所在目录；
2. 在库搜索路径中包含 `librknn3_api.so` 所在目录，并链接 `-lrknn3_api` ；
3. 根据部署架构选择对应平台的交叉编译工具链。

3. C API公共约定

本章描述RKNN3 C API在使用过程中的公共约定，包括对象生命周期、内存所有权、同步异步行为、线程安全、返回值与错误处理、平台差异及废弃规则等。各函数的具体约束见 5. 函数参考。

3.1 Context、Model和Session生命周期

RKNN3 API围绕三类核心对象组织：运行时上下文（context）、模型（model）和会话（session）。其典型生命周期如下：

1. **上下文初始化：** 调用 `rknn3_init` 创建上下文，必须在加载和运行模型前调用；不再使用时调用 `rknn3_destroy` 释放，每个context只销毁一次。
2. **模型加载与初始化：** 通过 `rknn3_load_model_from_path` / `rknn3_load_model_from_data` 加载模型（设备端文件系统上的模型文件可通过 `rknn3_load_model_from_device` 加载），再调用 `rknn3_model_init` 完成模型初始化配置。
3. **流式权重加载（可选）：** 当加载模型时权重参数为NULL，进入流式权重加载模式，需在 `rknn3_model_init` 之后通过 `rknn3_load_weight_chunk` 分段上传权重数据，累计达到权重总大小时流式状态自动完成。
4. **会话初始化（LLM场景）：** 调用 `rknn3_session_init` 创建会话；不再使用时调用 `rknn3_session_destroy` 销毁会话。
5. **资源释放顺序：** 先销毁会话（`rknn3_session_destroy`），再销毁上下文（`rknn3_destroy`）。

此外，部分接口对调用时机有要求：

- 模型解密密钥设置（`rknn3_set_decrypt_key_from_*`）必须在 `rknn3_load_model_from_*` 之前调用。
- `rknn3_set_kvcache_mem` 需在 `rknn3_model_init` 之后调用。
- 用户分配的内部内存（`rknn3_set_internal_mem`）和权重内存（`rknn3_set_weight_mem`）需在 `rknn3_model_init` 之前将 `config.user_mem_internal` / `config.user_mem_weight` 置1，再在 `rknn3_model_init` 之后调用相应设置接口。
- LoRA初始化（`rknn3_lora_init` / `rknn3_lora_init_from_data`）需在 `rknn3_model_init` 之后、会话使用LoRA之前调用。

3.2 内存所有权

- **API分配的内存：** 通过 `rknn3_create_mem` 分配的 tensor 内存由调用方所有，必须在不再使用时调用 `rknn3_destroy_mem` 释放，避免重复释放。
- **用户分配的内存：** `rknn3_set_kvcache_mem`、`rknn3_set_internal_mem`、`rknn3_set_weight_mem` 接受的用户分配内存仍由调用方所有，使用完毕后需由调用方通过 `rknn3_destroy_mem` 释放。
- **Page KV cache内存：** page KV cache的每个页都是独立的 `rknn3_tensor_mem` 对象，由调用方通过 `rknn3_create_mem` 创建并通过 `rknn3_destroy_mem` 销毁。`rknn3_set_kvcache_pages` 不复制页内容，也不转移页内存所有权；页处于绑定状态时必须保持有效。销毁前先使用 `n_pages = 0` 的绑定解除对应页表项。

- **外部内存封装：** `rknn3_create_mem_from_fd` 将已分配的内存（通过文件描述符表示）与RKNN3框架关联，实现零拷贝，封装出的 `rknn3_tensor_mem` 对象仍需通过 `rknn3_destroy_mem` 释放封装对象。
- **回调中的内存：** 回调函数中传入的 tensor 数据指针（如自定义算子属性数据、输出tensor）通常非调用方所有，仅在当前回调作用域内有效，调用方不应缓存或释放。

3.3 输入输出数据有效期

- 推理接口（`rknn3_run`）的 `inputs` / `outputs` 数组及其指向的 `rknn3_tensor`、内存必须在调用期间保持有效，且输入输出tensor的内存需由用户提前分配。
- CPU写入输入内存后、NPU访问前，需调用 `rknn3_mem_sync`（`RKNN3_MEMORY_SYNC_TO_DEVICE`）将CPU cache数据同步到内存；NPU写入输出内存后、CPU读取前，需调用 `rknn3_mem_sync`（`RKNN3_MEMORY_SYNC_FROM_DEVICE`）使CPU cache失效并从内存重新读取。未同步可能导致推理结果异常。
- 回调中获取的 tensor 指针（如 `LLMOutputCallback` 的 `output_tensors`、`rknn3_custom_op_attr` 的 `data`）仅在回调作用域内有效，回调返回后不应继续使用。

3.4 同步和异步行为

- **通用模型同步推理：** `rknn3_run` 为同步接口，推理过程中会阻塞直到完成。
- **LLM会话推理：** `rknn3_session_run` / `rknn3_session_run_async` 的推理结果通过回调函数异步获取（见 5.8 LLM配置与Callback）。
- **内存同步：** `rknn3_mem_sync` / `rknn3_mem_sync_range` 用于CPU与设备间的数据一致性同步；`rknn3_mem_sync_range` 仅同步指定 `[offset, offset+length)` 区间，适用于部分更新场景。

3.5 线程安全和Callback

- 同一个 `rknn3_context` 或 `rknn3_session` 的并发使用需调用方自行保证线程安全，避免多线程同时操作同一句柄。
- LLM推理结果通过回调异步返回，回调函数在推理过程中被触发；回调内不应执行长时间阻塞操作或递归发起同一会话的推理，以免影响推理流程。
- 回调中传入的临时数据（tensor指针、属性数据等）仅在回调作用域内有效，如需保留应自行拷贝。

3.6 返回值和错误处理

- 大多数接口返回 `int` 状态码：**0 表示成功，负值表示错误**。个别接口返回指针（如 `rknn3_create_mem`、`rknn3_create_mem_from_fd`、`rknn3_session_init`），失败时返回 `NULL`。
- 完整状态码定义见 6.2 状态码。
- 调用方应检查每次调用的返回值，错误处理时根据状态码定位问题（如 `RKNN3_ERR_INCOMPATIBLE_VERSION` 表示组件版本不匹配、`RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED` 表示流式权重加载未准备好等）。

3.7 平台差异

- **部署模式差异：** 协处理器模式通过PCIe/USB与协处理器通信，需 `librknn3_api_rkcp.so` + `rknn3_transfer_proxy` + 协处理器固件；通用SoC模式直接访问集成NPU，需 `librknn3_api_native.so` + `rknn3_server`。详见 1.3 头文件和运行时库。
- **接口可用性差异：** 部分接口仅在特定平台支持，例如 `rknn3_set_weight_mem` 当前版本仅支持RK3572。
- **调用流程一致性：** C API的调用流程在协处理器模式和通用SoC模式下保持一致，区别仅在于底层通信方式。

3.8 API废弃规则

- 接口的废弃与变更历史见 7. API变更、废弃和迁移。
- 各组件版本必须一致（见 1.4 API稳定性和兼容性），升级时需同步更新 `librknn3_api`、`rknn3_transfer_proxy` / `rknn3_server` 与固件。

4. 调用流程

RKNN3 C API的调用流程在协处理器模式和通用SoC模式下保持一致，区别仅在于底层通信方式（协处理器模式通过PCIe/USB与协处理器通信，通用SoC模式直接访问集成NPU）。以下分别介绍通用模型同步推理、通用模型异步推理、LLM Session推理、多模态大模型推理及外部内存使用流程。

4.1 通用模型推理

CNN/ViT等非大语言模型使用基础API的推理流程主要包括以下几个步骤。首先，用户需调用rknn3_init接口初始化运行时上下文，并通过rknn3_load_model_from_path或rknn3_load_model_from_data加载模型。模型加载完成后，需调用rknn3_model_init进行模型初始化配置。随后，用户可通过rknn3_query接口获取输入输出tensor的属性信息，并为输入输出分配内存。推理时，调用rknn3_run接口完成同步推理。最后，使用rknn3_destroy释放上下文资源，基础API的推理流程如图4-1所示。

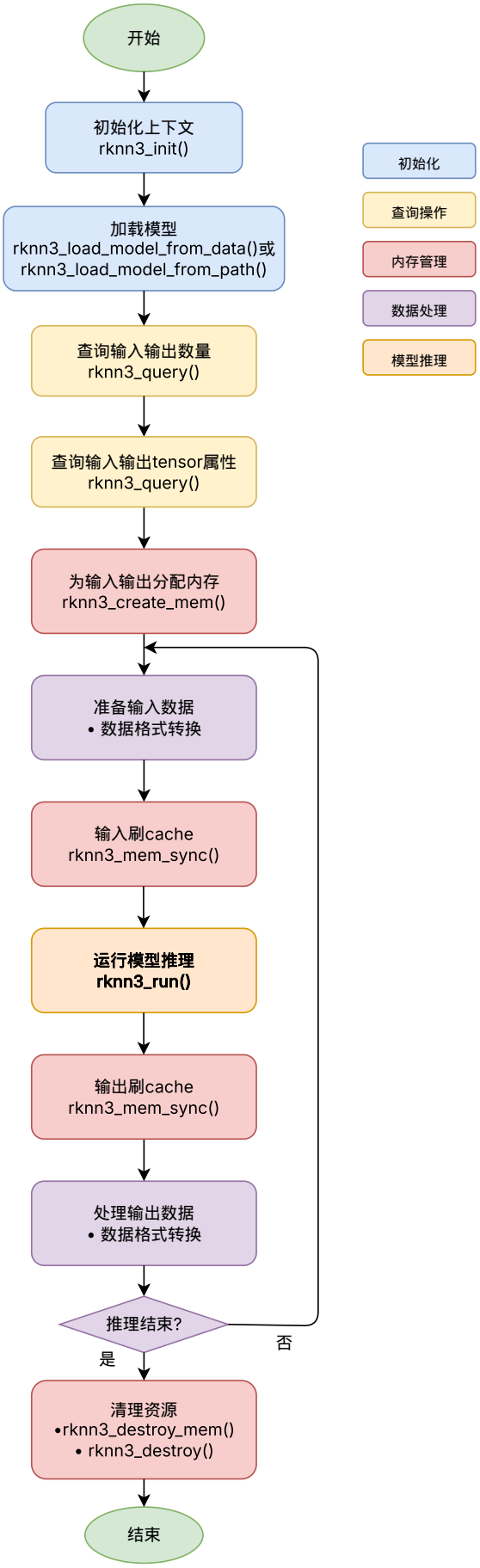


图4-1 基础API的推理流程

4.2 LLM Session推理

RKNN3大语言模型推理流程通常包括以下主要步骤。首先，用户需通过rknn3_find_devices获取可用设备列表，并调用rknn3_init初始化上下文，随后使用rknn3_load_model_from_path加载模型文件，并通过rknn3_model_init完成模型初始化配置。接下来，用户需准备分词器和embedding等相关资源，并设置rknn3_llm_param结构体以配置会话参数。之后，调用rknn3_session_init初始化会话，并通过rknn3_session_set_callback设置推理过程中的回调函数。推理前可根据需要设置聊天模板或清理KV缓存。

在推理阶段，用户构造rknn3_llm_input输入结构体，调用rknn3_session_run接口进行推理，推理结果通过回调函数异步获取。推理完成后，可根据需要统计性能数据或进行多轮对话。最后，需调用rknn3_session_destroy和rknn3_destroy接口释放会话和上下文资源，完成整个推理流程。大语言模型推理流程如图4-2所示。

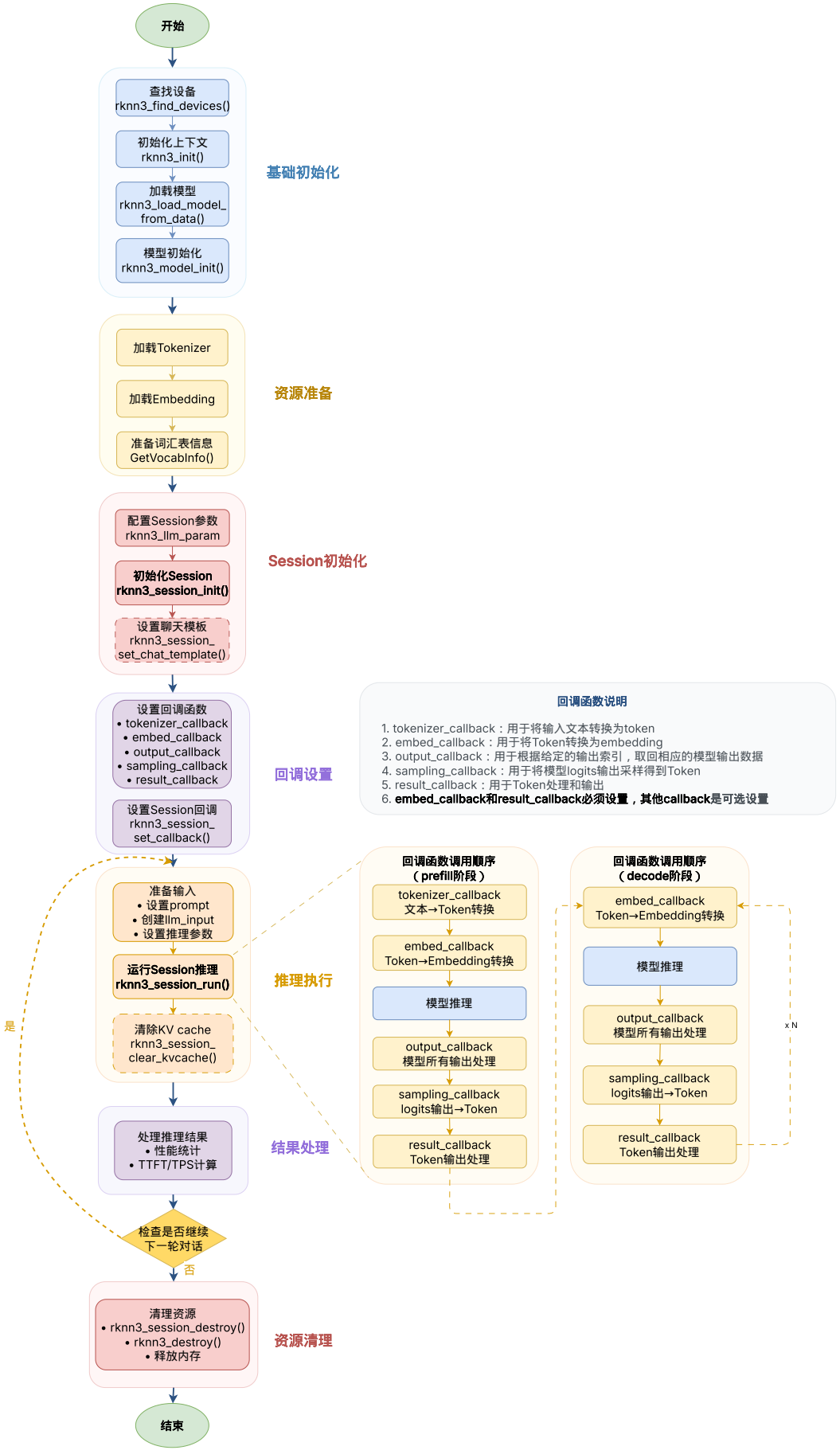


图4-2 大语言模型推理流程

4.3 多模态大模型推理

多模态大模型推理（支持语音、图像、文字、视频等多种模态输入）基于LLM Session流程，与纯文本LLM推理的关键区别在于：除LLM本身外，还需要独立的模态编码器模型（如视觉编码器、音频编码器）将图像、音频等非文本模态转换为嵌入向量（embedding），再将嵌入与文本提示一同送入LLM完成推理。多模态大模型推理的整体流程如图4-3所示。

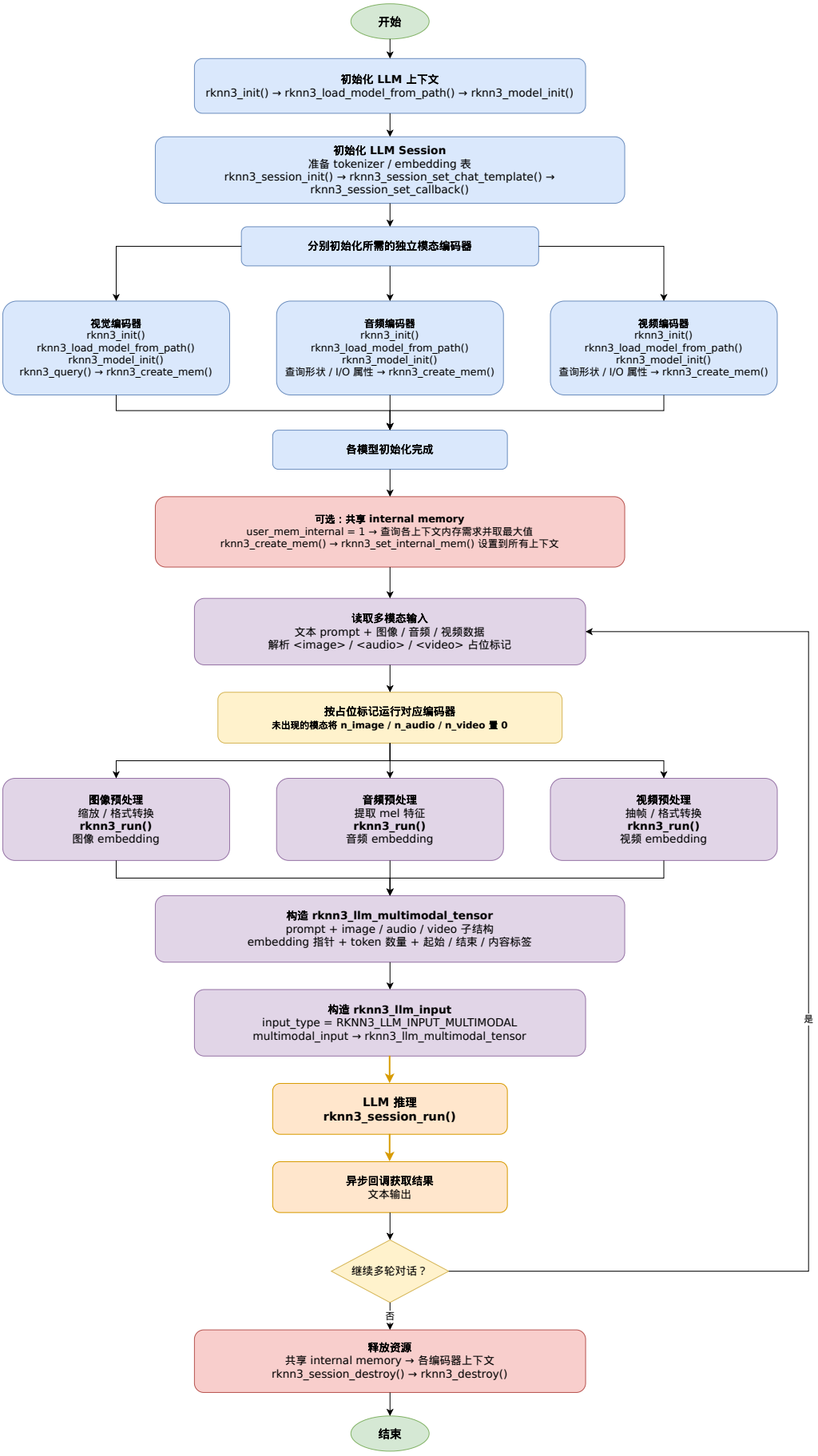


图4-3 多模态大模型推理流程

主要步骤说明如下：

1. **模型初始化：** 多模态推理涉及多个RKNN3上下文——每个模态编码器（如视觉编码器、音频编码器）各一个上下文，LLM一个上下文。各上下文的初始化流程一致：`rknn3_init` 创建上下文，`rknn3_load_model_from_path` 加载模型，`rknn3_model_init` 完成模型初始化。其中，模态编码器需通过 `RKNN3_QUERY_INPUT_ATTR` / `RKNN3_QUERY_OUTPUT_ATTR` 查询输入输出属性后用 `rknn3_create_mem` 分配内存；LLM则需调用 `rknn3_session_init` 创建会话，并通过 `rknn3_session_set_chat_template` 设置聊天模板、`rknn3_session_set_callback` 设置回调函数。此外还需准备分词器（tokenizer）与embedding表等辅助资源，供回调中的分词与token嵌入查找使用。
2. **内部内存共享：** 为节省内存，各模态编码器与LLM的上下文可共享同一块内部内存（internal memory）。初始化时将各上下文 `config.user_mem_internal` 置1，模型初始化后通过 `RKNN3_QUERY_CORE_NUMBER` 与 `RKNN3_QUERY_CORE_MEM_SIZE` 查询各上下文各核心所需内部内存大小，取最大值后用 `rknn3_create_mem` 分配，再通过 `rknn3_set_internal_mem` 设置到所有上下文。
3. **模态嵌入生成：** 根据输入prompt中的模态占位标记，分别运行对应的编码器生成嵌入。若prompt包含 `<image>`，对输入图像做预处理（缩放、格式转换等）后送入视觉编码器推理（`rknn3_run`），获取图像嵌入；若包含 `<audio>`，对输入音频做预处理（如提取mel特征）后送入音频编码器推理，获取音频嵌入；视频模态同理。未出现的模态将其数量（`n_image` / `n_audio` / `n_video`）置0即可跳过。
4. **构造多模态输入并推理：** 构造 `rknn3_llm_multimodal_tensor`，设置 `prompt` 文本，并将各模态嵌入及对应的标记标签填入 `image` / `audio` / `video` 子结构（嵌入指针、token数量、数量、起始/结束/内容占位标签等）。标签需与模型约定一致，例如图像使用 `<|vision_bos|>` / `<|vision_eos|>` / `<|IMAGE|>`，音频使用 `<|audio_bos|>` / `<|audio_eos|>` / `<|AUDIO|>`。在prompt中通过 `<image>` / `<audio>` / `<video>` 等占位标记指示各模态数据的插入位置，多种模态数据可以任意顺序和组合方式输入。随后构造 `rknn3_llm_input`，将 `input_type` 设为 `RKNN3_LLM_INPUT_MULTIMODAL`，`multimodal_input` 指向上述结构体，调用 `rknn3_session_run` 进行推理，结果通过回调函数异步获取。
5. **资源释放：** 推理完成后，先释放共享的内部内存，再依次释放各模态编码器上下文与LLM会话、上下文资源。

多模态输入结构体 `rknn3_llm_multimodal_tensor` 的字段定义见 6.5 LLM和VLM结构体。

4.4 外部内存使用流程

当用户希望自行分配并管理权重内存、内部内存或KV cache内存时，可按以下流程使用外部内存。三种外部内存的查询方式与设置时机各有不同：权重内存和内部内存需在 `rknn3_model_init` 之前开启对应配置标志，初始化后通过 `RKNN3_QUERY_CORE_MEM_SIZE` 查询大小；KV cache内存则在 `rknn3_model_init` 之后通过 `RKNN3_QUERY_ALLOCATION_INFO` 查询大小。三类外部内存的整体使用流程如图4-4所示。

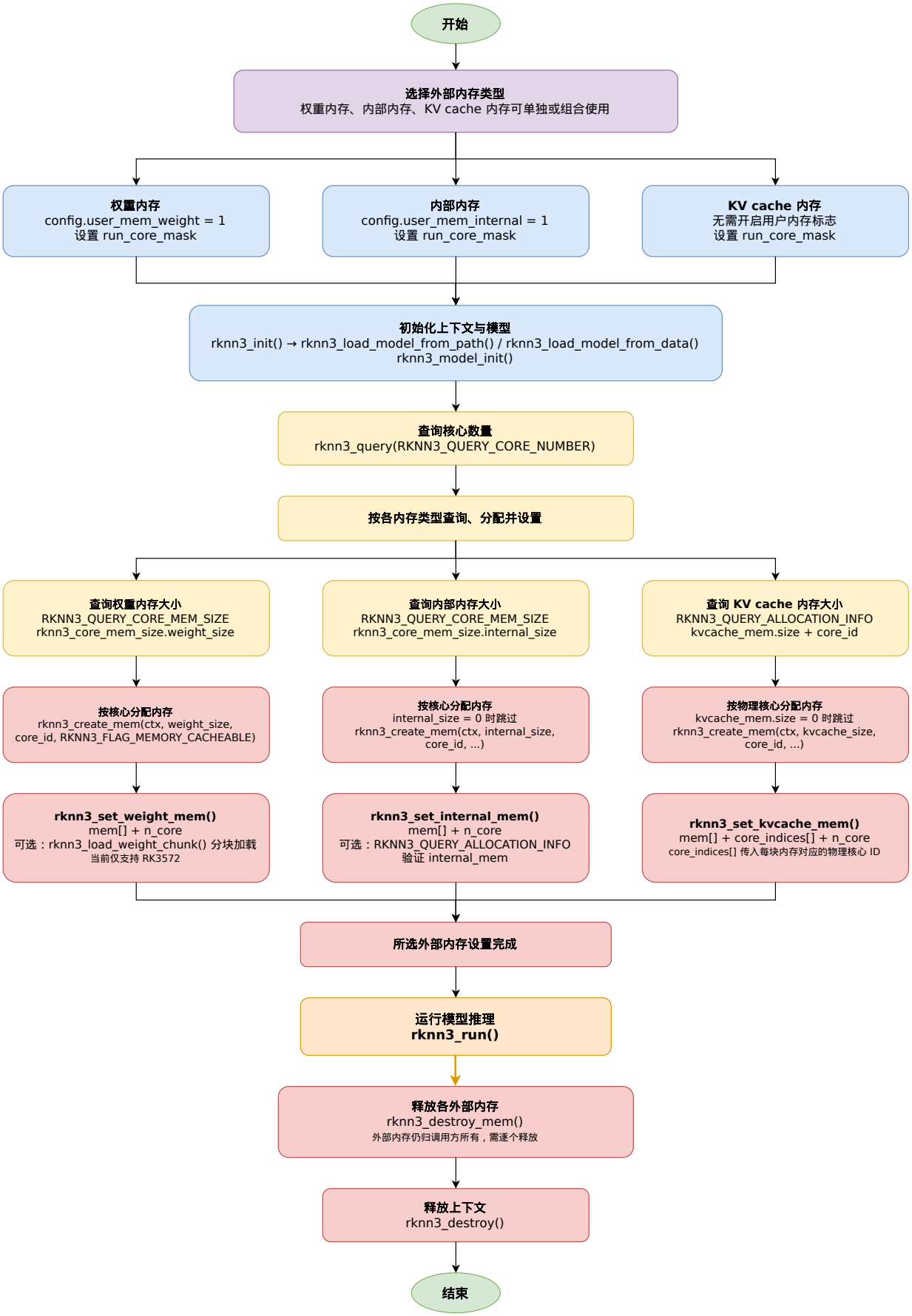


图4-4 外部内存使用流程

> **注意：** `user_mem_weight` / `user_mem_internal` 配置标志须在 `rknn3_model_init` 之前置位；权重内存与内部内存的查询、分配和设置需在 `rknn3_model_init` 之后、`rknn3_run` 之前完成。KV cache内存需在 `rknn3_model_init` 之后设置。三种外部内存可单独使用，也可组合使用。
>
> **重要提醒：** 若通过 `RKNN3_QUERY_ALLOCATION_INFO` 查询到的 KV cache 内存大小（`kvcache_mem.size`）非0，则在调用 `rknn3_run` 执行推理前，必须通过 `rknn3_set_kvcache_mem` 设置用户分配的 KV cache 外部内存；否则推理将因缺少 KV cache 内存而失败。若使用 `rknn3_session_run` 接口进行 LLM 推理，则无需设置 KV cache 外部内存，KV cache 内存由 session 内部管理。

各内存类型的具体步骤说明如下：

4.4.1 权重内存

权重内存用于存放模型权重数据。使用用户管理的权重内存时，需在模型初始化前开启 `user_mem_weight` 标志，初始化后查询各核心所需权重内存大小并分配设置。步骤如下：

1. **初始化上下文与加载模型：** 调用 `rknn3_init` 创建上下文，通过 `rknn3_load_model_from_path` 或 `rknn3_load_model_from_data` 加载模型。采用用户权重内存时，`rknn3_load_model_from_data` 的权重参数可传 `NULL`（流式分块加载时权重后续通过 `rknn3_load_weight_chunk` 上传），也可一并传入完整权重数据。
2. **设置配置并初始化模型：** 构造 `rknn3_config`，将 `config.user_mem_weight` 置1并设置 `run_core_mask`，随后调用 `rknn3_model_init` 完成模型初始化。
3. **查询核心数量：** 通过 `RKNN3_QUERY_CORE_NUMBER` 查询核心数量，用于后续按核心分配内存。
4. **查询权重内存大小：** 通过 `RKNN3_QUERY_CORE_MEM_SIZE` 查询各核心的权重内存大小（`rknn3_core_mem_size` 结构体的 `weight_size` 字段）。
5. **分配并设置权重内存：** 对每个核心，调用 `rknn3_create_mem(ctx, weight_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` 分配权重内存，收集到数组后调用 `rknn3_set_weight_mem(ctx, mem[], n_core)` 设置（当前版本仅支持RK3572）。
6. **流式加载权重（可选）：** 若步骤1中权重参数传 `NULL`，需通过 `rknn3_load_weight_chunk(ctx, offset, data, size)` 分块上传权重数据，直到全部权重上传完毕后方可推理。
7. **释放资源：** 推理完成后，由调用方通过 `rknn3_destroy_mem` 释放各权重内存，再调用 `rknn3_destroy` 释放上下文。

4.4.2 内部内存

内部内存（internal memory）用于存放模型推理过程中的中间张量等数据。使用用户管理的内部内存时，同样需在模型初始化前开启 `user_mem_internal` 标志，初始化后查询各核心所需内部内存大小并分配设置。步骤如下：

1. **初始化上下文与加载模型：** 调用 `rknn3_init` 创建上下文，通过 `rknn3_load_model_from_path` 或 `rknn3_load_model_from_data` 加载模型。
2. **设置配置并初始化模型：** 构造 `rknn3_config`，将 `config.user_mem_internal` 置1并设置 `run_core_mask`，随后调用 `rknn3_model_init` 完成模型初始化。
3. **查询核心数量：** 通过 `RKNN3_QUERY_CORE_NUMBER` 查询核心数量。

4. **查询内部内存大小：** 通过 `RKNN3_QUERY_CORE_MEM_SIZE` 查询各核心的内部内存大小（`rknn3_core_mem_size` 结构体的 `internal_size` 字段）。若 `internal_size` 为0则该核心无需分配。
5. **分配并设置内部内存：** 对每个核心，调用 `rknn3_create_mem(ctx, internal_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` 分配内部内存，收集到数组后调用 `rknn3_set_internal_mem(ctx, mem[], n_core)` 设置。
6. **验证（可选）：** 可通过 `RKNN3_QUERY_ALLOCATION_INFO` 查询内存分配信息（`rknn3_allocation_info` 结构体的 `internal_mem` 字段），确认内部内存已正确设置。
7. **释放资源：** 推理完成后，由调用方通过 `rknn3_destroy_mem` 释放各内部内存，再调用 `rknn3_destroy` 释放上下文。

> **提示：** 多模态场景下，多个上下文（如视觉编码器、音频编码器、LLM）可共享同一块内部内存以节省内存。具体做法是对各上下文分别开启 `user_mem_internal` 并查询各核心内存大小，取各上下文所需大小的最大值后分配一块共享内存，再通过 `rknn3_set_internal_mem` 设置到所有上下文（见 4.3 多模态大模型推理）。

4.4.3 KV cache内存

KV cache内存用于存放大语言模型推理过程中的Key/Value缓存。与权重内存和内部内存不同，KV cache内存无需在模型初始化前开启配置标志，而是在 `rknn3_model_init` 之后通过 `RKNN3_QUERY_ALLOCATION_INFO` 查询各核心所需大小并分配设置。步骤如下：

1. **初始化上下文与加载模型：** 调用 `rknn3_init` 创建上下文，通过 `rknn3_load_model_from_path` 或 `rknn3_load_model_from_data` 加载模型，设置 `run_core_mask` 并调用 `rknn3_model_init` 完成模型初始化。
2. **查询核心数量：** 通过 `RKNN3_QUERY_CORE_NUMBER` 查询核心数量。
3. **查询KV cache内存大小：** 通过 `RKNN3_QUERY_ALLOCATION_INFO` 查询各核心的内存分配信息（`rknn3_allocation_info` 结构体），其中 `kvcache_mem.size` 字段为该核心所需KV cache内存大小，`core_id` 字段为该核心的物理ID。若 `kvcache_mem.size` 为0则该核心无需分配。
4. **分配KV cache内存：** 对需要分配的核心，调用 `rknn3_create_mem(ctx, kvcache_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` 分配KV cache内存，同时记录各内存对应的 `core_id`。
5. **设置KV cache内存：** 调用 `rknn3_set_kvcache_mem(ctx, mem[], core_indices[], n_core)` 设置，其中 `core_indices[]` 为各内存对应的物理核心ID数组，`n_core` 为内存数量。注意该接口需要传入核心ID数组，与权重/内部内存接口（传入核心数量）不同。
6. **释放资源：** 推理完成后，由调用方通过 `rknn3_destroy_mem` 释放各KV cache内存，再调用 `rknn3_destroy` 释放上下文。

4.4.4 Page KV cache内存

分页KV cache适用于模型声明了 page cache 类型的LLM模型。它与传统的连续KV cache不同：运行时不再接收每个核心的一整块KV cache，而是接收按页分配的 `rknn3_tensor_mem` 数组，并按请求和cache类型建立页表绑定。完整流程如下：

1. **完成模型初始化：** 依次调用 `rknn3_init`、模型加载接口和 `rknn3_model_init`。page KV cache只能用于分页模型；非分页模型的 `rknn3_kvcache_page_info.n_cache_types` 为0，应使用

4.4.3 KV cache内存 中的 `rknn3_set_kvcache_mem` 流程。

2. **查询页规格：** 对每个参与运行的物理核心，将 `rknn3_kvcache_page_info.core_id` 设置为核心ID，然后调用 `rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO, &info, sizeof(info))`。`info.caches[]` 给出每种cache的 `cache_type`、每页字节数 `page_size` 和编译期最大页数 `max_page_num`；`max_request_num` 给出需要准备页表的最大并发请求数。
3. **查询分页配置并计算页数：** 通过 `RKNN3_QUERY_LLM_CONFIG` 和 `RKNN3_QUERY_ATTENTION_CONFIG` 查询 `max_ctx_len`、`tokens_per_page` 等配置。对于 `RKNN3_PAGE_CACHE_FULL_ATTENTION`，按 `ceil(max_ctx_len / tokens_per_page)` 分配页，不能超过 `max_page_num`；其他cache类型按 `max_page_num` 分配完整容量。
4. **创建每个page：** 对每个核心、请求和cache类型，调用 `rknn3_create_mem(ctx, page_size, core_id, RKNN3_FLAG_MEMORY_CACHEABLE)` 创建独立页。需要先由CPU清零或写入页内容时，调用 `rknn3_mem_sync(ctx, page, RKNN3_MEMORY_SYNC_TO_DEVICE)`。
5. **设置page KV cache：** 为每个 `(core_id, cache_type, request_index)` 创建一个 `rknn3_kvcache_page_binding`，令 `n_pages` 为页数、`pages` 指向 `rknn3_tensor_mem*` 数组，然后调用 `rknn3_set_kvcache_pages(ctx, bindings, n_bindings)`。调用时context必须处于idle状态；该接口只替换列出的绑定，未列出的绑定保持不变。
6. **解绑并销毁：** 推理完成或需要重建页表时，为每个已绑定键调用 `rknn3_set_kvcache_pages`，传入 `n_pages = 0` 和 `pages = NULL` 解除绑定；确认解绑成功后，对每个页调用 `rknn3_destroy_mem`。不要在仍被页表引用时销毁页内存。

示例： 下面示例展示一组已查询到的页规格的创建、设置和销毁。实际应用需要按每个核心、请求和cache类型重复该过程，并保存页对象以保证绑定期间其生命周期有效。

```
// page_attr 来自 RKNN3_QUERY_KVCACHE_PAGE_INFO，page_count 按 max_ctx_len
// 和 tokens_per_page 计算，且不能超过 page_attr->max_page_num。
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* pages[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    pages[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                RKNN3_FLAG_MEMORY_CACHEABLE);
    if (!pages[i]) {
        // 释放已经创建的页；尚未成功绑定，因此不需要解绑
        for (uint32_t j = 0; j < i; ++j)
            rknn3_destroy_mem(ctx, pages[j]);
        return RKNN3_ERR_OUT_OF_MEMORY;
    }
    memset(pages[i]->virt_addr, 0, pages[i]->size);
    rknn3_mem_sync(ctx, pages[i], RKNN3_MEMORY_SYNC_TO_DEVICE);
}

rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)pages, {0}
};
int ret = rknn3_set_kvcache_pages(ctx, &binding, 1);
if (ret != RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
    return ret;
}

// 推理完成：先解绑，再销毁page内存。
rknn3_kvcache_page_binding unbind = {
    core_id, page_attr->cache_type, request_index, 0, NULL, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &unbind, 1);
if (ret == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, pages[i]);
}
```

> **注意：** `rknn3_set_kvcache_pages` 成功绑定后，页内存仍归调用方所有，但调用方必须一直持有到解绑。`n_pages = 0` 只解除一个指定键；一次调用可以提交多个binding。校验失败时旧绑定保持不变；页表写入失败时本批次键会被解除，调用方应根据返回值处理本批次页的生命周期。分页模型不能同时使用 `rknn3_set_kvcache_mem`。

4.5 MatMul矩阵乘法

MatMul接口用于直接执行矩阵乘法，无需加载RKNN模型或创建Session。数学上，A为M×K矩阵，B为K×N矩阵，输出C为M×N矩阵。接口的连续逻辑存储顺序见 6.11 MatMul类型。

1. 调用 `rknn3_init` 创建父上下文。
2. 填写一个或多个 `rknn3_matmul_desc`，调用 `rknn3_matmul_init` 注册形状。
3. 调用 `rknn3_matmul_query_infos` 查询各形状A/B/C的大小、布局、对齐、核心及量化要求；使用 `rknn3_create_mem` 分配对应的NPU内存。
4. 调用 `rknn3_matmul_build` 完成构建，再调用 `rknn3_matmul_set_shape` 选择shape。

5. 调用 `rknn3_matmul_transpose` 将逻辑A/B转换到查询得到的NPU布局，再分别调用 `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_TO_DEVICE)`。
6. 对需要量化参数的类型，调用 `rknn3_matmul_set_quant` 提交参数。此步骤与B的转换先后顺序独立，均需在运行前完成。
7. 调用 `rknn3_matmul_run(matmul, a_mem, b_mem, c_mem)`，检查返回值。
8. 成功后，CPU读取C之前调用 `rknn3_mem_sync(..., RKNN3_MEMORY_SYNC_FROM_DEVICE)`。需要连续逻辑输出时，再调用 `rknn3_matmul_transpose` 将C转回NCHW。
9. 多shape使用时，按下述规则注册和切换形状、准备量化参数及A/B/C；每次执行均需传入A/B/C。
10. 先成功调用 `rknn3_matmul_destroy`，再通过 `rknn3_destroy_mem` 释放用户分配的A/B/C，最后销毁父上下文。

同一句柄上的形状选择、转换、量化、执行和输出转换应由应用程序串行组织。`run` 为同步接口；成功返回表示本次NPU执行完成。失败时不能使用C作为有效结果。若返回 `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN`，应保留句柄、父上下文及A/B/C，禁止修改、复用或释放这些缓冲区；MatMul句柄销毁成功后才可释放相关资源。

4.5.1 多shape注册与适用情况

多shape表示在同一个MatMul句柄中注册多个矩阵形状，通过 `rknn3_matmul_set_shape` 选择本次执行的形状。初始化时传入 `descs` 数组，查询得到的 `infos[i]` 与 `descs[i]` 一一对应，`shape_id` 为数组索引。初始化、查询和构建各完成一次，之后按需切换已注册的形状。

同一句柄内只有M和A_batch可以不同；type、core_mask、B_batch、K、N、group_size、B_layout及查询得到的B大小和布局必须一致。多shape使用普通权重B，即 `desc.B_layout=NCHW`，查询的NPU B布局为O1I1HWI2O2；NC1HWC2的feature B仅支持单shape W16A16。

典型场景是共享固定B，A/C随shape变化。例如K/N保持不变，M取不同值：B转换、同步一次供多个shape共享，每次切换后按当前shape准备A，并使用相应的C属性接收结果。A/C缓冲区可复用，不要求每次切换都重新分配。

4.5.2 多shape切换流程

切换形状无需重新初始化或构建，按以下顺序执行：

1. 完成本次执行，以及需要的C同步、输出转换或结果保存，再调用 `rknn3_matmul_set_shape(matmul, shape_id)`。
2. 使用 `infos[shape_id].A/B/C` 作为本次NPU属性。A/C缓冲区可按所有形状的最大需求预先分配，但必须满足当前形状的容量、布局、步长、对齐和核心要求；转换时仍需使用当前形状的属性。
3. 按新形状准备A。即使复用同一块内存，原形状的A数据也不能仅因缓冲区足够大而直接作为新形状的输入；应确保数据已按新形状及其布局组织。
4. 复用已转换并同步到设备的固定B，无需重新转换或同步B。
5. 确保当前形状所需量化值已提交，再执行 `run`，成功后按当前形状处理C。

首次选定shape后，将B转换、同步到设备一次。之后按以下顺序切换shape并执行：

```
set_shape(shape_id) -> 使用infos[shape_id]准备A/C
-> 首次使用该shape时按类型提交所需量化值
-> 转换、同步A -> run(A, 固定B, C)
-> 同步C到CPU -> 按当前shape按需转换或保存C
-> 切换下一个shape
```

每次 `run` 都必须传入A/B/C，B不能传NULL。CPU读取本次C之前需同步C；复用同一C缓冲区会覆盖之前的结果，应先完成所需的读取、转换或保存。如果输入已按查询的NPU布局准备好，可省略对应的逻辑转换；CPU写入后仍需完成缓存同步。

`set_shape` 只能选择初始化时注册的形状。需要使用未注册的形状、另一种计算类型或不同K/N等配置时，应另建MatMul句柄。

4.5.3 多shape的量化参数

切换shape时按计算类型准备量化参数，不能在每次运行前无条件重复调用 `set_quant`。

计算类型	首次提交	切换shape时的处理
W16A16 / W8A8O32	不调用 `set_quant`。	不调用 `set_quant`。
W4A16 / W6A16 / W8A16	构建并选择形状后，仅提交B量化值一次。	整个句柄复用已提交的B量化值，其他已注册形状无需再次提交。
W8A8 per-channel / per-layer	每个形状在首次运行前分别提交A/B/C量化值。	已提交的形状无需再次提交；首次切到尚未提交的形状时必须先提交。不同形状的B量化值保持一致，A/C可不同。

可以在执行循环开始前依次选择各形状并完成量化提交，也可以在首次使用对应形状时提交。切回已提交的形状会复用原有量化值。

当前接口不支持覆盖成功提交的量化值，重复提交会返回 `RKNN3_ERR_MATMUL_INVALID_STATE`；需要更换scale或zero point时，应另建MatMul句柄。

5. 函数参考

API 函数

RKNN3 C API 按功能归类如下：

分类	说明
5.1. 设备与上下文	设备发现、运行时上下文的初始化与销毁。
5.2. 模型加载与初始化	模型文件加载、模型初始化及流式权重加载。
5.3. 查询	查询模型属性、SDK版本、内存分配等信息。
5.4. 推理执行	同步/异步推理执行及动态形状设置。
5.5. 张量和内存	tensor 内存分配、释放、同步及用户自定义内存设置。
5.6. 模型解密	模型解密密钥设置，须在模型加载之前调用。
5.7. LLM Session	LLM 会话的创建、推理、暂停/恢复、销毁等生命周期管理。
5.8. LLM配置与Callback	LLM 会话的参数配置及回调函数类型定义。
5.9. KVCache	KV 缓存策略设置、清除、加载与保存。
5.10. LoRA	LoRA 的初始化、加载、启用与禁用。
5.11. 性能分析与调试	按层 dump 中间结果、算子性能分析、内存使用分析。
5.12. 自定义算子	自定义算子插件注册。
5.13. 图像处理	图像内存创建、销毁及颜色空间转换。
5.14. 辅助函数	提供 tensor 类型、量化类型和布局类型的字符串转换。
5.15. 结构化输出(grammar)	基于xgrammar的JSON Schema/EBNF/正则等结构化输出约束。
5.16. MatMul矩阵乘法	矩阵乘法的形状注册、查询、构建、转换、量化和执行。

5.1. 设备与上下文

设备发现、运行时上下文的初始化与销毁。

rknn3_init

该接口用于初始化RKNN3运行时上下文，必须在加载和运行模型前调用。context为输出参数，init_extend可用于指定设备ID等扩展信息。初始化完成后，需在不再使用时调用rknn3_destroy释放资源。多次初始化需确保资源正确释放，避免内存泄漏。

API	rknn3_init
功能	初始化一个新的RKNN3运行时上下文，该上下文用于在Rockchip NPU硬件上运行RKNN3模型。
参数	rknn3_context* context：指向将被初始化的RKNN3上下文句柄的指针
	rknn3_init_extend* init_extend：指向设备特定初始化信息的指针
返回值	int 状态码：见 6.2 状态码
注意	使用完成后，请调用 rknn3_destroy(context) 释放上下文资源

示例：

```
rknn3_context ctx = 0;
rknn3_init_extend extend = {0};
// extend.device_id = "xxx";  // 多设备时指定，单设备可设为NULL
int ret = rknn3_init(&ctx, &extend);
if (ret < 0) { /* 错误处理 */ }
```

rknn3_destroy

该接口用于销毁RKNN3运行时上下文并释放相关资源。调用后context句柄失效，不可再用于其他操作。多次销毁同一context会导致未定义行为，需确保每个context只销毁一次。

API	rknn3_destroy
功能	销毁RKNN3运行时上下文并释放资源。
参数	rknn3_context context：要销毁的RKNN3上下文句柄
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_destroy(ctx);
ctx = 0;
```

rknn3_find_devices

该接口用于获取当前系统可用的RKNN3设备列表。pdevs为设备信息结构体指针。调用前需确保pdevs有效。返回0表示无设备或未实现。

API	rknn3_find_devices
功能	获取可用的RKNN3设备列表。
参数	rknn3_devices* pdevs：将接收设备列表的结构的指针
返回值	int 状态码：见 6.2 状态码
注意	此函数用系统上所有可用的RKNN3设备的信息填充提供的rknn3_devices结构

示例：

```
rknn3_devices devs;
int ret = rknn3_find_devices(&devs);
if (ret == 0 && devs.n_devices > 0) {
    printf("found %u devices, device[0]: id=%s\n", devs.n_devices,
devs.devices[0].id);
}
```

rknn3_set_input_name_alias

该接口用于设置模型输入名称的别名，以适配Session内置的输入名称。

API	rknn3_set_input_name_alias
功能	设置模型输入名称到Session输入名称的别名映射。

API	rknn3_set_input_name_alias
参数	rknn3_context context：通过 rknn3_init 获取的 RKNN3 上下文句柄
	const char* src_input_name：模型中的实际输入名称（如 "input_embedding"）
	const char* dst_input_name：Session内置的输入名称（如 "input_embeds"）
返回值	int 状态码：见 6.2 状态码
注意	当模型输入名称与Session内置的输入名称不一致时，调用本接口进行映射。支持重复调用以更新映射关系。例如：模型输入名为 "input_embedding"，而Session内置的是 "input_embeds"，可调用本接口进行别名设置。

示例：

```
// 将模型输入名映射到 session 期望的输入名
rknn3_set_input_name_alias(ctx, "input_embedding", "input_embeds");
```

5.2. 模型加载与初始化

模型文件加载（Host端路径/内存/设备端文件系统）、模型初始化及流式权重加载。

rknn3_load_model_from_path

该接口用于从指定的模型文件路径和权重文件路径加载RKNN3模型到已初始化的上下文中。model_path和weight_path分别为模型和权重文件的路径，weight_path可为NULL。当weight_path为NULL时，进入流式权重加载模式，权重数据需在rknn3_model_init之后通过rknn3_load_weight_chunk接口分段上传。加载前需确保context已初始化，路径有效。加载失败时返回错误码，成功后可进行模型初始化和推理。

API	rknn3_load_model_from_path
功能	从文件路径加载RKNN3模型到指定的上下文中。
参数	rknn3_context context：RKNN3上下文句柄
	const char* model_path：RKNN3模型文件的路径
	const char* weight_path：RKNN3权重文件的路径，传NULL进入流式权重加载模式
返回值	int 状态码：见 6.2 状态码
注意	当weight_path为NULL时，进入流式权重加载模式，需在rknn3_model_init之后调用rknn3_load_weight_chunk分段上传权重数据

示例：

```
// 普通模式：指定权重文件路径
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", "/path/to/model.weight");

// 流式权重加载模式：weight_path 传 NULL
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", NULL);
```

rknn3_load_model_from_data

该接口用于从内存数据加载RKNN3模型，适用于模型和权重已在内存中的场景。model_data和weight_data分别为模型和权重的内存指针，model_size和weight_size为对应数据大小。当weight_data为NULL或

weight_size为0时，进入流式权重加载模式，权重数据需在rknn3_model_init之后通过rknn3_load_weight_chunk接口分段上传。调用前需确保context已初始化，数据指针和大小有效。加载失败返回错误码，成功后可进行模型初始化和推理。

API	rknn3_load_model_from_data
功能	从内存数据加载RKNN3模型。
参数	rknn3_context context：RKNN3上下文句柄
	const void* model_data：指向内存中模型数据的指针
	uint64_t model_size：模型数据的大小（字节）
	const void* weight_data：指向内存中权重数据的指针，传NULL进入流式权重加载模式
	uint64_t weight_size：权重数据的大小（字节），传0进入流式权重加载模式
返回值	int 状态码：见 6.2 状态码
注意	当weight_data为NULL或weight_size为0时，进入流式权重加载模式，需在rknn3_model_init之后调用rknn3_load_weight_chunk分段上传权重数据

示例：

```
// 从内存加载模型和权重
rknn3_load_model_from_data(ctx, model_data, model_size, weight_data, weight_size);

// 流式模式：weight_data 传 NULL，weight_size 传 0
rknn3_load_model_from_data(ctx, model_data, model_size, NULL, 0);
```

rknn3_load_model_from_device

该接口用于从设备端文件系统（emmc）加载RKNN3模型。与 `rknn3_load_model_from_path` 不同，`model_path` 和 `weight_path` 指向的是设备（如协处理器侧）文件系统上的文件，而非主控Host侧的文件。调用后Host将路径下发至设备，设备端在本地读取并解析模型文件。

API	rknn3_load_model_from_device
功能	从设备端文件系统（emmc）加载RKNN3模型到指定的上下文中。
参数	rknn3_context context：RKNN3上下文句柄
	const char* model_path：设备端文件系统上RKNN3模型文件的路径
	const char* weight_path：设备端文件系统上RKNN3权重文件的路径，传NULL进入流式权重加载模式
返回值	int 状态码：见 6.2 状态码
注意	`model_path` / `weight_path` 指设备端文件系统路径；weight_path传NULL时进入流式权重加载模式

示例：

```
// 从设备端文件系统加载模型和权重
rknn3_load_model_from_device(ctx, "/data/model.rknn", "/data/model.weight");

// 流式权重加载模式：weight_path 传 NULL
rknn3_load_model_from_device(ctx, "/data/model.rknn", NULL);
```

rknn3_model_init

该接口用于初始化已加载模型的运行配置，包括优先级、超时时间、核心掩码等。config为配置参数指针。调用前需确保模型已加载，config参数有效。初始化失败返回错误码，成功后模型可用于推理。

在流式权重加载模式下（即加载模型时weight参数为NULL），rknn3_model_init仅完成模型配置等，权重数据不会在此阶段同步。函数返回后，调用方需通过rknn3_load_weight_chunk接口分段上传权重数据。

API	rknn3_model_init
功能	初始化RKNN3模型。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_config* config：模型配置参数
返回值	int 状态码：见 6.2 状态码
注意	在流式权重加载模式下，此接口不传输权重数据，需在调用后通过rknn3_load_weight_chunk分段上传权重

示例：

```
rknn3_config config = {0};
config.run_core_mask = RKNN3_NPU_CORE_0;
config.run_timeout = 0;  // 0 表示无超时
int ret = rknn3_model_init(ctx, &config);
```

rknn3_load_weight_chunk

该接口用于在流式权重加载模式下，将权重数据分段上传到NPU设备。调用方可以按任意块大小逐段写入权重数据，无需在Host端维护完整的权重内存副本。接口内部根据各NPU核心的权重内存分配信息自动将数据拷贝到正确地址。

前置条件：必须在rknn3_model_init成功返回后调用（即流式权重加载已准备就绪）。当累计上传的字节数达到权重总大小时，流式状态自动完成，后续可正常进行推理。

API	rknn3_load_weight_chunk
功能	在流式权重加载模式下，将一块权重数据上传到NPU设备。
参数	rknn3_context context：RKNN3上下文句柄
	uint64_t weight_offset：权重数据在权重文件中的全局字节偏移
	const void* data：指向权重数据块的指针
	uint64_t size：数据块的大小（字节）
返回值	int 状态码：见 6.2 状态码
注意	必须在rknn3_model_init之后调用；weight_offset为相对于权重文件起始位置的全局偏移；当累计上传字节数达到权重总大小时，流式状态自动完成

示例：

```
// 流式权重加载：循环读取权重文件分段上传
FILE* fp = fopen("model.weight", "rb");
uint64_t offset = 0;
char buf[CHUNK_SIZE];
size_t n;
while ((n = fread(buf, 1, sizeof(buf), fp)) > 0) {
    rknn3_load_weight_chunk(ctx, offset, buf, n);
    offset += n;
}
fclose(fp);
```

5.3. 查询

查询模型属性、SDK版本、内存分配等信息。

rknn3_query

该接口用于查询模型和运行时的各种信息，如输入输出属性、SDK版本等。cmd指定查询类型，详见 rknn3_query_cmd，info为结果缓冲区指针，size为缓冲区大小。调用前需确保context和info有效，size与查询类型对应结构体大小一致。

API	rknn3_query
功能	查询RKNN3信息或状态。
参数	rknn3_context context：RKNN3模型的上下文
	rknn3_query_cmd cmd：查询命令类型
	void* info：用于存储查询结果的缓冲区指针
	uint64_t size：info缓冲区的大小（字节）
返回值	int 状态码：见 6.2 状态码
注意	此函数用于查询有关RKNN3模型和运行时的各种信息，如SDK版本、设备信息、模型信息等。返回的具体信息取决于指定的查询命令

示例：

```
// 查询输入输出数量
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// 查询第 0 个输入 tensor 属性
rknn3_tensor_attr attr;

memset(&attr, 0x00, sizeof(attr));
attr.index = 0;
rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, &attr, sizeof(attr));
```

5.4. 推理执行

同步/异步推理执行及动态形状设置。

rknn3_run

该接口用于执行同步推理，inputs和outputs分别为输入输出tensor数组，n_inputs和n_outputs为数量。调用前需确保context、inputs、outputs均已正确分配和配置。推理过程中会阻塞直到完成，返回值为状态码。输入输出tensor的内存需由用户提前分配。

API	**rknn3_run**
功能	执行RKNN3模型推理。
参数	rknn3_context context：从rknn3_init获得的RKNN3上下文句柄
	const rknn3_tensor inputs[]：包含输入数据的输入 tensor 数组
	uint32_t n_inputs：输入 tensor 的数量
	rknn3_tensor outputs[]：用于存储推理结果的输出 tensor 数组
	uint32_t n_outputs：输出 tensor 的数量
返回值	int 状态码：见 6.2 状态码
注意	此函数使用指定的RKNN3模型执行推理。它通过inputs数组接收输入数据，并将结果写入outputs数组。在调用此函数之前，必须正确分配和配置输入和输出 tensor

示例：

```
// 完整推理流程：查询属性 -> 分配内存 -> 填充输入 -> 同步 -> 推理 -> 同步 -> 读取输出

// 1. 查询输入输出数量
rknn3_input_output_num io_num;
rknn3_query(ctx, RKNN3_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));

// 2. 查询各输入/输出 tensor 属性，按 aligned_size 和 core_id 分配内存
rknn3_tensor inputs[io_num.n_input];
rknn3_tensor outputs[io_num.n_output];
for (uint32_t i = 0; i < io_num.n_input; i++) {
    inputs[i].attr = malloc(sizeof(rknn3_tensor_attr));
    inputs[i].attr->index = i;
    rknn3_query(ctx, RKNN3_QUERY_INPUT_ATTR, inputs[i].attr,
sizeof(rknn3_tensor_attr));
    inputs[i].mem = rknn3_create_mem(ctx, inputs[i].attr->aligned_size,
                                     inputs[i].attr->core_id,
RKNN3_FLAG_MEMORY_CACHEABLE);
}
// outputs 同理 ...

// 3. 填充输入数据，同步到设备
memcpy(inputs[0].mem->virt_addr, input_data, inputs[0].attr->aligned_size);
rknn3_mem_sync(ctx, inputs[0].mem, RKNN3_MEMORY_SYNC_TO_DEVICE);

// 4. 执行推理
rknn3_run(ctx, inputs, io_num.n_input, outputs, io_num.n_output);

// 5. 同步输出到 CPU，读取结果
rknn3_mem_sync(ctx, outputs[0].mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
// 读取 outputs[0].mem->virt_addr 中的推理结果 ...

// 6. 释放内存
for (uint32_t i = 0; i < io_num.n_input; i++) {
    rknn3_destroy_mem(ctx, inputs[i].mem);
    free(inputs[i].attr);
}
// outputs 同理 ...
```

rknn3_set_shape

该接口用于为动态输入模型设置当前形状。shape_id为模型中预定义的形状ID。调用前需确保context有效，shape_id合法。仅动态输入模型需调用，静态模型无需设置。

API	rknn3_set_shape
功能	为动态输入设置模型形状。
参数	rknn3_context context：RKNN3模型的上下文句柄
	int32_t shape_id：要设置的形状的ID。
返回值	int 状态码：见 6.2 状态码

示例：

```
// 动态形状模型：设置当前使用的形状 ID
rknn3_set_shape(ctx, shape_id);
```

5.5. 张量和内存

tensor 内存分配、释放、同步及用户自定义内存设置。

rknn3_create_mem_from_fd

该接口允许用户将已分配的内存（通过文件描述符表示）与 RKNN3 推理框架关联，从文件描述符创建 tensor 内存对象，从而实现高效的内存复用和零拷贝数据传输。调用前需确保context有效，相关参数（如fd、大小等）正确。分配的内存需在不再使用时通过rknn3_destroy_mem释放，避免内存泄漏。

API	**rknn3_create_mem_from_fd**
功能	从文件描述符创建 tensor 内存对象。
参数	rknn3_context context：RKNN3上下文句柄
	int32_t fd：内存的文件描述符
	void* virt_addr：内存的虚拟地址
	uint64_t size：内存的大小（字节）
	uint64_t offset：从fd引用的内存开始的偏移量
返回值	rknn3_tensor_mem* 指向创建的 tensor 内存对象的指针，如果创建失败则为NULL

示例：

```
// 从已有的 fd 创建 tensor 内存（零拷贝场景）
rknn3_tensor_mem* mem = rknn3_create_mem_from_fd(ctx, fd, virt_addr, size, offset);
```

rknn3_create_mem

该接口用于为tensor分配或注册内存。调用前需确保context有效，输入和输出tensor的core_id设置成rknn3_query接口获取的rknn3_tensor_attr结构体的core_id成员。分配的内存需在不再使用时通过rknn3_destroy_mem释放，避免内存泄漏。

API	**rknn3_create_mem**
功能	为RKNN3 tensor 创建内存缓冲区。
参数	rknn3_context context：RKNN3上下文句柄
	uint64_t size：要分配的内存大小（字节）
	int32_t core_id：内存分配的目标NPU核心ID
	rknn3_mem_alloc_flags flags：控制分配行为的内存分配标志
返回值	rknn3_tensor_mem* 指向分配的 tensor 内存的指针，如果分配失败则为NULL
注意	此函数分配可用于RKNN3 tensor 操作的内存。内存在指定的核心上分配，具有给定的标志

示例：

```
// 在 NPU 核心 0 上分配 1MB cacheable 内存
rknn3_tensor_mem* mem = rknn3_create_mem(ctx, 1024 * 1024, 0,
RKNN3_FLAG_MEMORY_CACHEABLE);
```

rknn3_destroy_mem

该接口用于释放通过RKNN3 API分配或注册的tensor内存。调用前需确保context和mem有效。重复释放同一内存会导致未定义行为。

API	rknn3_destroy_mem
功能	销毁为RKNN3 tensor 分配的内存。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_tensor_mem* mem：指向要销毁的 tensor 内存结构的指针
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_destroy_mem(ctx, mem);
mem = NULL;
```

rknn3_mem_sync

该接口用于在CPU和设备间同步内存数据。mode指定同步方向，常用于推理前后确保数据一致性。调用前需确保context和mem有效，mode设置正确。未同步可能导致推理结果异常。

API	rknn3_mem_sync
功能	在CPU和设备之间同步内存数据。
参数	rknn3_context context：RKNN3模型的上下文
	rknn3_tensor_mem* mem： tensor 的内存句柄
	rknn3_mem_sync_mode mode：表示刷新CPU cache和DDR数据的模式。 **RKNN3_MEMORY_SYNC_TO_DEVICE**：表示CPU cache数据同步到DDR中，通常用于CPU写入内存后，NPU访问相同内存前使用该模式将cache中的数据写回DDR。 **RKNN3_MEMORY_SYNC_FROM_DEVICE**：表示DDR数据同步到CPU cache，通常用于NPU写入内存后，使用该模式让下次CPU访问相同内存时，cache数据无效，CPU从DDR重新读取数据。采用FROM_DEVICE模式时，数据按块传输，块大小可通过环境变量 `MEM_SYNC_CHUNK_SIZE`（单位：字节，默认2MiB）配置。 **RKNN3_MEMORY_SYNC_BIDIRECTIONAL**：表示CPU cache数据同步到DDR同时令CPU重新从DDR读取数据。
返回值	int 状态码：见 6.2 状态码

示例：

```
// CPU 写入后，同步到设备（NPU 读取前）
rknn3_mem_sync(ctx, input_mem, RKNN3_MEMORY_SYNC_TO_DEVICE);

// NPU 写入后，同步到 CPU（CPU 读取前）
rknn3_mem_sync(ctx, output_mem, RKNN3_MEMORY_SYNC_FROM_DEVICE);
```

rknn3_mem_sync_range

该接口用于在CPU和设备间同步指定范围的内存数据。与 `rknn3_mem_sync` 同步整块内存不同，本接口仅同步 `[offset, offset+length)` 区间的数据，适用于只需更新部分内存的场景，可减少同步开销。`offset` 为相对于 `mem->virt_addr` 的字节偏移，`length` 为同步的字节数，要求 `offset + length` 不超过 `mem->size`。

API	rknn3_mem_sync_range
功能	在CPU和设备之间同步指定范围的内存数据。
参数	rknn3_context context：RKNN3模型的上下文
	rknn3_tensor_mem* mem： tensor 的内存句柄
	rknn3_mem_sync_mode mode：同步模式，取值同 rknn3_mem_sync 中的 mode 参数
	uint64_t offset：相对于 `mem->virt_addr` 的起始字节偏移
	uint64_t length：同步的字节数（`offset + length` 不得超过 `mem->size`）
返回值	int 状态码：0表示成功，负值表示错误，见 6.2 状态码
注意	`offset + length` 超过 `mem->size`、`length` 为 0 或 `offset` 大于等于 `mem->size` 时返回错误码

示例：

```
// 仅同步 [offset, offset+length) 区间的数据到设备
rknn3_mem_sync_range(ctx, mem, RKNN3_MEMORY_SYNC_TO_DEVICE, 1024, 4096);

// 仅从设备同步 4096 字节到 CPU
rknn3_mem_sync_range(ctx, mem, RKNN3_MEMORY_SYNC_FROM_DEVICE, 0, 4096);
```

rknn3_set_kvcache_pages

为分页模型设置KV cache页表绑定。每个binding由物理核心、cache类型、请求索引和一组page内存组成；接口只设置调用参数中列出的binding，未列出的binding保持原状。调用前必须通过 `RKNN3_QUERY_KVCACHE_PAGE_INFO` 获取页大小和cache类型，并通过 `rknn3_create_mem` 为每一页分配 `page_size` 字节的内存。

API	rknn3_set_kvcache_pages
功能	为分页KV cache设置或更新页表绑定。
参数	rknn3_context context：RKNN3上下文句柄
	const rknn3_kvcache_page_binding* bindings：页表绑定数组
	uint32_t n_bindings：绑定数量，必须大于0
返回值	int 状态码：见 6.2 状态码
注意	仅适用于page KV cache模型；调用时context必须处于idle状态。`n_pages = 0` 且 `pages = NULL` 表示解绑一个 `(core_id, cache_type, request_index)`。接口不拥有page内存，成功绑定后页必须保持有效直到解绑。

调用示例：

```
// 先查询指定核心的page规格；page_attr 为 info.caches[type]。
rknn3_kvcache_page_info info = {0};
info.core_id = core_id;
int ret = rknn3_query(ctx, RKNN3_QUERY_KVCACHE_PAGE_INFO,
                      &info, sizeof(info));
if (ret != RKNN3_SUCCESS || type >= info.n_cache_types)
    return RKNN3_ERR_ARGUMENT_INVALID;
rknn3_cache_page_attr* page_attr = &info.caches[type];

// page_count 应按模型配置计算，且不超过 page_attr->max_page_num。
uint32_t page_count = (max_ctx_len + tokens_per_page - 1) / tokens_per_page;
rknn3_tensor_mem* page_mems[page_count];
for (uint32_t i = 0; i < page_count; ++i) {
    page_mems[i] = rknn3_create_mem(ctx, page_attr->page_size, core_id,
                                    RKNN3_FLAG_MEMORY_CACHEABLE);
}

rknn3_kvcache_page_binding binding = {
    core_id, page_attr->cache_type, request_index,
    page_count, (const rknn3_tensor_mem* const*)page_mems, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &binding, 1);

// 不再使用该请求的page KV cache时，先解绑，再销毁page内存。
rknn3_kvcache_page_binding unbind = {
    core_id, page_attr->cache_type, request_index, 0, NULL, {0}
};
ret = rknn3_set_kvcache_pages(ctx, &unbind, 1);
if (ret == RKNN3_SUCCESS) {
    for (uint32_t i = 0; i < page_count; ++i)
        rknn3_destroy_mem(ctx, page_mems[i]);
}
```

> **错误处理：** 参数校验失败时旧绑定保持不变；页表写入失败时本批次的键会被解除。调用方应保存所有页对象，并仅在确认对应页不再被页表引用后销毁。分页模型不要再调用 `rknn3_set_kvcache_mem` 设置连续KV cache。

rknn3_set_kvcache_mem

该接口用于为指定NPU核心设置KV cache内存。mem为KV cache内存数组，npu_core_indices为核心索引数组，n_core为核心数量。调用前需确保context、mem、npu_core_indices有效，n_core大于0。mem和npu_core_indices需一一对应。

API	**rknn3_set_kvcache_mem**
功能	设置KV cache内存。
参数	rknn3_context context：RKNN3模型的上下文句柄
	rknn3_tensor_mem* mem[]：KV cache内存相关信息
	int* npu_core_indices：NPU核心索引
	int n_core：要设置kvcache的NPU核心数
返回值	int 状态码：见 6.2 状态码

示例：

```
// KV cache 内存来源：通过 RKNN3_QUERY_ALLOCATION_INFO 查询各核心所需大小，
// 再用 rknn3_create_mem 分配，最后调用 rknn3_set_kvcache_mem 设置。
// 前提：已在 rknn3_model_init 之后调用。

// 1. 查询核心数量
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));

// 2. 查询各核心的内存分配信息（含 kvcache 大小）
rknn3_allocation_info alloc_infos[core_num];
rknn3_query(ctx, RKNN3_QUERY_ALLOCATION_INFO, alloc_infos,
            sizeof(rknn3_allocation_info) * core_num);

// 3. 按各核心 kvcache_mem.size 分配内存
rknn3_tensor_mem* kvcache_mems[core_num];
int core_indices[core_num];
int n = 0;
for (uint32_t i = 0; i < core_num; i++) {
    uint64_t size = alloc_infos[i].kvcache_mem.size;
    if (size == 0) continue;
    kvcache_mems[n] = rknn3_create_mem(ctx, size, alloc_infos[i].core_id,
                                       RKNN3_FLAG_MEMORY_CACHEABLE);
    core_indices[n] = alloc_infos[i].core_id;
    n++;
}

// 4. 设置 KV cache 内存
rknn3_set_kvcache_mem(ctx, kvcache_mems, core_indices, n);

// 使用完毕后释放
for (int i = 0; i < n; i++)
    rknn3_destroy_mem(ctx, kvcache_mems[i]);
```

rknn3_set_internal_mem

为多个核心设置用户分配的内部Tensor内存。每个mem的core_id字段指定目标核心。

API	rknn3_set_internal_mem
功能	设置多个核心的用户分配内部Tensor内存
参数	rknn3_context context：RKNN3上下文
	rknn3_tensor_mem* mem[]：用户分配的内存对象数组
	uint32_t n_core：核心数量
返回值	int 状态码：见 6.2 状态码

示例：

```
// 内部内存来源：通过 RKNN3_QUERY_CORE_MEM_SIZE 查询各核心所需大小，
// 再用 rknn3_create_mem 分配，最后调用 rknn3_set_internal_mem 设置。
// 注意：需在 rknn3_model_init 之前将 config.user_mem_internal 置 1。

// 1. 初始化模型时标记使用用户分配的内部内存
rknn3_config config = {0};
config.user_mem_internal = 1;
rknn3_model_init(ctx, &config);

// 2. 查询核心数量
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));

// 3. 查询各核心的内部内存大小
rknn3_core_mem_size core_mem_sizes[core_num];
rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, core_mem_sizes,
            sizeof(rknn3_core_mem_size) * core_num);

// 4. 按各核心 internal_size 分配内存
rknn3_tensor_mem* internal_mems[core_num];
for (uint32_t i = 0; i < core_num; i++) {
    if (core_mem_sizes[i].internal_size > 0) {
        internal_mems[i] = rknn3_create_mem(ctx, core_mem_sizes[i].internal_size,
                                            core_mem_sizes[i].core_id,
                                            RKNN3_FLAG_MEMORY_CACHEABLE);
    } else {
        internal_mems[i] = NULL;
    }
}

// 5. 设置用户分配的内部内存
rknn3_set_internal_mem(ctx, internal_mems, core_num);

// 使用完毕后释放
for (uint32_t i = 0; i < core_num; i++)
    if (internal_mems[i]) rknn3_destroy_mem(ctx, internal_mems[i]);
```

rknn3_set_weight_mem

为多个核心设置用户分配的权重内存。每个mem的core_id字段指定目标核心。注意：当前版本仅支持RK3572。

API	rknn3_set_weight_mem
功能	设置多个核心的用户分配权重Tensor内存。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_tensor_mem* mem[]：用户分配的内存对象数组，每个mem的core_id字段指定目标核心
	uint32_t n_core：核心数量
返回值	int 状态码：见 6.2 状态码
注意	当前版本仅支持RK3572

示例：

```
// 权重内存来源：通过 RKNN3_QUERY_CORE_MEM_SIZE 查询各核心权重大小，
// 再用 rknn3_create_mem 分配，最后调用 rknn3_set_weight_mem 设置。
// 注意：需在 rknn3_model_init 之前将 config.user_mem_weight 置 1。
//       当前版本仅支持 RK3572。

// 1. 初始化模型时标记使用用户分配的权重内存
rknn3_config config = {0};
config.user_mem_weight = 1;
rknn3_model_init(ctx, &config);

// 2. 查询核心数量和各核心权重内存大小
uint32_t core_num = 0;
rknn3_query(ctx, RKNN3_QUERY_CORE_NUMBER, &core_num, sizeof(core_num));
rknn3_core_mem_size core_mem_sizes[core_num];
rknn3_query(ctx, RKNN3_QUERY_CORE_MEM_SIZE, core_mem_sizes,
            sizeof(rknn3_core_mem_size) * core_num);

// 3. 按各核心 weight_size 分配内存
rknn3_tensor_mem* weight_mems[core_num];
for (uint32_t i = 0; i < core_num; i++) {
    weight_mems[i] = rknn3_create_mem(ctx, core_mem_sizes[i].weight_size,
                                      core_mem_sizes[i].core_id,
                                      RKNN3_FLAG_MEMORY_CACHEABLE);
}

// 4. 设置用户分配的权重内存
rknn3_set_weight_mem(ctx, weight_mems, core_num);

// 使用完毕后释放
for (uint32_t i = 0; i < core_num; i++)
    rknn3_destroy_mem(ctx, weight_mems[i]);
```

rknn3_set_kvcache_group

为当前上下文设置激活的KV缓存分组。分组对应RKNN3 Toolkit转换模型时配置的 `kvcache_buffer_len` 列表中的每一项，每个分组对应不同的KV缓存缓冲区大小，即对应不同的最大上下文长度（max context len）。切换分组等价于切换max context len。

可通过 `RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` 查询每个核心的分组数量（`n_groups`）及各分组的KV缓存大小（`kvcache_sizes`）。`group_id` 必须在 `[0, n_groups)` 范围内。调用后，内部KV缓存缓冲区地址会更新为对应分组的地址，并切换当前上下文的执行ID。

API	**rknn3_set_kvcache_group**
功能	为当前上下文设置激活的KV缓存分组，等价于切换max context len。
参数	rknn3_context context：RKNN3上下文句柄
	int32_t group_id：要激活的分组ID（必须在 [0, n_groups) 范围内）
返回值	int 状态码：见 6.2 状态码
注意	分组与Toolkit转换时设置的 `kvcache_buffer_len` 列表一一对应；若使用外部分配的KV缓存内存，切换分组后需确保内存大小满足目标分组要求，必要时重新分配并调用 `rknn3_set_kvcache_mem`

示例：

```
// 查询分组信息后，切换到分组 1（等价于切换 max context len）
rknn3_kvcache_len_group_info info[core_number];
rknn3_query(ctx, RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO, info,
            sizeof(rknn3_kvcache_len_group_info) * core_number);
rknn3_set_kvcache_group(ctx, 1);  // 切换到 group_id=1
```

5.6. 模型解密

RKNN SDK 支持离线加密 RKNN 模型，并在运行时进行解密。模型解密密钥设置，须在模型加载之前调用。

> 模型加密只是模型保护的一部分，无法完全杜绝被破解的风险，建议用户尽量避免模型被第三方直接接触，并在系统层面做好整体防护。

rknn3_set_decrypt_key_from_path

该接口用于从文件路径加载 RSA 加密的用户密钥信封，并关联到 context。为保护用户模型的知识产权，RKNPU3 运行时提供了模型加密与解密机制。加密方案采用 AES-256-CTR 对称加密用于加密模型文件本体，RSA-OAEP 非对称加密用于保护 AES 密钥。调用前需确保 context 已初始化，key_path 指向有效的密钥信封文件。

API	**rknn3_set_decrypt_key_from_path**
功能	从文件路径加载 RSA 加密的用户密钥信封，并关联到 context
参数	rknn3_context context：RKNN3 运行时上下文
	const char* key_path：密钥信封文件路径
返回值	int 状态码：见 6.2 状态码
注意	此接口必须在 `rknn3_load_model_from_path` / `rknn3_load_model_from_data` 之前调用

示例：

```
// 必须在 rknn3_load_model_from_path 之前调用
rknn3_set_decrypt_key_from_path(ctx, "/path/to/key_envelope.bin");
rknn3_load_model_from_path(ctx, "/path/to/model.rknn", "/path/to/model.weight");
```

rknn3_set_decrypt_key_from_data

该接口用于从内存加载 RSA 加密的用户密钥信封，并关联到 context。用于在模型数据来自内存传输的场景。调用前需确保 `context` 已初始化，`key_data` 指向有效的密钥信封内存，`key_size` 为该数据的字节长度。

API	**rknn3_set_decrypt_key_from_data**
功能	从内存加载 RSA 加密的用户密钥信封，并关联到 context
参数	rknn3_context context：RKNN3 运行时上下文
	const void* key_data：密钥信封内存地址
	uint64_t key_size：密钥信封数据长度（字节）
返回值	int 状态码：见 6.2 状态码

API	rknn3_set_decrypt_key_from_data
注意	此接口必须在 `rknn3_load_model_from_path` / `rknn3_load_model_from_data` 之前调用

示例：

```
// 从内存加载密钥信封，必须在加载模型之前调用
rknn3_set_decrypt_key_from_data(ctx, key_data, key_size);
```

5.7. LLM Session

LLM 会话的创建、推理、暂停/恢复、销毁等生命周期管理。

rknn3_session_init

该接口用于初始化新的RKNN3会话，支持LLM等高级功能。context为上下文，params为会话参数，n_params为参数数量。调用前需确保context和params有效。初始化成功返回会话句柄，失败返回NULL。会话需通过rknn3_session_destroy释放。

API	rknn3_session_init
功能	使用指定参数初始化新的RKNN3会话。
参数	rknn3_context context：用于会话的RKNN3上下文指针
	rknn3_llm_param* params：包含会话配置参数的rknn3_llm_param结构指针
	int n_params：参数数量
返回值	rknn3_session* 如果成功返回会话句柄，如果初始化失败则返回NULL

示例：

```
rknn3_llm_param params[1] = {0};
params[0].max_context_len = 512;
params[0].sampling_param.top_k = 40;
params[0].sampling_param.top_p = 0.9;
// ... 配置 vocab_info 等 ...
rknn3_session* session = rknn3_session_init(ctx, params, 1);
```

rknn3_session_destroy

该接口用于销毁RKNN3会话并释放相关资源。session为会话指针。调用后session指针失效，不可再用于其他操作。需确保每个会话只销毁一次，避免资源泄漏或重复释放。

API	rknn3_session_destroy
功能	销毁RKNN3会话并释放相关资源。
参数	rknn3_session* session：要销毁的RKNN3会话指针
返回值	int 状态码：见 6.2 状态码
注意	调用此函数后，会话指针变为无效，不应再使用

示例：

```
rknn3_session_destroy(session);
session = NULL;
```

rknn3_session_run

使用提供的输入和参数运行推理。调用前需确保session、inputs、param有效。

API	rknn3_session_run
功能	使用提供的输入和参数运行推理。
参数	rknn3_session* session：RKNN3会话句柄指针
	rknn3_llm_input inputs[]：包含输入数据的输入 tensor 数组
	uint32_t n_inputs：提供的输入 tensor 数量
	rknn3_llm_infer_param* param：推理参数配置指针
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_llm_input input = {0};
input.input_type = RKNN3_LLM_INPUT_PROMPT;
input.llm_input.prompt = "你好，请介绍一下自己";

rknn3_llm_infer_param param = {0};
param.keep_history = 1;
param.max_new_tokens = 128;

rknn3_session_run(session, &input, 1, &param);
```

rknn3_session_run_async

为大型语言模型会话异步运行推理。调用前需确保session、inputs、param有效。异步推理需结合回调或等待机制获取结果。

API	rknn3_session_run_async
功能	为大型语言模型会话异步运行推理。
参数	rknn3_session* session：RKNN3会话句柄指针
	rknn3_llm_input inputs[]：模型的输入 tensor 数组
	uint32_t n_inputs：输入 tensor 数量
	rknn3_llm_infer_param* param：推理参数配置指针
返回值	int 状态码：见 6.2 状态码
注意	此函数对给定的LLM会话执行异步推理。它允许使用提供的输入和参数对模型进行非阻塞执行

示例：

```
// 异步推理，结果通过回调获取
rknn3_session_run_async(session, &input, 1, &param);
// 结果在 result_callback 中异步返回
```

rknn3_session_stop

终止当前会话推理。调用后仅终止当前推理，后续可继续发起新推理。适用于需要中断长时间推理的场景。

API	rknn3_session_stop
功能	终止当前会话推理。
参数	rknn3_session* session：RKNN3会话指针
返回值	int 状态码：见 6.2 状态码

示例：

```
// 中断当前推理（后续可继续发起新推理）
rknn3_session_stop(session);
```

rknn3_session_pause

暂停RKNN3会话的执行。调用后当前推理暂停，可通过 `rknn3_session_resume` 恢复。

API	rknn3_session_pause
功能	暂停RKNN3会话的执行。
参数	rknn3_session* session：要暂停的RKNN3会话指针
返回值	int 状态码：0表示成功，负值表示错误，见 6.2 状态码

示例：

```
rknn3_session_pause(session);
// 推理暂停，可通过 rknn3_session_resume 恢复
```

rknn3_session_resume

恢复已暂停的RKNN3会话的执行。

API	rknn3_session_resume
功能	恢复RKNN3会话的执行。
参数	rknn3_session* session：要恢复的RKNN3会话指针
返回值	int 状态码：0表示成功，负值表示错误，见 6.2 状态码

示例：

```
rknn3_session_resume(session);
// 从暂停处恢复推理
```

rknn3_session_query_state

该接口用于查询当前会话的运行状态，包括已生成token数、KV策略等。session为会话指针，state为状态结构体指针。调用前需确保session和state有效。可用于监控推理进度和会话状态。

API	rknn3_session_query_state
功能	查询RKNN3会话的当前状态。
参数	rknn3_session* session：要查询的RKNN3会话指针
	RKLLMRunState* state：用于存储查询运行状态的指针
返回值	int 状态码：见 6.2 状态码

示例：

```
RKLLMRunState state;
rknn3_session_query_state(session, &state);
printf("total=%llu, prefill=%llu, decode=%llu\n",
       state.n_total_tokens, state.n_prefill_tokens, state.n_decode_tokens);
```

5.8. LLM配置与Callback

LLM 会话的参数配置及回调函数类型定义。

rknn3_session_set_callback

该接口用于为会话设置回调函数，支持自定义结果处理、采样、分词、嵌入、输出tensor回调等。session为会话指针，callback为回调结构体指针。调用前需确保session和callback有效。回调函数在推理过程中被触发，便于用户自定义推理流程。

API	rknn3_session_set_callback
功能	为RKNN3会话设置回调函数。
参数	rknn3_session* session：RKNN3会话实例指针
	RKLLMCallback* callback：包含回调函数的RKLLMCallback结构指针
返回值	int 状态码：见 6.2 状态码

示例：

```
RKLLMCallback callback = {0};
callback.result_callback = my_result_callback;
callback.result_userdata = tokenizer;
callback.sampling_callback = my_sampling_callback;
callback.sampling_userdata = &embed_info;
rknn3_session_set_callback(session, &callback);
```

rknn3_session_set_chat_template

该接口用于设置LLM的聊天模板，包括系统提示、前缀和后缀。session为会话句柄，system_prompt、prompt_prefix、prompt_postfix为模板内容。调用前需确保session有效，字符串内容符合模型要求。自定义模板有助于调整模型行为和对话风格。

API	rknn3_session_set_chat_template
功能	设置LLM的聊天模板，包括系统提示、前缀和后缀。
参数	rknn3_session* session：RKNN3会话句柄
	const char* system_prompt：定义语言模型上下文或行为的系统提示
	const char* prompt_prefix：聊天中用户输入前添加的前缀
	const char* prompt_postfix：聊天中用户输入后添加的后缀
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_session_set_chat_template(session,
    "You are a helpful assistant.",  // system_prompt
    "User: ",                         // prompt_prefix
    "\nAssistant: "                   // prompt_postfix
);
```

rknn3_session_set_llm_param

该接口用于为RKNN3会话设置LLM相关参数（如采样参数、词表信息等）。需确保session和params指针，以及参数数量有效。

API	rknn3_session_set_llm_param
功能	配置LLM相关参数（如采样参数、词表信息等），支持在Session初始化后更新LLM相关参数。
参数	rknn3_session* session：RKNN3会话句柄
	rknn3_llm_param* params：LLM参数数组指针（每个logits输出对应一个LLM参数）
	int n_params：LLM参数数量（等于logits输出数量）
返回值	int 状态码：见 6.2 状态码
注意	params需按输出顺序依次设置；如模型有多个logits输出，需对应配置

示例：

```
// 在 session 初始化后更新采样参数
rknn3_llm_param params[1] = {0};
params[0].sampling_param.temperature = 0.8;
params[0].sampling_param.top_k = 50;
rknn3_session_set_llm_param(session, params, 1);
```

rknn3_session_set_function_tools

该接口用于为会话设置函数工具描述（function tools），便于模型在对话中调用工具。调用前需确保session和tools字符串有效。

API	rknn3_session_set_function_tools
功能	设置LLM会话的函数工具描述。
参数	rknn3_session* session：RKNN3会话句柄

API	rknn3_session_set_function_tools
	const char* tools：函数工具描述字符串
返回值	int 状态码：见 6.2 状态码

示例：

```
const char* tools = "[{\"name\":\"get_weather\",\"description\":\"查询天气\"}]";
rknn3_session_set_function_tools(session, tools);
```

LLMResultCallback

LLM结果回调函数类型。

函数指针类型	LLMResultCallback
功能	处理LLM生成结果的回调函数
参数	void* userdata：用户自定义数据指针
	RKLLMResult* result：指向LLM结果的指针
	LLMCallState state：LLM调用状态（如完成、错误等）
返回值	int 状态码：见 6.2 状态码

示例：

```
int my_result_callback(void* userdata, RKLLMResult* result, LLMCallState state) {
    if (state == RKLLM_RUN_NORMAL) {
        for (int i = 0; i < result->num_tokens; i++)
            printf("token[%d] = %d\n", i, result->token_ids[i]);
    } else if (state == RKLLM_RUN_FINISH) {
        printf("generation finished\n");
    }
    return 0;
}
```

LLMSamplingCallback

LLM采样回调函数类型。

函数指针类型	LLMSamplingCallback
功能	处理采样logits的回调函数
参数	void* userdata：用户自定义数据指针
	float16* logits：指向logits数组的指针
	char* logits_name：logits的名称
返回值	int：返回所选token ID (>=0)表示成功，负值表示错误码，见 6.2 状态码

示例：

```
int my_sampling_callback(void* userdata, float16* logits, char* logits_name) {
    // 自定义采样：简单 argmax
    int max_id = 0;
    for (int i = 1; i < vocab_size; i++) {
        if (fp16_to_fp32(logits[i]) > fp16_to_fp32(logits[max_id]))
            max_id = i;
    }
    return max_id;  // 返回选中的 token ID
}
```

LLMGetEmbedCallback

获取LLM嵌入向量的回调函数类型。

函数指针类型	LLMGetEmbedCallback
功能	获取LLM嵌入向量的回调函数
参数	void* userdata：用户自定义数据指针
	int32_t* tokens：token ID数组
	uint64_t num_tokens：tokens数组中的token数量
	void* embed：存储嵌入输出的缓冲区指针
	uint64_t len：嵌入缓冲区的长度（字节）
返回值	int 状态码：见 6.2 状态码

示例：

```
int my_embed_callback(void* userdata, int32_t* tokens, uint64_t num_tokens,
                      void* embed, uint64_t len) {
    // 将 embedding 结果拷贝到 embed 缓冲区
    memcpy(embed, my_embed_data, len);
    return 0;
}
```

LLMTokenizerCallback

LLM分词器回调函数类型。

函数指针类型	LLMTokenizerCallback
功能	处理文本分词的回调函数
参数	void* userdata：用户自定义数据指针
	const char* text：输入文本字符串指针
	int32_t text_len：文本长度
	int32_t* tokens：token ID数组指针
	int32_t n_tokens_max：最大生成token数量
返回值	int：返回生成的token数量 (>=0)表示成功，负值表示错误码，见 6.2 状态码

示例：

```
int my_tokenizer_callback(void* userdata, const char* text, int32_t text_len,
                          int32_t* tokens, int32_t n_tokens_max) {
    // 调用自定义分词器将文本转为 token ID 数组
    int n = my_tokenize(text, text_len, tokens, n_tokens_max);
    return n;  // 返回生成的 token 数量
}
```

LLMOutputCallback

获取输出tensor的回调函数类型。

函数指针类型	LLMOutputCallback
功能	获取输出tensor的回调函数
参数	void* userdata：用户自定义数据指针
	rknn3_tensor* output_tensors：输出tensor数组指针
	uint32_t n_output_tensors：输出tensor数量
	LLMOutputCallbackState state：输出回调的状态
返回值	int：成功返回0，错误返回非零值，见 6.2 状态码

示例：

```
int my_output_callback(void* userdata, rknn3_tensor* output_tensors,
                       uint32_t n_output_tensors, LLMOutputCallbackState state) {
    if (state == RKLLM_OUTPUT_CALLBACK_DECODE_FINISHED) {
        for (uint32_t i = 0; i < n_output_tensors; i++) {
            float16* data = (float16*)output_tensors[i].mem->virt_addr;
            // 处理输出 tensor 数据 ...
        }
    }
    return 0;
}
```

LLMInputCallback

提供自定义输入tensor的回调函数类型，适用于需要对每层输入进行自定义处理的场景（例如 gemma4 的 per_layer_inputs）。调用时框架会将当前推理的输入tensor数组及上下文参数（`LLMInputCallbackParam`）传入回调，用户在回调中填充对应的 tensor 数据。

LLMInputCallbackParam 参数结构体：

字段	类型	说明
tokens	int32_t*	当前输入的token ID数组指针
num_tokens	int32_t	token数量
shape_id	int32_t	当前输入tensor的shape ID
pos	int32_t	当前输入在整个上下文中的位置
mrope_pos	int32_t	MRoPE位置
mrope_start	int32_t	MRoPE在整个上下文中的起始位置

字段	类型	说明
system_prompt_seqlen	int32_t	system prompt的序列长度
valid_system_prompt_seqlen	int32_t	system prompt的有效序列长度
max_position_embeddings	int32_t	模型支持的最大位置编码数
reserved	uint8_t[128]	保留字段，用于未来扩展

函数指针类型	LLMInputCallback
功能	为推理框架提供自定义输入tensor数据的回调函数
参数	void* userdata：用户自定义数据指针
	rknn3_tensor* input_tensors：输入tensor数组指针，由框架分配，用户在回调中填充数据
	uint32_t n_input_tensors：输入tensor数量
	LLMInputCallbackParam param：当前推理步骤的上下文参数，含token信息、位置信息等
返回值	int：成功返回0，错误返回非零值，见 6.2 状态码

示例：

```
int my_input_callback(void* userdata, rknn3_tensor* input_tensors,
                      uint32_t n_input_tensors, LLMInputCallbackParam param) {
    // 根据 param.pos 等信息填充自定义输入 tensor
    for (uint32_t i = 0; i < n_input_tensors; i++) {
        // input_tensors[i].mem->virt_addr = ...;
    }
    return 0;
}
```

5.9. KVCache

KV 缓存策略设置、清除、加载与保存。

rknn3_session_set_kvcache_policy

设置RKNN3会话的KV cache策略。适用于Transformer等需要缓存机制的模型。调用前需确保session有效，相关参数正确。KV cache操作需与推理流程配合使用，避免缓存不一致。

API	rknn3_session_set_kvcache_policy
功能	设置RKNN3会话的KV cache策略。
参数	rknn3_session* session：RKNN3会话句柄
	rknn3_kvcache_policy policy：要设置的KV cache策略
	rknn3_kvcache_policy_param* param：指定KV cache策略的参数
返回值	int 状态码：见 6.2 状态码

示例：

```
// 使用检查点策略
rknn3_kvcache_policy_param param = {0};
param.save_checkpoint.checkpoint_start_pos = 0;
param.save_checkpoint.checkpoint_interval = 128;
param.save_checkpoint.max_checkpoint_count = 10;
rknn3_session_set_kvcache_policy(session, RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT,
&param);
```

rknn3_session_clear_kvcache

根据指定策略清除RKNN3会话的KV cache。

API	rknn3_session_clear_kvcache
功能	根据指定策略清除RKNN3会话的KV cache。
参数	rknn3_session* session：RKNN3会话句柄指针
	rknn3_kvcache_clear_policy clear：清除KV cache的策略，定义如何清除缓存
	rknn3_kvcache_clear_policy_param* param：清除KV cache策略的参数。当clear为RKNN3_KVCACHE_KEEP_N_TOKENS时必填
返回值	int 状态码：见 6.2 状态码

示例：

```
// 清除 KV cache 但保留 system prompt
rknn3_session_clear_kvcache(session, RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT, nullptr);

// 清除 KV cache 但保留 n_tokens
clear_param.keep_n_tokens.n_tokens = 512;
rknn3_session_clear_kvcache(session, RKNN3_KVCACHE_KEEP_N_TOKENS, &clear_param);
```

rknn3_session_load_kvcache_from_path

从指定文件路径加载KV-Cache。

API	rknn3_session_load_kvcache_from_path
功能	从指定路径加载KV-Cache文件到RKNN3会话。
参数	rknn3_session* session：RKNN3会话指针
	const char* kvcache_path：KV-Cache文件的路径
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_session_load_kvcache_from_path(session, "/path/to/kvcache.bin");
```

rknn3_session_load_kvcache_from_data

从内存数据加载KV-Cache。

API	rknn3_session_load_kvcache_from_data
功能	从内存数据加载KV-Cache到RKNN3会话。
参数	rknn3_session* session：RKNN3会话指针
	const void* kvcache_data：KV-Cache二进制数据
	int64_t size：KV-Cache数据长度（字节数，单位: bytes）
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_session_load_kvcache_from_data(session, kvcache_data, kvcache_size);
```

rknn3_session_save_kvcache

将KV cache保存到指定路径。

API	rknn3_session_save_kvcache
功能	将KV cache保存到指定路径。
参数	rknn3_session* session：RKNN3会话指针
	const char* kvcache_path：保存KV cache的路径
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_session_save_kvcache(session, "/path/to/kvcache.bin");
```

5.10. LoRA

LoRA 的初始化、加载、启用与禁用。

rknn3_lora_init

向RKNN3上下文注册LoRA权重文件并加载元数据。此接口需在 `rknn3_model_init` 之后、会话使用LoRA之前调用。该函数可被重复调用以注册多个权重文件，所有已注册文件中的适配器名称必须唯一。调用后可通过 `RKNN3_QUERY_LORA_NUM` 和 `RKNN3_QUERY_LORA_INFO` 查询已加载的LoRA信息，再通过 `rknn3_lora_load` 加载指定的适配器。

API	rknn3_lora_init
功能	向RKNN3上下文注册LoRA权重文件并加载元数据。可被重复调用以注册多个权重文件，所有已注册文件中的适配器名称必须唯一。
参数	rknn3_context context：RKNN3上下文句柄
	const char* lora_weight_path：LoRA权重文件路径
返回值	int 状态码：见 6.2 状态码

示例：

```
// 从文件加载 LoRA 元数据（需在 model_init 之后调用）
rknn3_lora_init(ctx, "/path/to/lora_weights.bin");

// 查询已加载的 LoRA 信息
rknn3_lora loras[RKNN3_MAX_LORA_NUM];
rknn3_query(ctx, RKNN3_QUERY_LORA_INFO, loras, sizeof(loras));
```

rknn3_lora_init_from_data

向RKNN3上下文注册内存中的LoRA权重源并加载元数据。适用于LoRA权重已在内存中的场景。该函数可被重复调用以注册多个权重缓冲区，每个缓冲区必须保持有效直到RKNN3上下文被释放。使用方式同 `rknn3_lora_init`。

API	rknn3_lora_init_from_data
功能	向RKNN3上下文注册内存中的LoRA权重源并加载元数据。可被重复调用以注册多个权重缓冲区，每个缓冲区必须保持有效直到RKNN3上下文被释放。
参数	rknn3_context context：RKNN3上下文句柄
	const void* weight_data：LoRA权重数据缓冲区指针
	uint64_t weight_size：缓冲区大小（字节）
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_lora_init_from_data(ctx, lora_data, lora_size);
```

rknn3_lora_load

将指定的LoRA加载到RKNN3上下文中。调用前需先通过 `rknn3_lora_init` 或 `rknn3_lora_init_from_data` 完成LoRA初始化，并通过 `RKNN3_QUERY_LORA_INFO` 查询获取合法的 `rknn3_lora` 描述符。

API	rknn3_lora_load
功能	将一个LoRA加载到RKNN3上下文中。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_lora* lora：要加载的LoRA描述符指针
返回值	int 状态码：见 6.2 状态码

示例：

```
// 加载指定 LoRA 到上下文（需先 lora_init 并查询获取 lora 描述符）
rknn3_lora_load(ctx, &loras[0]);
```

rknn3_lora_unload

从RKNN3上下文中卸载指定的LoRA，释放其占用的资源。卸载后该适配器不再可用于任何会话。

API	rknn3_lora_unload
功能	从RKNN3上下文中卸载一个LoRA。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_lora* lora：要卸载的LoRA描述符指针
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_lora_unload(ctx, &loras[0]);
```

rknn3_lora_enable

在上下文级别启用已加载的LoRA，使之对该上下文下的所有推理生效。若需要针对特定会话独立控制LoRA，应使用 `rknn3_session_enable_lora`。

生效规则：本接口仅在调用时更新一次LoRA配置，不会在后续每次推理时重复更新。若某个session通过 `rknn3_session_enable_lora` 启用了不同LoRA，则该session每次调用 `rknn3_session_run` 时都会覆盖本接口所做的配置。因此，不建议同时使用两种接口控制LoRA。

API	rknn3_lora_enable
功能	在RKNN3上下文级别启用一个已加载的LoRA。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_lora* lora：要启用的LoRA描述符指针
返回值	int 状态码：见 6.2 状态码
注意	映射到同一pattern槽位的不同适配器不能在上下文级执行作用域中同时启用。

示例：

```
// 上下文级别启用（影响所有推理）
rknn3_lora_enable(ctx, &loras[0]);
```

rknn3_lora_disable

在上下文级别禁用已启用的LoRA。禁用后该适配器不再对推理生效，但仍保留已加载状态，可通过 `rknn3_lora_enable` 重新启用。

生效规则：同 `rknn3_lora_enable` ，建议不要混用上下文级和会话级的接口控制LoRA，以避免生效顺序不确定的问题。

API	rknn3_lora_disable
功能	在RKNN3上下文级别禁用一个已启用的LoRA。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_lora* lora：要禁用的LoRA描述符指针
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_lora_disable(ctx, &loras[0]);
```

rknn3_session_enable_lora

为指定会话启用LoRA。LoRA权重通过 `rknn3_lora_load` 加载在上下文（context）级别，不同session共享同一份权重；本接口仅控制该session是否应用已加载的LoRA，不会重复加载权重。调用前需确保session和lora有效且lora已通过 `rknn3_lora_load` 加载。

生效规则：通过本接口启用的LoRA配置会在每次调用 `rknn3_session_run` 时重复更新，因此若同时使用了 `rknn3_lora_enable` 对不同LoRA进行上下文级启用， `rknn3_session_run` 会以session级别的配置覆盖上下文级别的配置。因此，建议不要混用两种接口对LoRA进行控制，以避免生效顺序不确定。

API	rknn3_session_enable_lora
功能	为指定RKNN3会话启用已在上下文中加载的LoRA。LoRA权重由上下文统一管理，多个session可共享同一份权重。
参数	rknn3_session* session：RKNN3会话指针
	rknn3_lora* lora：要启用的LoRA指针
返回值	int 状态码：见 6.2 状态码
注意	映射到同一pattern槽位的不同适配器不能在同一会话中同时启用。不同会话可复用该槽位，运行时会话级LoRA绑定会在推理前重新应用。

示例：

```
// 启用已加载的 LoRA（仅影响当前 session）
rknn3_session_enable_lora(session, &lora);
```

rknn3_session_disable_lora

为指定会话禁用LoRA。禁用后该session的推理不再应用LoRA，但权重仍保留在上下文中，其他session不受影响，可随时通过 `rknn3_session_enable_lora` 重新启用。调用前需确保session和lora有效。

生效规则：同 `rknn3_session_enable_lora`，建议不要混用上下文级和会话级的接口控制LoRA，以避免生效顺序不确定的问题。

API	rknn3_session_disable_lora
功能	为指定RKNN3会话禁用LoRA。该操作仅影响当前session，上下文中的LoRA权重仍保留，不影响其他session。
参数	rknn3_session* session：RKNN3会话指针
	rknn3_lora* lora：要禁用的LoRA指针
返回值	int 状态码：见 6.2 状态码

示例：

```
// 禁用当前 session 的 LoRA（权重仍保留在 context 中）
rknn3_session_disable_lora(session, &lora);
```

5.11. 性能分析与调试

按层 dump 中间结果、算子性能分析、内存使用分析。

rknn3_dump_features

按层执行模型并将每层输出的Tensor以npy格式dump到指定目录。若未提供输出Tensor会自动分配。

API	rknn3_dump_features
功能	按层运行模型并保存中间Tensor数据
参数	rknn3_context context：RKNN3上下文
	const rknn3_tensor inputs[]：输入Tensor数组
	uint32_t n_inputs：输入Tensor数量
	rknn3_tensor outputs[]：可选的输出Tensor数组，为NULL或数量为0时由内部自动分配
	uint32_t n_outputs：输出Tensor数量，可设为0以启用内部输出分配
	const char* dump_dir：保存npy文件的目录路径
	uint32_t core_mask：需要dump的NPU核心掩码，设为0时使用 `rknn3_model_init` 配置的 `run_core_mask`
	const char *dump_selector：dump模式字符串。NULL、空字符串、"" 或 "all" 表示dump所选核心上的所有算子
返回值	int 状态码：见 6.2 状态码
注意	dump目录需可写；仅执行 `core_mask` 中置位的核心进行feature dump，所选核心掩码应为模型运行核心掩码的子集

示例：

```
// dump 所有算子的中间输出到指定目录（输出 tensor 自动分配）
rknn3_dump_features(ctx, inputs, n_inputs, NULL, 0, "/tmp/dump", 0, "all");
```

rknn3_profile_ops

按层执行模型并打印算子性能信息，包括时间、Cycles和带宽。若未提供输出Tensor会自动分配。

API	rknn3_profile_ops
功能	按层运行模型并打印算子性能信息。
参数	rknn3_context context：RKNN3上下文句柄
	const rknn3_tensor inputs[]：输入Tensor数组
	uint32_t n_inputs：输入Tensor数量
	rknn3_tensor outputs[]：输出Tensor数组，可为NULL

API	rknn3_profile_ops
	uint32_t n_outputs：输出Tensor数量，可为0以启用内部输出分配
	uint32_t log_level：日志级别（0：仅汇总；1：算子明细+汇总；2：算子明细(含时间/周期/带宽)+汇总）
返回值	int 状态码：见 6.2 状态码
注意	按层profiling使用的核心来自 `rknn3_model_init` 设置的 run_core_mask

示例：

```
// 打印算子级性能信息（log_level=2 含时间/周期/带宽）
rknn3_profile_ops(ctx, inputs, n_inputs, NULL, 0, 2);
```

rknn3_profile_mem

该接口用于打印各NPU核心的内存使用信息，需在 rknn3_model_init 之后调用。

API	rknn3_profile_mem
功能	打印各NPU核心的内存使用信息。
参数	rknn3_context context：RKNN3上下文句柄
返回值	int 状态码：见 6.2 状态码
注意	建议在 rknn3_model_init 之后调用以确保内存已分配

示例：

```
// 在 model_init 之后调用，打印各核心内存使用
rknn3_profile_mem(ctx);
```

5.12. 自定义算子

自定义算子插件注册。

rknn3_register_custom_ops_plugins

注册自定义算子插件（目前支持后处理插件）。该接口加载插件并将其注册到指定上下文。

API	rknn3_register_custom_ops_plugins
功能	注册自定义算子插件。
参数	rknn3_context ctx：RKNN3上下文句柄
	const char* plugin_path：插件共享库路径
	int64_t size：保留字段，可置0
返回值	int 状态码：见 6.2 状态码

示例：

```
// 注册后处理插件共享库
rknn3_register_custom_ops_plugins(ctx, "/path/to/postprocess_plugin.so", 0);
```

rknn3_register_custom_op_func

自定义算子注册函数指针类型，用于在插件库内注册自定义算子。

函数指针类型	rknn3_register_custom_op_func
功能	根据算子索引返回自定义算子描述结构体指针
参数	int op_index：算子索引（从0开始）
返回值	rknn3_custom_op*：对应的自定义算子描述结构体指针；若无该索引则返回NULL

示例：

```
// 插件库内的导出函数，按索引返回自定义算子描述
rknn3_custom_op* my_register_func(int op_index) {
    if (op_index == 0) return &my_postprocess_op;
    return NULL;
}
```

5.13. 图像处理

图像内存创建、销毁及颜色空间转换。

rknn3_im_mem_create

该接口用于创建 RKNN3 图像内存对象 `rknn3_im_mem`，通常用于模型推理前后的图像预处理/后处理流程。

API	rknn3_im_mem_create
功能	在设备上分配一个 RKNN3 图像内存对象。
参数	rknn3_context context：RKNN3 上下文句柄
	int32_t width：图像宽度（像素）
	int32_t height：图像高度（像素）
	rknn3_im_fmt format：图像格式，见 rknn3_im_fmt
	int32_t size：图像内存大小（字节），为0时自动按宽高与格式计算
	int32_t core_id：分配目标NPU核心ID
	rknn3_mem_alloc_flags flags：内存分配标志
	rknn3_im_mem* im_mem：输出参数，接收创建好的 `rknn3_im_mem` 对象指针
返回值	int 状态码：见 6.2 状态码
注意	创建成功后，需配合 `rknn3_im_mem_destroy` 接口释放该对象

示例：

```
rknn3_im_mem im_mem;
// 分配 1920x1080 RGB888 图像内存
rknn3_im_mem_create(ctx, 1920, 1080, RKNN3_IM_FMT_RGB888, 0, 0,
                    RKNN3_FLAG_MEMORY_CACHEABLE, &im_mem);
```

rknn3_im_mem_destroy

该接口用于销毁通过 `rknn3_im_mem_create` 创建的 RKNN3 图像内存对象，并释放其占用的设备内存。

API	rknn3_im_mem_destroy
功能	销毁 RKNN3 图像内存对象并释放设备内存。
参数	rknn3_context context：RKNN3 上下文句柄
	rknn3_im_mem* im_mem：待销毁的 `rknn3_im_mem` 对象指针
返回值	int 状态码：见 6.2 状态码
注意	调用后 `im_mem` 不可再被使用

示例：

```
rknn3_im_mem_destroy(ctx, &im_mem);
```

rknn3_im_cvt_color

该接口用于在 RKNN3 图像内存中进行图像颜色格式转换，可在模型推理前将输入图像从一种 RGB/YUV 格式转换为另一种格式。

API	rknn3_im_cvt_color
功能	在 RKNN3 图像内存对象之间进行颜色空间转换。
参数	rknn3_context context：RKNN3 上下文句柄
	rknn3_im_mem* src：源图像内存对象
	rknn3_im_mem* dst：目标图像内存对象
返回值	int 状态码：见 6.2 状态码
注意	`src` 和 `dst` 均应由 `rknn3_im_mem_create` 创建

示例：

```
// 将 RGB888 转为 BGR888（src 和 dst 均需先用 im_mem_create 创建）
rknn3_im_cvt_color(ctx, &src_im_mem, &dst_im_mem);
```

5.14. 辅助函数

提供 tensor 类型、量化类型和布局类型的字符串转换。

rknn3_get_type_string

获取 tensor 类型的字符串表示。

API	rknn3_get_type_string
功能	获取 tensor 类型的字符串表示。
参数	rknn3_tensor_type type： tensor 的类型。
返回值	const char* 类型字符串

示例：

```
const char* type_str = rknn3_get_type_string(RKNN3_TENSOR_FLOAT32);
// type_str = "FP32"
```

rknn3_get_qnt_type_string

获取量化类型的字符串表示。

API	rknn3_get_qnt_type_string
功能	获取量化类型的字符串表示。
参数	rknn3_tensor_qnt_type type： tensor 的量化类型。
返回值	const char* 量化类型字符串

示例：

```
const char* qnt_str =
rknn3_get_qnt_type_string(RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC);
// qnt_str = "PER_CHANNEL_SYMMETRIC"
```

rknn3_get_layout_string

获取布局类型的字符串表示。

API	rknn3_get_layout_string
功能	获取布局类型的字符串表示。
参数	rknn3_tensor_layout layout： tensor 的布局类型。
返回值	const char* 布局类型字符串

示例：

```
const char* layout_str = rknn3_get_layout_string(RKNN3_TENSOR_NHWC);
// layout_str = "NHWC"
```

5.15. 结构化输出(grammar)

基于 xgrammar 的 LLM 结构化输出约束（JSON Schema / EBNF / 正则 / 内置 JSON 等），用于约束模型生成满足特定语法规则的文本。使用流程如下：

1. **初始化 grammar 引擎（上下文级）：** 调用 `rknn3_init_grammar` 传入编码后的词表（`token_id -> byte string`）与停止 token ID，一次性构建 XGrammar 的 TokenizerInfo 与

GrammarCompiler，供该上下文下所有会话共享。

2. **创建 grammar 句柄：** 调用 `rknn3_grammar_create` 创建绑定到上下文的 `rknn3_grammar` 句柄。每个会话应持有独立句柄以保证约束互不影响。
3. **设置语法规则：** 调用 `rknn3_grammar_set` 设置 JSON Schema / EBNF 等规则字符串及类型；亦可通过 `rknn3_session_set_grammar` 在会话上直接设置。
4. **采样约束：** 在自定义采样回调中，采样前调用 `rknn3_grammar_apply_mask` 将非法 token 的 logits 置为 -inf；采样并接受 token 后调用 `rknn3_grammar_accept_token` 推进匹配器状态。
5. **清理与释放：** 通过 `rknn3_grammar_clear` / `rknn3_session_clear_grammar` 取消约束恢复自由生成，通过 `rknn3_grammar_reset` 重置匹配器状态，最后通过 `rknn3_grammar_destroy` 释放句柄。

> **注意：** `rknn3_grammar_apply_mask` 必须在采样下一个 token **之前**调用；`rknn3_grammar_accept_token` 必须在采样并接受 token **之后**调用。
>
> **注意：** 若调用 `rknn3_session_set_grammar` 在会话上设置语法，`rknn3_session_run` 会在 decode 阶段自动应用掩码并接受 token，无需手动调用 `rknn3_grammar_apply_mask` / `rknn3_grammar_accept_token`。

rknn3_init_grammar

在上下文（模型）级别初始化 grammar 引擎。根据 `config` 中携带的编码词表构建 XGrammar 的 TokenizerInfo 与 GrammarCompiler。调用方需在调用本接口前自行构建 `encoded_vocab`（例如通过分词器的 TokenToPiece 遍历 token ID）。

API	rknn3_init_grammar
功能	在上下文（模型）级别初始化基于xgrammar的grammar引擎。
参数	rknn3_context context：RKNN3上下文句柄
	const rknn3_grammar_config* config：携带编码词表、词表大小与停止token ID的配置指针
返回值	int 状态码：见 6.2 状态码
注意	一次性、上下文级操作，所有会话共享；须在创建/使用grammar句柄前调用

示例：

```
rknn3_grammar_config config = {0};
config.vocab_size = vocab_size;
config.encoded_vocab = encoded_vocab;       // token_id -> byte string 序列化数据
config.encoded_vocab_len = encoded_vocab_len;
config.n_stop_tokens = 1;
config.stop_tokens = &eos_id;
rknn3_init_grammar(ctx, &config);
```

rknn3_grammar_create

创建一个绑定到上下文的 grammar 句柄。句柄共享上下文级的 TokenizerInfo / GrammarCompiler，并持有自身的语法规则与按请求的 GrammarMatcher 状态。每个会话应持有独立句柄以保持约束独立。

API	rknn3_grammar_create
功能	创建绑定到上下文的 grammar 句柄。
参数	rknn3_context context：RKNN3上下文句柄（须已调用 `rknn3_init_grammar`）
返回值	rknn3_grammar* 成功返回非NULL的grammar句柄，失败返回NULL

示例：

```
rknn3_grammar* grammar = rknn3_grammar_create(ctx);
if (!grammar) { /* 错误处理 */ }
```

rknn3_grammar_destroy

销毁 grammar 句柄并释放该句柄相关资源。不影响上下文级的词表/编译器，可在所属会话销毁后安全调用。

API	rknn3_grammar_destroy
功能	销毁 grammar 句柄并释放相关资源。
参数	rknn3_context context：RKNN3上下文句柄
	rknn3_grammar* grammar：要销毁的grammar句柄（可为NULL，空操作）
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_grammar_destroy(ctx, grammar);
grammar = NULL;
```

rknn3_grammar_set

在句柄上设置语法规则，对下一次 `rknn3_session_run` 生效。语法字符串与类型见 rknn3_grammar_type。传 NULL 或空字符串配合 `RKNN3_GRAMMAR_BUILTIN_JSON` 可启用任意合法 JSON 约束；其他类型下空/NULL字符串表示禁用约束。`options` 为编译选项（见 rknn3_grammar_set_options），每次设置都会重新编译语法。

API	rknn3_grammar_set
功能	在 grammar 句柄上设置语法规则。
参数	rknn3_grammar* grammar：grammar句柄
	const char* grammar_str：语法规则字符串（JSON Schema / EBNF / 正则等）
	rknn3_grammar_type grammar_type：语法字符串的类型
	const rknn3_grammar_set_options* options：编译选项，传NULL表示全部使用默认值
返回值	int 状态码：见 6.2 状态码

示例：

```
// 约束输出为合法的 JSON 对象
const char* schema = "{\"type\":\"object\",\"properties\":{\"name\":
{\"type\":\"string\"}},\"required\":[\"name\"]}";
rknn3_grammar_set(grammar, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);

// 任意合法 JSON
rknn3_grammar_set(grammar, NULL, RKNN3_GRAMMAR_BUILTIN_JSON, NULL);
```

rknn3_grammar_clear

清除句柄上的语法规则，恢复无约束生成。句柄被保留，之后可再次通过 `rknn3_grammar_set` 重新设置。

API	rknn3_grammar_clear
功能	清除 grammar 句柄上的语法规则。
参数	rknn3_grammar* grammar：grammar句柄
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_grammar_clear(grammar);  // 恢复无约束生成
```

rknn3_grammar_apply_mask

将语法 bitmask 应用到 logits（将非法 token 的 logits 置为 -inf），必须在采样下一个 token 之前调用。内部通过 GrammarMatcher::FillNextTokenBitmask 填充 bitmask，并调用 ApplyTokenBitmaskInplaceCPU 应用。`output_idx` 用于多输出场景，单输出传 0。

API	rknn3_grammar_apply_mask
功能	将语法 bitmask 应用到 logits，置非法 token 的 logits 为 -inf。
参数	rknn3_grammar* grammar：grammar句柄
	float16* logits：float16 logits数组指针（原地修改）
	int32_t vocab_size：词表大小（logits数组长度）
	int32_t output_idx：多输出场景下的输出索引（单输出传0）
返回值	int 状态码：见 6.2 状态码

示例：

```
// 在自定义采样回调中：先应用掩码，再采样
rknn3_grammar_apply_mask(grammar, logits, vocab_size, 0);
int token_id = my_sampling(logits, vocab_size);
```

rknn3_grammar_accept_token

通知 grammar 句柄一个 token 已被接受，推进匹配器状态。必须在采样并接受 token 之后调用。若 token 被语法拒绝（采样器与掩码不匹配），返回负值。

API	rknn3_grammar_accept_token
功能	通知 grammar 句柄已接受的 token，推进匹配器状态。
参数	rknn3_grammar* grammar：grammar句柄
	int32_t token_id：采样并接受的 token ID
	int32_t output_idx：多输出场景下的输出索引（单输出传0）
返回值	int 状态码：0表示token合法；负值表示token被语法拒绝，见 6.2 状态码

示例：

```
rknn3_grammar_accept_token(grammar, token_id, 0);
```

rknn3_grammar_reset

重置 grammar 匹配器状态（例如清空对话历史时）。句柄保留，可继续使用。

API	rknn3_grammar_reset
功能	重置 grammar 匹配器状态。
参数	rknn3_grammar* grammar：grammar句柄
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_grammar_reset(grammar);
```

rknn3_session_set_grammar

便捷接口：在会话上设置语法（内部创建/使用 grammar 句柄）。要求上下文已调用 `rknn3_init_grammar`。会话内部持有一个 `rknn3_grammar` 句柄，`rknn3_session_run` 会在 decode 阶段自动应用掩码并接受 token。

API	rknn3_session_set_grammar
功能	在会话上设置语法约束。
参数	rknn3_session* session：RKNN3会话句柄
	const char* grammar_str：语法规则字符串
	rknn3_grammar_type grammar_type：语法字符串的类型
	const rknn3_grammar_set_options* options：编译选项，传NULL表示全部使用默认值
返回值	int 状态码：见 6.2 状态码
注意	须先对上下文调用 `rknn3_init_grammar`

示例：

```
rknn3_session_set_grammar(session, schema, RKNN3_GRAMMAR_JSON_SCHEMA, NULL);
```

rknn3_session_clear_grammar

便捷接口：清除会话上的语法约束，恢复无约束生成。

API	rknn3_session_clear_grammar
功能	清除会话上的语法约束。
参数	rknn3_session* session：RKNN3会话句柄
返回值	int 状态码：见 6.2 状态码

示例：

```
rknn3_session_clear_grammar(session);  // 恢复无约束生成
```

5.16. MatMul矩阵乘法

以下接口声明于 `rknn3_matmul_api.h`，均返回 `int` 状态码：成功为 `RKNN3_SUCCESS`，失败为对应错误码，见 6.2 状态码。调用流程见 4.5 MatMul矩阵乘法。

rknn3_matmul_init

创建MatMul句柄并注册可执行的形状。

```
int rknn3_matmul_init(rknn3_context context,
                      const rknn3_matmul_desc* descs,
                      uint32_t n_descs,
                      rknn3_matmul_context* matmul);
```

参数	说明
context	已初始化的父上下文，在MatMul句柄销毁前必须保持有效。
descs	输入形状描述数组，各结构体先清零，再填写所需字段。
n_descs	注册形状数，必须大于0。
matmul	输出句柄地址，0为无效句柄。

`descs[i]` 对应形状索引i。一个句柄固定一种类型和一个NPU核心，多形状约束见 `rknn3_matmul_desc`。初始化成功后可以查询内存要求，执行前还需构建和选择形状。目标平台不支持的形状或类型组合可能初始化失败，应检查返回值。

rknn3_matmul_query_infos

查询全部已注册形状的A/B/C属性及量化要求。

```
int rknn3_matmul_query_infos(rknn3_matmul_context matmul,
                             rknn3_matmul_info* infos,
                             uint32_t n_descs);
```

参数	说明
matmul	初始化成功的句柄。
infos	用户提供的输出数组，`infos[i]` 对应 `descs[i]`。
n_descs	输出数组容量，不小于注册形状数。

初始化成功后即可查询，无需先调用 `build`。容量不足时返回错误且不写入；容量超过注册数时，多余项不修改。本接口不选择当前形状。应按查询结果分配NPU缓冲区，转换时直接使用查询属性，不能从M/K/N自行推算NPU填充与步长。

rknn3_matmul_build

构建句柄使用的运行资源。

```
int rknn3_matmul_build(rknn3_matmul_context matmul,
                       const rknn3_matmul_config* config);
```

`matmul` 为初始化成功的句柄。默认配置直接传 `NULL`；传入配置结构体时必须全部清零。构建不会分配用户的A/B/C，也不会选择形状。成功后不可重复构建，否则返回 `RKNN3_ERR_MATMUL_INVALID_STATE`。

rknn3_matmul_set_shape

选择已注册形状，供后续转换、量化和执行使用。

```
int rknn3_matmul_set_shape(rknn3_matmul_context matmul, uint32_t shape_id);
```

`matmul` 必须已构建；`shape_id` 为从0开始的注册索引。非法索引不会改变原选择。此接口只选择形状，不转换或绑定B。已成功提交的量化值保留，切回该形状时无需重新提交。

rknn3_matmul_transpose

在连续逻辑布局与NPU布局之间转换数据。此处的“transpose”表示存储布局转换，接口不提供任意矩阵转置或数据类型转换。

```
int rknn3_matmul_transpose(rknn3_matmul_context matmul,
                           const rknn3_matmul_tensor_attr* src_attr,
                           const rknn3_tensor_mem* src,
                           const rknn3_matmul_tensor_attr* dst_attr,
                           rknn3_tensor_mem* dst);
```

参数	说明
matmul	已构建且已选择形状的句柄。
src_attr / dst_attr	源与目标属性，逻辑端由用户填写，NPU端使用当前形状的查询属性。
src / dst	源与目标内存，两端均需CPU映射；所有参数均必填，包括转换B时的目标参数。

支持的布局方向如下：

用途	源布局	目标布局
A或feature B输入	NCHW	NC1HWC2
普通B输入	NCHW	O1I1HWI2O2
C输出	NC1HWC2	NCHW

两端的规范形状和 `dtype` 必须一致，并匹配当前形状。逻辑端属性先清零，填写 `n_dims=4`、`shape`、`dtype`、`layout=NCHW` 和逻辑数据字节数 `size`；`n_strides=0` 表示连续存储，不支持任意逻辑步长。

逻辑FP16使用 `uint16_t` 保存半精度浮点位模式，不能直接传入float数组；INT8/INT32分别使用 `int8_t` / `int32_t`。W4A16/W6A16的逻辑B每个值占一个 `int8_t`，`dtype` 仍为INT4/INT6，值应分别在[-8,7] / [-32,31]范围内。逻辑B字节数为 `B_batch*K*N`，由本接口完成打包。

实际CPU地址为 `virt_addr + offset`，可用容量为 `size - offset`；NPU端还需满足查询的对齐及核心要求。源与目标内存不得重叠。此接口仅转换数据，不执行NPU任务、不同步缓存，也不将B保存到句柄；运行时仍必须传入B。

rknn3_matmul_set_quant

为当前形状提交量化值。

```
int rknn3_matmul_set_quant(rknn3_matmul_context matmul,
                           const rknn3_matmul_quantization_t* quant_A,
                           const rknn3_matmul_quantization_t* quant_B,
                           const rknn3_matmul_quantization_t* quant_C);
```

`matmul` 必须已构建并选择形状。参数字段应与 `info.A/B/C.quant_info` 的查询要求一致，量化个数与数据类型见 6.11 MatMul类型。

- W8A8 per-channel/per-layer：A/B/C三个参数均必填，每个形状分别成功提交一次。不同形状的B量化值必须一致，A/C可不同。
- W4A16/W6A16/W8A16：仅传B，即 `rknn3_matmul_set_quant(matmul, NULL, &quant_B, NULL)`，整个句柄成功提交一次即可。
- W16A16/W8A8O32：不调用本接口。

本接口复制量化数组，成功返回后用户可释放原数组。W4A16/W6A16/W8A16同时完成共享设备Param的分配、打包和同步，不依赖B权重数据；后续运行复用Param。W8A8保留每个形状的量化值，在每次run中使用本次B权重打包并同步该形状的Param，首次运行时分配Param。成功提交后再次提交相同形状的W8A8参数或同一句柄的分组量化参数会返回 `RKNN3_ERR_MATMUL_INVALID_STATE`。

rknn3_matmul_run

同步执行当前形状的矩阵乘法。

```
int rknn3_matmul_run(rknn3_matmul_context matmul,
                     rknn3_tensor_mem* A,
                     rknn3_tensor_mem* B,
                     rknn3_tensor_mem* C);
```

`matmul` 必须已构建、已选择形状，并已提交所需量化值。A/B/C均为用户持有的NPU缓冲区，必须非空，并符合当前形状查询的大小、布局、步长、对齐及核心要求；普通打包B还必须有CPU映射。

每次运行均使用本次传入的B，用户可在下一次运行前更新或更换B。普通B使用查询的权重布局；单形状W16A16的feature B使用NC1HWC2布局。运行前需转换并同步A/B，成功返回后才可同步并读取C。运行期间不得修改输入或并发访问输出。

未提交所需量化值时返回 `RKNN3_ERR_MATMUL_INVALID_STATE`。成功表示NPU执行完成；失败不保证C有效。完成状态未知时的资源保留要求见 4.5 MatMul矩阵乘法。

rknn3_matmul_destroy

释放MatMul句柄及其运行资源。

```
int rknn3_matmul_destroy(rknn3_matmul_context matmul);
```

本接口不释放用户提供的A/B/C。成功返回后句柄失效，不可继续使用或重复销毁。如果返回 `RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN`，句柄及其资源仍保留；MatMul句柄销毁成功前，不得销毁父上下文或修改、复用、释放A/B/C。

使用示例

以下片段展示W8A8O32的形状注册和构建。该类型无需提交量化参数。

```
#include "rknn3_api.h"
#include "rknn3_matmul_api.h"
#include <stddef.h>

/* context已由rknn3_init创建；调用者负责销毁输出句柄。 */
int prepare_matmul(rknn3_context context,
                   rknn3_matmul_context* matmul,
                   rknn3_matmul_info* info)
{
    rknn3_matmul_desc desc = {0};
    desc.M = 16;
    desc.K = 64;
    desc.N = 64;
    desc.A_batch = desc.B_batch = 1;
    desc.type = RKNN3_MATMUL_W8A8O32;
    desc.A_layout = desc.B_layout = desc.C_layout = RKNN3_TENSOR_NCHW;
    desc.core_mask = RKNN3_NPU_CORE_0;

    int ret = rknn3_matmul_init(context, &desc, 1, matmul);
    if (ret != RKNN3_SUCCESS) return ret;
    ret = rknn3_matmul_query_infos(*matmul, info, 1);
    if (ret != RKNN3_SUCCESS) return ret;
    ret = rknn3_matmul_build(*matmul, NULL);
    if (ret != RKNN3_SUCCESS) return ret;
    return rknn3_matmul_set_shape(*matmul, 0);
}
```

调用前将输出句柄置0；即使后续查询或构建失败，只要初始化已成功，调用者仍应检查 `rknn3_matmul_destroy` 的结果并释放句柄。

本例逻辑数组使用 `int8_t a[64*16]`、`int8_t b[64*64]`、`int32_t c[64*16]`，索引分别为 `a[k*16+m]`、`b[k*64+n]`、`c[n*16+m]`。A/C与常见的M×K和M×N行优先数组顺序不同，应在输入和输出时按该顺序组织数据。

6. 类型参考

6.1. 常量

tensor 相关常量

常量名	值	描述
RKNN3_MAX_DIMS	16	tensor 的最大维度
RKNN3_MAX_STRIDE_DIMS	17	tensor 的最大步长维度
RKNN3_MAX_NAME_LEN	256	tensor 名称的最大长度
RKNN3_MAX_DYNAMIC_SHAPE_NUM	512	每个输入的最大动态形状数量
RKNN3_MAX_LORA_NUM	32	最大LoRA数量
RKNN3_MAX_SPECIAL_BOS_ID_NUM	64	特殊序列开始(BOS)Token的ID 的最大数量
RKNN3_MAX_SPECIAL_EOS_ID_NUM	64	特殊序列结束(EOS)Token的ID 的最大数量
RKNN3_MAX_KVCACHE_LEN_GROUPS	16	KV缓存长度分组的最大数量
RKNN3_MAX_ATTENTION_TYPE_NUM	3	注意力类型的最大数量
RKNN3_MAX_CACHE_PAGE_TYPES	16	单个核心支持的最大page KV cache类型数量
RKNN3_GRAMMAR_SET_OPTIONS_INIT	`{ -1, -1, -1, -1, -1, NULL, NULL, NULL }`	rknn3_grammar_set_options 结构体的默认初始化宏，各字段取-1/NULL表示使用xgrammar默认值

设备相关常量

常量名	值	描述
RKNN3_MAX_DEVS	64	最大设备数量
RKNN3_MAX_DEV_LEN	64	设备ID/类型的最大长度
RKNN3_MAX_NPU_NODE_NUM	128	最大NPU节点数量

6.2. 状态码

状态码	值	描述
RKNN3_SUCCESS	0	执行成功
RKNN3_ERR_FAIL	-1	执行失败
RKNN3_ERR_ARGUMENT_INVALID	-2	参数无效
RKNN3_ERR_MODEL_INVALID	-3	模型无效
RKNN3_ERR_CTX_INVALID	-4	上下文无效
RKNN3_ERR_RUN_TASK_FAILED	-5	任务运行失败
RKNN3_ERR_OUT_OF_MEMORY	-6	内存不足

状态码	值	描述
RKNN3_ERR_TIMEOUT	-7	执行超时
RKNN3_ERR_INPUT_INVALID	-8	输入无效
RKNN3_ERR_OUTPUT_INVALID	-9	输出无效
RKNN3_ERR_DEVICE_UNAVAILABLE	-10	设备不可用
RKNN3_ERR_DEVICE_UNMATCH	-11	设备不匹配
RKNN3_ERR_TARGET_PLATFORM_UNMATCH	-12	目标平台不匹配
RKNN3_ERR_COMMUNICATION	-13	通信错误
RKNN3_ERR_MEM_SYNC_FAILED	-14	内存同步失败
RKNN3_ERR_KEY_INVALID	-15	解密密钥无效或 RSA 解密失败
RKNN3_ERR_DECRYPT_FAILED	-16	模型解密失败（密钥不匹配或数据损坏）
RKNN3_ERR_WEIGHT_LOAD_NOT_PREPARED	-17	流式权重加载未准备好，请先调用 rknn3_model_init
RKNN3_ERR_INCOMPATIBLE_VERSION	-18	组件版本不兼容：librknn3_api、rknn3_transfer_proxy 与协处理器固件（或非协处理器的 rknn3_server）三者版本必须一致
RKNN3_ERR_MATMUL_INVALID_STATE	-19	MatMul调用状态无效，例如未构建、未选择形状、缺少量化值或重复提交。
RKNN3_ERR_MATMUL_COMPLETION_UNKNOWN	-20	无法确认MatMul任务完成；保留句柄、父上下文及A/B/C；MatMul句柄销毁成功前，不得修改、复用或释放相关缓冲区。
RKNN3_WARN_NPU_CORE_UNUSED	-100	NPU核心未使用，仅作为警告，不影响模型执行

6.3. 枚举

RKNN3 枚举类型按功能归类如下：

分类	说明
6.3.1. Tensor 与查询枚举	包含 4 个枚举
6.3.2. 内存与核心枚举	包含 4 个枚举
6.3.3. KV Cache 枚举	包含 6 个枚举
6.3.4. LLM 枚举	包含 4 个枚举
6.3.5. 图像处理枚举	包含 2 个枚举
6.3.6. 自定义算子枚举	包含 2 个枚举
6.3.7. 结构化输出枚举	包含 1 个枚举

6.3.1. Tensor 与查询枚举

RKNN3_QUERY_CMD

查询命令枚举，用于rknn3_query函数。

枚举值	描述
RKNN3_QUERY_IN_OUT_NUM	查询输入和输出 tensor 的数量
RKNN3_QUERY_INPUT_ATTR	查询输入 tensor 的属性
RKNN3_QUERY_OUTPUT_ATTR	查询输出 tensor 的属性
RKNN3_QUERY_SDK_VERSION	查询SDK和驱动版本
RKNN3_QUERY_CORE_MEM_SIZE	查询每个核心的权重和内部内存大小
RKNN3_QUERY_CUSTOM_STRING	查询自定义字符串
RKNN3_QUERY_ORIGIN_INPUT_ATTR	查询原始输入 tensor 的属性
RKNN3_QUERY_ORIGIN_OUTPUT_ATTR	查询原始输出 tensor 的属性
RKNN3_QUERY_DEVICE_MEM_INFO	查询RKNN3内存信息的属性
RKNN3_QUERY_CORE_NUMBER	查询核心数量
RKNN3_QUERY_ALLOCATION_INFO	查询分配信息
RKNN3_QUERY_DYNAMIC_SHAPE_CONFIG	查询完整的动态形状配置
RKNN3_QUERY_DYNAMIC_SHAPE_INFO	查询所有支持的形状组合
RKNN3_QUERY_LLM_CONFIG	查询LLM特有的配置，比如 chat template
RKNN3_QUERY_POSTPROCESS_IN_OUT_NUM	查询后处理的输入输出数量，仅在启用后处理时有效
RKNN3_QUERY_POSTPROCESS_OUTPUT_ATTR	查询后处理输出tensor属性，仅在启用后处理时有效
RKNN3_QUERY_POSTPROCESS_DYNAMIC_SHAPE_INFO	查询后处理的动态形状信息，仅在启用后处理时有效
RKNN3_QUERY_LORA_NUM	查询已初始化的LoRA数量，需在 `rknn3_lora_init` / `rknn3_lora_init_from_data` 之后调用，结果为 uint32_t
RKNN3_QUERY_LORA_INFO	查询LoRA信息列表，结果写入 `rknn3_lora` 数组（大小为 `RKNN3_MAX_LORA_NUM`）
RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO	查询KV缓存长度分组信息（分组数量、每组每核心的KV缓存大小）
RKNN3_QUERY_KVCACHE_PAGE_INFO	查询指定物理核心的page KV cache信息（最大并发请求数、cache类型、每页大小和最大页数）
RKNN3_QUERY_ATTENTION_CONFIG	查询所有 `rknn3_attention_config` 注意力配置项，用于获取分页参数等配置
RKNN3_QUERY_CMD_MAX	查询命令枚举的最大值

RKNN3_TENSOR_TYPE

tensor 数据类型枚举。

枚举值	描述
RKNN3_TENSOR_FLOAT32	float32类型
RKNN3_TENSOR_FLOAT16	float16类型
RKNN3_TENSOR_INT8	int8类型
RKNN3_TENSOR_UINT8	uint8类型

枚举值	描述
RKNN3_TENSOR_INT16	int16类型
RKNN3_TENSOR_UINT16	uint16类型
RKNN3_TENSOR_INT32	int32类型
RKNN3_TENSOR_UINT32	uint32类型
RKNN3_TENSOR_INT64	int64类型
RKNN3_TENSOR_UINT64	uint64类型
RKNN3_TENSOR_BOOL	bool类型
RKNN3_TENSOR_INT4	int4类型
RKNN3_TENSOR_FLOAT8E4M3FN	float8e4m3fn类型
RKNN3_TENSOR_BFLOAT16	bfloat16类型
RKNN3_TENSOR_FLOAT8E8M0	float8e8m0类型
RKNN3_TENSOR_FLOAT4E2M1	float4e2m1 (fp4)类型
RKNN3_TENSOR_INT6	int6类型
RKNN3_TENSOR_TYPE_MAX	tensor 数据类型枚举的最大值

RKNN3_TENSOR_QNT_TYPE

量化类型枚举。

枚举值	描述
RKNN3_TENSOR_QNT_NONE	无量化
RKNN3_TENSOR_PER_LAYER_SYMMETRIC	按层对称量化
RKNN3_TENSOR_PER_LAYER_ASYMMETRIC	按层非对称量化
RKNN3_TENSOR_PER_CHANNEL_SYMMETRIC	按通道对称量化
RKNN3_TENSOR_PER_CHANNEL_ASYMMETRIC	按通道非对称量化
RKNN3_TENSOR_PER_GROUP_SYMMETRIC	按组对称量化
RKNN3_TENSOR_PER_GROUP_ASYMMETRIC	按组非对称量化
RKNN3_TENSOR_QNT_MAX	量化类型枚举的最大值

RKNN3_TENSOR_LAYOUT

tensor 数据布局枚举。

枚举值	描述
RKNN3_TENSOR_UNDEFINED	未定义
RKNN3_TENSOR_NCHW	数据布局为NCHW
RKNN3_TENSOR_NHWC	数据布局为NHWC
RKNN3_TENSOR_NC1HWC2	数据布局为NC1HWC2
RKNN3_TENSOR_CHWN	保留值，暂未使用

枚举值	描述
RKNN3_TENSOR_HWIO	保留值，暂未使用
RKNN3_TENSOR_OIHW	保留值，暂未使用
RKNN3_TENSOR_O1I1HWI2O2	保留值，暂未使用
RKNN3_TENSOR_LAYOUT_MAX	tensor 数据布局枚举的最大值

6.3.2. 内存与核心枚举

RKNN3_MEM_ALLOC_FLAGS

创建RKNN3 tensor 内存的内存分配标志。

枚举值	描述
RKNN3_FLAG_MEMORY_FLAGS_DEFAULT	与RKNN3_FLAG_MEMORY_CACHEABLE相同
RKNN3_FLAG_MEMORY_CACHEABLE	创建cacheable内存
RKNN3_FLAG_MEMORY_NON_CACHEABLE	创建non-cacheable内存

RKNN3_MEM_SYNC_MODE

rknn3_mem_sync函数的内存同步模式。

枚举值	描述
RKNN3_MEMORY_SYNC_TO_DEVICE	表示CPU cache数据同步到DDR中，通常用于CPU写入内存后，NPU访问相同内存前使用该模式将cache中的数据写回DDR
RKNN3_MEMORY_SYNC_FROM_DEVICE	表示DDR数据同步到CPU cache，通常用于NPU写入内存后，使用该模式让下次CPU访问相同内存时，cache数据无效，CPU从DDR重新读取数据
RKNN3_MEMORY_SYNC_BIDIRECTIONAL	表示CPU cache数据同步到DDR同时令CPU重新从DDR读取数据

RKNN3_MEM_TYPE

内存类型枚举。

枚举值	描述
RKNN3_MEMORY_TYPE_NPU_DRAM	NPU DRAM内存
RKNN3_MEMORY_TYPE_EXT_DDR	外部DDR内存

RKNN3_CORE_MASK

在目标NPU核心上运行的模式。

枚举值	描述
RKNN3_NPU_CORE_AUTO	默认，表示自动调度模型，自动运行在当前空闲的NPU核上
RKNN3_NPU_CORE_0	在NPU核心0上运行
RKNN3_NPU_CORE_1	在NPU核心1上运行

枚举值	描述
RKNN3_NPU_CORE_2	在NPU核心2上运行
RKNN3_NPU_CORE_3	在NPU核心3上运行
RKNN3_NPU_CORE_4	在NPU核心4上运行
RKNN3_NPU_CORE_5	在NPU核心5上运行
RKNN3_NPU_CORE_6	在NPU核心6上运行
RKNN3_NPU_CORE_7	在NPU核心7上运行
RKNN3_NPU_CORE_ALL	表示工作在所有的NPU核心上

6.3.3. KV Cache 枚举

RKNN3_KVCACHE_POLICY

KV缓存的策略。

枚举值	描述
RKNN3_KVCACHE_POLICY_DEFAULT	默认策略，等同于RKNN3_KVCACHE_POLICY_RECURRENT
RKNN3_KVCACHE_POLICY_RECURRENT	使用递归缓存策略
RKNN3_KVCACHE_POLICY_NORMAL	使用普通缓存策略，仅在max_context_len范围内使用KV缓存
RKNN3_KVCACHE_POLICY_SAVE_CHECKPOINT	使用检查点缓存策略

RKNN3_KVCACHE_CLEAR_POLICY

清除KV缓存的策略。

枚举值	描述
RKNN3_KVCACHE_CLEAR_ALL	完全清除所有KV缓存条目
RKNN3_KVCACHE_KEEP_SYSTEM_PROMPT	清除KV缓存但保留系统提示
RKNN3_KVCACHE_KEEP_N_TOKENS	清除KV缓存但保留N个token，保留数量由 `rknn3_kvcache_clear_policy_param.keep_n_tokens.n_tokens` 指定

RKNN3_KVCACHE_DTYPE

KV缓存的数据类型。这些类型为内部定义数据类型，影响KV缓存大小和管理，用户无须解析。

枚举值	描述
RKNN3_KVCACHE_DTYPE_UNDEFINED	未定义
RKNN3_KVCACHE_DTYPE_INT4_TO_F16	INT4转FLOAT16
RKNN3_KVCACHE_DTYPE_INT4_TO_F8	INT4转FLOAT8
RKNN3_KVCACHE_DTYPE_INT8_TO_F16	INT8转FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT4_TO_F16	FLOAT4转FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT4_TO_F8	FLOAT4转FLOAT8

枚举值	描述
RKNN3_KVCACHE_DTYPE_FLOAT8_TO_F16	FLOAT8转FLOAT16
RKNN3_KVCACHE_DTYPE_FLOAT8_TO_F8	FLOAT8转FLOAT8
RKNN3_KVCACHE_DTYPE_FLOAT16	FLOAT16

RKNN3_KVCACHE_STORE_METHOD

KV缓存的存储方式。这些类型为内部定义数据类型，影响KV缓存大小和管理，用户无须解析。

枚举值	描述
RKNN3_KVCACHE_STORE_METHOD_UNDEFINED	未定义
RKNN3_KVCACHE_STORE_METHOD_NORMAL	普通存储
RKNN3_KVCACHE_STORE_METHOD_GROUP_QUANT	分组量化存储

RKNN3_ATTENTION_TYPE

LLM注意力类型枚举，用于注意力配置。

枚举值	描述
RKNN3_ATTENTION_TYPE_FULL_ATTENTION	全注意力
RKNN3_ATTENTION_TYPE_SLIDING_ATTENTION	滑动注意力
RKNN3_ATTENTION_TYPE_LINEAR_ATTENTION	线性注意力

RKNN3_PAGE_CACHE_TYPE

Page KV cache的cache类别。枚举值与模型schema中的cache类别保持一致。

枚举值	值	描述
RKNN3_PAGE_CACHE_GENERAL	11	通用cache
RKNN3_PAGE_CACHE_FULL_ATTENTION	12	全注意力cache
RKNN3_PAGE_CACHE_SLIDING_ATTENTION	13	滑动注意力cache
RKNN3_PAGE_CACHE_LINEAR_ATTENTION	14	线性注意力cache
RKNN3_PAGE_CACHE_CONV_STREAMING	15	卷积流式cache
RKNN3_PAGE_CACHE_FULL_RESIDUAL	17	全注意力残差cache
RKNN3_PAGE_CACHE_SLIDING_RESIDUAL	18	滑动注意力残差cache

6.3.4. LLM 枚举

RKNN3_LLM_INPUT_TYPE

定义可以输入到LLM的输入类型。

枚举值	描述
RKNN3_LLM_INPUT_PROMPT	输入是文本提示

枚举值	描述
RKNN3_LLM_INPUT_TOKEN	输入是一系列token
RKNN3_LLM_INPUT_EMBED	输入是嵌入向量
RKNN3_LLM_INPUT_MULTIMODAL	多模态输入
RKNN3_LLM_INPUT_AUX	其他类型输入
RKNN3_LLM_INPUT_MAX	输入类型枚举的最大值

LLMCALLSTATE

描述LLM调用的可能状态。

枚举值	描述
RKLLM_RUN_NORMAL	LLM调用处于正常运行状态
RKLLM_RUN_WAITING	LLM调用正在等待完整的UTF-8编码字符
RKLLM_RUN_FINISH	LLM调用已完成执行
RKLLM_RUN_STOP	用户停止了LLM调用
RKLLM_RUN_MAX_NEW_TOKEN_REACHED	已达到最大新Token数
RKLLM_RUN_ERROR	LLM调用期间发生错误
RKLLM_RUN_PAUSE	LLM调用已暂停
RKLLM_RUN_RESUME	LLM调用从暂停恢复

RKNN3_LLM_TASK_TYPE

LLM任务类型枚举。

枚举值	描述
RKNN3_LLM_TASK_GENERATE	生成任务
RKNN3_LLM_TASK_EMBEDDING	向量化/嵌入任务
RKNN3_LLM_TASK_RERANKER	重排序任务

LLMOUTPUTCALLBACKSTATE

输出回调阶段枚举。

枚举值	描述
RKLLM_OUTPUT_CALLBACK_PREFILL_PROCESSING	预填充阶段回调处理中
RKLLM_OUTPUT_CALLBACK_PREFILL_FINISHED	预填充阶段回调结束
RKLLM_OUTPUT_CALLBACK_DECODE_PROCESSING	解码阶段回调处理中
RKLLM_OUTPUT_CALLBACK_DECODE_FINISHED	解码阶段回调结束

6.3.5. 图像处理枚举

RKNN3_IM_FMT

图像格式枚举，用于 RKNN3 图像相关接口。

枚举值	描述
RKNN3_IM_FMT_RGB888	RGB888 格式，常用于通用图像预处理
RKNN3_IM_FMT_BGR888	BGR888 格式，常用于 OpenCV 图像处理
RKNN3_IM_FMT_GRAY8	8 位灰度图格式
RKNN3_IM_FMT_YCbCr_420_SP	YCbCr 4:2:0 半平面格式，多用于视频编解码
RKNN3_IM_FMT_YCrCb_420_SP	YCrCb 4:2:0 半平面格式，多用于视频编解码
RKNN3_IM_FMT_YCbCr_422_SP	YCbCr 4:2:2 半平面格式，多用于视频编解码
RKNN3_IM_FMT_YCrCb_422_SP	YCrCb 4:2:2 半平面格式，多用于视频编解码
RKNN3_IM_FMT_UNKNOWN	未知或暂不支持的图像格式

RKNN3_IM_PROC_FLAG

图像处理标志位枚举。

枚举值	描述
RKNN3_IM_PROC_FLAG_NONE	无处理
RKNN3_IM_PROC_FLAG_CROP	裁剪
RKNN3_IM_PROC_FLAG_RESIZE	缩放
RKNN3_IM_PROC_FLAG_FILL	填充
RKNN3_IM_PROC_FLAG_COLOR_SPACE_CONVERT	色彩空间转换
RKNN3_IM_PROC_FLAG_DECODE	解码
RKNN3_IM_PROC_FLAG_ENCODE	编码

6.3.6. 自定义算子枚举

RKNN3_OP_TARGET_TYPE

自定义算子后端目标枚举。

枚举值	描述
RKNN3_OP_TARGET_TYPE_CPU	运行在CPU
RKNN3_OP_TARGET_TYPE_MAX	枚举最大值

RKNN3_OP_PLUGIN_TYPE

自定义算子插件类型枚举。

枚举值	描述
RKNN3_OP_PLUGIN_TYPE_POSTPROCESS	后处理插件
RKNN3_OP_PLUGIN_TYPE_CUSTOM_OP	自定义算子插件（当前暂不支持）

枚举值	描述
RKNN3_OP_PLUGIN_TYPE_MAX	枚举最大值

6.3.7. 结构化输出枚举

RKNN3_GRAMMAR_TYPE

结构化输出的语法类型枚举，用于 `rknn3_grammar_set` / `rknn3_session_set_grammar`。

枚举值	描述
RKNN3_GRAMMAR_JSON_SCHEMA	JSON Schema字符串
RKNN3_GRAMMAR_EBNF	EBNF语法字符串
RKNN3_GRAMMAR_BUILTIN_JSON	内置JSON语法（任意合法JSON）
RKNN3_GRAMMAR_REGEX	正则表达式
RKNN3_GRAMMAR_STRUCTURAL_TAG	结构化标签（JSON序列化）

6.4. 通用结构体

Tensor 相关结构体

RKNN3_INPUT_OUTPUT_NUM

RKNN3_QUERY_IN_OUT_NUM的信息结构体。

成员变量	数据类型	含义
n_input	uint32_t	输入的数量
n_output	uint32_t	输出的数量

RKNN3_QUANT_INFO

量化信息结构体。

成员变量	数据类型	含义
scale	float	缩放数据
zero_point	int32_t	零点数据

RKNN3_TENSOR_ATTR

存储RKNN3_QUERY_INPUT_ATTR/RKNN3_QUERY_OUTPUT_ATTR查询结果的结构体。

成员变量	数据类型	含义
index	uint32_t	输入参数：输入/输出 tensor 的索引，在调用rknn3_query前需要设置index的值
name	char[RKNN3_MAX_NAME_LEN]	tensor 的名称
n_dims	uint32_t	维度的数量

成员变量	数据类型	含义
shape	uint32_t[RKNN3_MAX_DIMS]	有效维度数组
aligned_size	uint64_t	按字节计算的对齐形状的 tensor 大小
n_stride	uint32_t	步长的数量
stride	uint64_t[RKNN3_MAX_STRIDE_DIMS]	tensor 的步长，例如，16x16 tensor 的步长为[16*16, 16, 1]
n_elems	uint32_t	tensor 的元素数量
dtype	rknn3_tensor_type	tensor 的数据类型
layout	rknn3_tensor_layout	tensor 的数据布局
qnt_type	rknn3_tensor_qnt_type	tensor 的量化类型
qnt_info	rknn3_quant_info	tensor 的量化信息
n_orig_dims	uint32_t	原始维度的数量
orig_shape	uint32_t[RKNN3_MAX_DIMS]	原始有效维度数组
orig_layout	rknn3_tensor_layout	原始 tensor 的数据布局
core_id	int32_t	tensor 缓冲区的核心ID

RKNN3_TENSOR_MEM

tensor 内存信息结构体。

成员变量	数据类型	含义
virt_addr	void*	tensor 缓冲区的虚拟地址
phys_addr	uint64_t	tensor 缓冲区的物理地址
fd	int32_t	tensor 缓冲区的文件描述符
buffer_id	uint64_t	tensor 缓冲区的缓冲区ID，用于内存管理
offset	uint64_t	内存的偏移量
size	uint64_t	tensor 缓冲区的大小
flags	uint64_t	tensor 缓冲区的标志，保留字段
core_id	int32_t	NPU核心ID
priv_data	void*	tensor 缓冲区的私有数据
mem_type	rknn3_mem_type	tensor 缓冲区的内存类型

RKNN3_TENSOR

表示包含内存和属性信息的RKNN3 tensor 结构体。

该结构体包含了RKNN3操作中使用的 tensor 的内存信息和属性，是RKNN3运行时处理 tensor 的基本数据结构。

成员变量	数据类型	含义
mem	rknn3_tensor_mem*	tensor 的内存信息

成员变量	数据类型	含义
attr	rknn3_tensor_attr*	tensor 的属性

模型与配置结构体

RKNN3_SDK_VERSION

SDK版本信息结构体。

成员变量	数据类型	含义
api_version	char[256]	RKNN3 API的版本
drv_version	char[256]	RKNN3驱动的版本

RKNN3_CORE_MEM_SIZE

每个核心的权重和内部内存大小信息结构体。

成员变量	数据类型	含义
core_id	int32_t	内存归属的物理NPU核心ID
weight_size	uint64_t	权重内存大小（字节）
internal_size	uint64_t	内部Tensor内存大小（字节）
reserved	uint8_t[32]	保留字段

RKNN3_CUSTOM_STRING

自定义字符串信息结构体。

成员变量	数据类型	含义
string	char[1024]	自定义字符串，最大长度为1024字节

RKNN3_CONFIG

模型加载的控制参数结构体。

成员变量	数据类型	含义
priority	int32_t	运行优先级
run_timeout	uint32_t	运行超时时间（毫秒），0表示无超时
run_core_mask	uint32_t	模型执行的核心掩码
user_mem_weight	uint8_t	权重内存是否由用户分配
user_mem_internal	uint8_t	内部内存是否由用户分配
user_sram	uint8_t	SRAM内存是否由用户分配
reserved	uint8_t[128]	保留字段

RKNN3_SHAPE_INFO

RKNN3模型 tensor 形状信息结构体。

该结构体包含了RKNN3模型输入和输出 tensor 的形状的信息，包括 tensor 属性和形状配置详情。

成员变量	数据类型	含义
shape_id	int32_t	此形状组合的唯一ID
n_inputs	uint32_t	输入 tensor 的数量
input_attrs	rknn3_tensor_attr*	输入 tensor 属性数组
n_outputs	uint32_t	输出 tensor 的数量
output_attrs	rknn3_tensor_attr*	输出 tensor 属性数组
is_default	uint8_t	是否为默认形状
reserved	uint8_t[31]	保留字段

RKNN3_SHAPE_CONFIG

动态形状设置的配置结构体。

该结构体包含了RKNN3模型中动态形状推理的形状组合和当前活动形状的信息。

成员变量	数据类型	含义
n_shapes	uint32_t	可用的形状组合数量
current_shape_id	int32_t	当前活动形状配置的ID，值为-1表示没有活动形状

RKNN3_ALLOCATION_INFO

RKNN3模型的内存分配信息结构体。

该结构体提供了不同内存类型（命令、权重、内部、KV cache）及其在NPU核心上分布的详细内存分配信息。

成员变量	数据类型	含义
core_id	int32_t	NPU核心ID
command_mem	rknn3_tensor_mem	命令内存的信息
weight_mem	rknn3_tensor_mem	权重内存的信息
internal_mem	rknn3_tensor_mem	内部内存的信息
kvcache_mem	rknn3_tensor_mem	KV cache内存的信息
reserved	uint8_t[128]	保留字段

设备相关结构体

RKNN3_INIT_EXTEND

设备特定初始化信息结构体。

该结构体用于在RKNN3运行时上下文初始化期间指定设备特定参数，包括设备ID和为将来使用保留的空间。

成员变量	数据类型	含义
device_id	char*	输入参数，指示选择哪个设备。如果只连接了一个设备，可以设置为NULL
reserved	uint8_t[128]	保留字段

RKNN3_NODE_MEM_INFO

RKNN3设备节点内存信息结构体。

该结构体提供了RKNN3设备中每个节点的详细内存信息，包括可用于分配的总内存和可用内存。

成员变量	数据类型	含义
total	uint64_t	此节点可用的总内存，单位为字节
free	uint64_t	此节点可用的空闲内存，单位为字节

RKNN3_DEV_MEM_INFO

RKNN3设备节点内存信息结构体。

该结构体提供了RKNN3设备中每个节点的详细内存信息，包括可用于分配的总内存和可用内存。

成员变量	数据类型	含义
node_num	uint32_t	设备中的节点数量
sys_total	uint64_t	设备的总系统内存，单位为字节
sys_free	uint64_t	设备的可用系统内存，单位为字节
node_mem_info	rknn3_node_mem_info[RKNN3_MAX_NPU_NODE_NUM]	每个节点的内存信息

RKNN3_DEVICE

表示RKNN3设备的结构体。

该结构体包含了特定RKNN3设备的信息，包括其ID、类型和内存信息。

成员变量	数据类型	含义
id	char[RKNN3_MAX_DEV_LEN]	设备ID
type	char[RKNN3_MAX_DEV_LEN]	设备类型
mem_info	rknn3_dev_mem_info	设备的内存信息

RKNN3_DEVICES

包含RKNN3设备信息的结构体。

该结构体包含了可用RKNN3设备的数量。

成员变量	数据类型	含义
n_devices	uint32_t	设备数量
devices	rknn3_device[RKNN3_MAX_DEVS]	设备信息

6.5. LLM和VLM结构体

RKNN3_LLM_CONFIG

LLM配置结构体。

该结构体包含了初始化和运行LLM所需的基础配置。

成员变量	数据类型	含义
chat_template	char*	聊天模板字符串
vocab_size	uint32_t	词表大小
embedding_dim	uint32_t	嵌入维度（通常等于hidden size）
max_ctx_len	uint32_t	最大上下文长度
max_position_embeddings	uint32_t	最大位置编码长度
kvcache_store_method	rknn3_kvcache_store_method	KV缓存存储方式
kvcache_dtype	rknn3_kvcache_dtype	KV缓存数据类型
kvcache_group_size	uint32_t	KV缓存分组量化的组大小
kvcache_residual_depth	uint32_t	KV缓存剩余深度
model_type	char*	模型类型字符串
task_type	rknn3_llm_task_type	LLM任务类型
rope_cache_host_storage	uint8_t	RoPE缓存是否存储在主机端
n_attention_kvcache_lens	uint32_t	有效的注意力KV缓存长度配置数量
attention_kvcache_lens	rknn3_attention_kvcache_lens [RKNN3_MAX_ATTENTION_TYPE_NUM]	各注意力类型的KV缓存长度配置
reserved	uint8_t[127]	保留字段

RKNN3_VOCAB_INFO

RKNN3模型词汇信息结构体。

该结构体包含了RKNN3模型中使用的词汇信息，包括大小和特殊Token ID。

成员变量	数据类型	含义
vocab_size	int	词汇表大小
special_bos_id	int[RKNN3_MAX_SPECIAL_BOS_ID_NUM]	特殊序列开始(BOS)Token的ID
special_eos_id	int[RKNN3_MAX_SPECIAL_EOS_ID_NUM]	特殊序列结束(EOS)Token的ID
n_special_bos_id	int	特殊序列开始(BOS)Token ID 的数量
n_special_eos_id	int	特殊序列结束(EOS)Token ID 的数量
linefeed_id	int	换行Token的ID
skip_special_token	bool	生成时是否跳过特殊Token
ignore_eos_token	bool	生成时是否忽略EOS Token
reserved	uint8_t[64]	保留字段，用于未来扩展

RKNN3_LLM_EXTEND_PARAM

LLM扩展参数结构体。

成员变量	数据类型	含义
reserved	uint8_t[128]	保留字段

RKNN3_SAMPLING_PARAMS

定义LLM实例的采样参数结构体。

成员变量	数据类型	含义
top_k	int32_t	用于Token生成的Top-K采样参数
top_p	float	Top-P（核心）采样参数
temperature	float	采样温度，影响Token选择的随机性
repeat_penalty	float	生成中重复Token的惩罚
frequency_penalty	float	生成过程中对频繁Token的惩罚
presence_penalty	float	基于Token在输入中的存在对其进行惩罚

RKNN3_LLM_PARAM

定义配置LLM实例的参数结构体。

成员变量	数据类型	含义
logits_name	char*	输出logits的名称。仅当模型有多个输出时需要显式设置该字段，否则可以为NULL
max_context_len	int32_t	上下文中的最大Token数量
sampling_param	rknn3_sampling_params	Token生成的采样参数
vocab_info	rknn3_vocab_info	词汇信息
extend_param	rknn3_llm_extend_param	扩展参数

RKNN3_LORA

定义模型微调中使用的LoRA(Low-Rank Adaptation)参数结构体。

成员变量	数据类型	含义
lora_name	char[RKNN3_MAX_NAME_LEN]	LoRA的名称
scale	float	应用LoRA的缩放因子

RKNN3_LLM_MULTIMODAL_TENSOR

表示多模态输入（例如文本、图像、音频和视频）的结构体。

顶层成员

成员变量	数据类型	含义
name	const char*	此 tensor 的名称
prompt	const char*	文本提示输入
tokens	int32_t*	token 输入。与 prompt 输入是二选一关系，用户可提供任意一种输入。注意：1.如果两种输入都提供，则默认按照 prompt 输入推理。2.选择 token 输入时，用户需要自行将文本和多模态标签按照规则解析成 token。

成员变量	数据类型	含义
n_tokens	uint64_t	token 输入中 token 的数量。选择 token 输入时需要设置。
enable_thinking	bool	控制是否启用“思考模式”
image	struct	图像子结构
audio	struct	音频子结构
video	struct	视频子结构

image子结构相关成员

成员变量	数据类型	含义
image_embed	float16*	图像的嵌入（大小为 n_image × n_image_tokens × embedding_dim × sizeof(float16)）
n_image_tokens	uint64_t	每张图像的 token 数量
n_image	uint64_t	图像数量
image_start	const char*	多模态输入中图像的起始标签
image_end	const char*	多模态输入中图像的结束标签
image_content	const char*	多模态输入中图像内容占位符标签
image_width	uint64_t	图像宽度
image_height	uint64_t	图像高度

audio子结构相关成员

成员变量	数据类型	含义
audio_embed	float16*	音频的嵌入（大小为 n_audio × n_audio_tokens × embedding_dim × sizeof(float16)）
n_audio_tokens	uint64_t	每段音频的 token 数量
n_audio	uint64_t	音频数量
audio_start	const char*	多模态输入中音频起始标签
audio_end	const char*	多模态输入中音频结束标签
audio_content	const char*	多模态输入中音频内容占位符标签

video子结构相关成员

成员变量	数据类型	含义
video_embed	float16*	视频的嵌入（大小为 n_video × n_frame_per_video × n_frame_tokens × embedding_dim × sizeof(float16)）
n_frame_tokens	uint64_t	每帧的 token 数量

成员变量	数据类型	含义
n_frame_per_video	uint64_t	每个视频包含的帧数
n_video	uint64_t	视频数量
video_start	const char*	多模态输入中视频起始标签
video_end	const char*	多模态输入中视频结束标签
video_content	const char*	多模态输入中视频内容占位符标签
frame_width	uint64_t	视频帧的宽度
frame_height	uint64_t	视频帧的高度

RKNN3_LLM_TENSOR

表示大型语言模型操作的 tensor 结构体。

该结构体包含了处理语言模型嵌入所需的基本组件，包括 tensor 名称、嵌入向量、Token ID和Token数量。

成员变量	数据类型	含义
name	const char*	此 tensor 的名称
prompt	const char*	如果input_type为RKNN3_LLM_INPUT_PROMPT，则为文本提示输入
embed	float16*	如果input_type为RKNN3_LLM_INPUT_EMBED，则为指向嵌入向量的指针（大小为n_tokens * hidden_size）
tokens	int32_t*	如果input_type为RKNN3_LLM_INPUT_TOKEN，则为Token ID数组
n_tokens	uint64_t	嵌入中表示的Token数量
enable_thinking	bool	控制是否启用"思考模式"

RKNN3_LLM_INPUT

通过联合体表示LLM不同类型输入的结构体。

成员变量	数据类型	含义
role	const char*	消息角色："user"（用户输入）、"tool"（函数结果）
input_type	rknn3_llm_input_type	指定提供的输入类型（例如token、embed、aux）
llm_input	rknn3_llm_tensor	当input_type为RKNN3_LLM_INPUT_EMBED时为嵌入向量；当input_type为RKNN3_LLM_INPUT_TOKEN时为Token数组；当input_type为RKNN3_LLM_INPUT_PROMPT时使用prompt字段
multimodal_input	rknn3_llm_multimodal_tensor	当input_type为RKNN3_LLM_INPUT_MULTIMODAL时为多模态输入
aux_input	rknn3_aux_tensor	如果input_type为RKNN3_LLM_INPUT_AUX，则为AUX输入,目前rknn3_aux_tensor和rknn3_tensor同类型

RKNN3_LLM_INFER_PARAM

LLM推理参数结构体，定义了推理过程中的参数。

成员变量	数据类型	含义
keep_history	int	确定历史记录保留标志（1：保留历史记录，0：丢弃历史记录）
max_new_tokens	int32_t	生成的新Token最大数量
prefill_only	bool	是否仅执行prefill阶段。设为true时模型仅做prefill并跳过decode阶段，但仍会对prefill输出token进行采样。默认为false
disable_sampling	bool	是否禁用采样。设为true时模型不进行采样，适用于仅需prefill输出且不希望获取prefill输出token的场景（如embedding和reranker模型）。通常与 `prefill_only` 配合使用（单独设置 `disable_sampling` 无效）。默认为false
keep_grammar	bool	是否在多次运行之间保留grammar matcher状态。设为true时，每次 `rknn3_session_run` 开始/结束时不会重新初始化per-request grammar matcher，从而在多次运行之间保留grammar状态以支持连续生成。默认为false
reserved	uint8_t[127]	保留字段，用于未来扩展

RKLLMRESULT

表示LLM推理结果的结构体。

成员变量	数据类型	含义
token_ids	int*	指向LLM生成的token的指针
num_tokens	int	生成的token数量

RKLLMRUNSTATE

结构体RKLLMRunState包含LLM运行的状态信息。

成员变量	数据类型	含义
n_total_tokens	uint64_t	当前已处理的总Token数量
n_reuse_tokens	uint64_t	当前输入命中KV Cache缓存的Token数量，即KV Cache复用的Token数量
n_max_tokens	uint64_t	可处理的最大Token数量
n_prefill_tokens	uint64_t	预填充阶段处理的Token数量
n_decode_tokens	uint64_t	解码阶段已生成的Token数量
n_input_tokens	uint64_t	输入到模型的Token数量（与n_prefill_tokens相同）
n_output_tokens	uint64_t	模型输出的Token数量（n_output_tokens = n_decode_tokens + 1，其中1为预填充输出token）
input_tokens	const int32_t*	指向输入模型Token数组的指针，用户不应释放该指针
output_tokens	const int32_t*	指向模型输出Token数组的指针，用户不应释放该指针
kvcache_policy	rknn3_kvcache_policy	KV缓存策略

成员变量	数据类型	含义
n_loras_enabled	int32_t	启用的LoRA数量
loras_enabled	rknn3_lora*	启用的LoRA列表

6.6. KVCache结构体

RKNN3_KVCACHE_POLICY_PARAM

定义KV缓存策略的参数结构体。 若模型包含system prompt，recurrent参数中的 n_keep 与 n_keep_aligned 会被忽略。

成员变量	数据类型	含义
recurrent.n_keep	int64_t	递归策略下保留的缓存条目数量；若模型包含system prompt则忽略
recurrent.n_keep_aligned	int64_t	递归策略下对齐到kvcache_group_size的保留条目数量
save_checkpoint.checkpoint_start_pos	int64_t	checkpoint保存的起始位置（如0），需按128对齐，不对齐时自动调整为对齐值 (起始位置不保存检查点)
save_checkpoint.checkpoint_interval	int64_t	保存checkpoint的token间隔（如每128 token保存一次），需按128对齐，不对齐时自动调整为对齐值
save_checkpoint.max_checkpoint_count	int64_t	从 checkpoint_start_pos 起最多保存多少个checkpoint，须小于(max_context_len - checkpoint_start_pos) / checkpoint_interval，否则自动调整为最大可用值
save_checkpoint.checkpoint_tail_overwrite	bool	设为true时，末尾/最后一个checkpoint槽位作为滚动覆盖槽位。(max_checkpoint_count - 1)个checkpoint保存在固定位置，当token位置超过(checkpoint_start_pos + max_checkpoint_count * checkpoint_interval)后，后续checkpoint将覆盖末尾槽位。例如：max_checkpoint_count=5表示4个固定checkpoint + 1个滚动checkpoint末尾
reserved	uint8_t[64]	保留字段

RKNN3_ATTENTION_KVCACHE_LENS

KV缓存缓冲区长度配置结构体，用于一种注意力类型。

成员变量	数据类型	含义
attention_type	rknn3_attention_type	注意力类型
n_kvcache_buffer_lens	uint32_t	有效的KV缓存缓冲区长度数量
kvcache_buffer_lens	int32_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	KV缓存缓冲区长度数组

RKNN3_KVCACHE_LEN_GROUP_INFO

KV缓存长度分组信息结构体，用于RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO查询。

使用方法：先通过RKNN3_QUERY_CORE_NUMBER获取core_number，然后分配 `rknn3_kvcache_len_group_info info[core_number]`，并以 `sizeof(rknn3_kvcache_len_group_info) * core_number` 作为size进行查询。

成员变量	数据类型	含义
core_id	int32_t	NPU核心的物理ID
n_groups	uint32_t	KV缓存长度分组数量（0表示单组传统模式）
active_group_id	int32_t	当前激活的分组ID（默认0）
kvcache_sizes	uint64_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	kvcache_sizes[group_id]为该组在该核心上的KV缓存大小

RKNN3_ATTENTION_CONFIG

注意力配置结构体。通过 `RKNN3_QUERY_ATTENTION_CONFIG` 查询；page KV cache分配时重点使用 `attention_type`、`kvcache_buffer_len`、`tokens_per_page` 和 `rollback_tokens_align` 字段。

成员变量	数据类型	含义
attention_type	rknn3_attention_type	注意力类型
kvcache_buffer_len	int32_t	该注意力类型的KV cache缓冲区长度
kvcache_buffer_lens	int32_t[RKNN3_MAX_KVCACHE_LEN_GROUPS]	各KV cache长度分组的缓冲区长度
n_kvcache_buffer_lens	int32_t	`kvcache_buffer_lens` 中的有效数量
tokens_per_page	uint32_t	FullAttention分页时每页包含的token数量
rollback_tokens_align	uint32_t	KV cache回滚位置的对齐粒度

RKNN3_CACHE_PAGE_ATTR

单个page KV cache类别的页属性，由 `rknn3_kvcache_page_info.caches[]` 返回。

成员变量	数据类型	含义
cache_type	int32_t	page cache类别，取值见 `rknn3_page_cache_type`
page_size	uint64_t	每个page的字节数；创建page内存时作为 `rknn3_create_mem` 的size参数
max_page_num	uint32_t	编译期规划的最大页数；设置绑定后不会改变
reserved	uint8_t[32]	保留字段

RKNN3_KVCACHE_PAGE_INFO

指定物理核心的page KV cache规格和并发请求信息。调用 `rknn3_query` 前必须设置 `core_id`；老式非分页模型返回 `n_cache_types == 0`。

成员变量	数据类型	含义
core_id	int32_t	输入字段：要查询的物理NPU核心ID
max_request_num	uint32_t	continuous batching支持的最大并发请求数
n_cache_types	uint32_t	有效的cache类别数量
caches	rknn3_cache_page_attr[RKNN3_MAX_CACHE_PAGE_TYPES]	各cache类别的页属性
reserved	uint8_t[128]	保留字段

RKNN3_KVCACHE_PAGE_BINDING

`rknn3_set_kvcache_pages` 使用的page KV cache页表绑定。

成员变量	数据类型	含义
core_id	int32_t	物理NPU核心ID
cache_type	int32_t	要绑定的cache类别
request_index	uint32_t	并发请求索引，范围应小于对应 `max_request_num`
n_pages	uint32_t	`pages` 数组中的页数；设为0表示解除该键的绑定
pages	const rknn3_tensor_mem *const*	page内存数组；每个page应为对应核心上独立分配的 `page_size` 字节内存
reserved	uint8_t[64]	保留字段

`pages` 指向的页内存由调用方所有，运行时不会复制页内容或释放这些内存。绑定成功后，调用方必须保持页对象有效，直到使用 `n_pages = 0` 的binding成功解绑。

6.7. 自定义算子结构体

RKNN3_CUSTOM_OP_ATTR

自定义算子属性描述结构体，用于通过 `get_param` 查询单个算子属性。

成员变量	数据类型	含义
name	char[RKNN3_MAX_NAME_LEN]	算子属性名称
dtype	rknn3_tensor_type	算子属性的数据类型
n_elems	uint32_t	数组元素个数
data	void*	算子属性数组指针（非拥有，仅在当前回调作用域内有效，调用方不应缓存或释放）

RKNN3_CUSTOM_OP_CONTEXT

自定义算子上下文结构体，由框架管理，用户可通过user_data存放自定义信息。

成员变量	数据类型	含义
rknn_ctx	rknn3_context	RKNN3上下文句柄
priv_data	void*	框架私有数据

成员变量	数据类型	含义
user_data	void*	用户自定义数据
get_param	`int (*)(rknn3_custom_op_context*, const char*, rknn3_custom_op_attr*)`	按属性名查询算子参数的函数指针，由框架在调用init/compute回调前设置，用户不应覆盖。调用后返回的`op_attr.data`非拥有，仅在当前回调作用域内有效

RKNN3_CUSTOM_OP

自定义算子结构体，包含算子元信息与回调函数集合。

成员变量	数据类型	含义
op_type	const char*	自定义算子类型名称
plugin_type	rknn3_op_plugin_type	插件类型
target	rknn3_op_target_type	后端执行目标
version	uint32_t	算子版本号
author	const char*	作者信息
description	const char*	算子描述
init	`int (*)(rknn3_custom_op_context*)`	初始化回调（可选）
prepare	`int (*)(rknn3_custom_op_context*, rknn3_tensor*, uint32_t, rknn3_tensor*, uint32_t)`	预处理回调（可选）
compute	`int (*)(rknn3_custom_op_context*, rknn3_tensor*, uint32_t, rknn3_tensor*, uint32_t)`	计算回调（必选）
deinit	`int (*)(rknn3_custom_op_context*)`	反初始化回调（可选）
get_output_num	`int (*)(rknn3_custom_op_context*)`	获取输出数量回调（后处理插件，可选）
get_attrs	`int (*)(rknn3_custom_op_context*, rknn3_tensor_attr*, uint32_t, rknn3_tensor_attr*, uint32_t)`	获取输入输出tensor属性回调（后处理插件，可选）

6.8. 图像处理结构体

RKNN3_IM_RECT

图像矩形区域结构体，用于描述图像处理中的兴趣区域（ROI）。

成员变量	数据类型	含义
x	int	左上角 X 坐标
y	int	左上角 Y 坐标
width	int	矩形宽度（像素）
height	int	矩形高度（像素）

RKNN3_IM_METADATA

与 RKNN3 图像内存关联的元数据结构体，用于主机与设备之间的额外信息记录。

成员变量	数据类型	含义
peer_im_mem_addr	uint64_t	对端（远端）图像内存对象的地址

RKNN3_IM_MEM

图像内存信息结构体，描述存储在设备内存中的图像缓冲区，供 `rknn3_im_mem_create`、`rknn3_im_cvt_color` 等图像接口使用。

成员变量	数据类型	含义
width	int	图像宽度（像素）
height	int	图像高度（像素）
stride	int	图像缓冲区步长（字节）
format	rknn3_im_fmt	图像格式，见 rknn3_im_fmt
sync_to_host	bool	是否需要将图像数据回写到主机，默认 false
data_mem	rknn3_tensor_mem*	底层 tensor 内存信息指针
metadata	rknn3_im_metadata	主机与设备共享的额外元数据

6.9. Callback类型

本节汇总LLM推理过程中使用的回调函数指针类型及回调容器结构体。各回调类型的详细说明、参数语义与示例见 5.8 LLM配置与Callback。

LLMRESULTCALLBACK

LLM结果回调函数类型。处理LLM生成结果，详细说明见 LLMResultCallback。

函数指针类型	LLMResultCallback
签名	`int (*LLMResultCallback)(void* userdata, RKLLMResult* result, LLMCallState state)`
返回值	int 状态码：见 6.2 状态码

LLMSAMPLINGCALLBACK

LLM采样回调函数类型。处理采样logits，详细说明见 LLMSamplingCallback。

函数指针类型	LLMSamplingCallback
签名	`int (*LLMSamplingCallback)(void* userdata, float16* logits, char* logits_name)`
返回值	int：返回所选token ID (>=0)表示成功，负值表示错误码，见 6.2 状态码

LLMGETEMBEDCALLBACK

获取LLM嵌入向量的回调函数类型。详细说明见 LLMGetEmbedCallback。

函数指针类型	LLMGetEmbedCallback
签名	`int (*LLMGetEmbedCallback)(void* userdata, int32_t* tokens, uint64_t num_tokens, void* embed, uint64_t len)`
返回值	int 状态码：见 6.2 状态码

LLMTOKENIZERCALLBACK

LLM分词器回调函数类型。详细说明见 LLMTokenizerCallback。

函数指针类型	LLMTokenizerCallback
签名	`int (*LLMTokenizerCallback)(void* userdata, const char* text, int32_t text_len, int32_t* tokens, int32_t n_tokens_max)`
返回值	int：返回生成的token数量 (>=0)表示成功，负值表示错误码，见 6.2 状态码

LLMOUTPUTCALLBACK

获取输出tensor的回调函数类型。详细说明见 LLMOutputCallback。

函数指针类型	LLMOutputCallback
签名	`int (*LLMOutputCallback)(void* userdata, rknn3_tensor* output_tensors, uint32_t n_output_tensors, LLMOutputCallbackState state)`
返回值	int：成功返回0，错误返回非零值，见 6.2 状态码

LLMINPUTCALLBACK

提供自定义输入tensor的回调函数类型。详细说明（含 `LLMInputCallbackParam` 参数结构体）见 LLMInputCallback。

函数指针类型	LLMInputCallback
签名	`int (*LLMInputCallback)(void* userdata, rknn3_tensor* input_tensors, uint32_t n_input_tensors, LLMInputCallbackParam param)`
返回值	int：成功返回0，错误返回非零值，见 6.2 状态码

RKLLMCALLBACK

结构体RKLLMCallback包含LLM操作的回调函数集合，通过 `rknn3_session_set_callback` 注册。

成员变量	数据类型	含义
result_callback	LLMResultCallback	LLM返回结果的回调函数
result_userdata	void*	LLMResultCallback的用户数据

成员变量	数据类型	含义
sampling_callback	LLMSamplingCallback	可选：仅在需要自定义采样时使用
sampling_userdata	void*	LLMSamplingCallback的用户数据
tokenizer_callback	LLMTokenizerCallback	可选：仅在需要自定义分词器时使用
tokenizer_userdata	void*	LLMTokenizerCallback的用户数据
embed_callback	LLMGetEmbedCallback	可选：仅在需要自定义嵌入检索时使用
embed_userdata	void*	LLMGetEmbedCallback的用户数据
output_callback	LLMOutputCallback	可选：仅在需要获取输出tensor时使用
output_userdata	void*	LLMOutputCallback的用户数据
output_tensors	rknn3_tensor*	输出tensor数组指针
n_output_tensors	uint32_t	输出tensor数量
input_callback	LLMInputCallback	可选：仅在需要提供自定义输入tensor时使用
input_userdata	void*	LLMInputCallback的用户数据
input_tensors_index	int32_t*	输入tensor的索引数组
n_input_tensors	uint32_t	输入tensor数量

6.10. 结构化输出结构体

RKNN3_GRAMMAR_CONFIG

grammar 引擎初始化配置结构体（上下文级别），携带调用方构建的编码词表（token_id -> byte string）与停止 token ID，运行时据此一次性构建 XGrammar 的 TokenizerInfo 与 GrammarCompiler。

`encoded_vocab` 布局：对每个 token i（i ∈ [0, vocab_size)），依次存放 `int32_t len_i`，随后是 `len_i` 字节的该 token 的字符串表示。

成员变量	数据类型	含义
vocab_size	int32_t	encoded_vocab 中的 token 数量
encoded_vocab	const uint8_t*	序列化的词表数据块（布局见上）
encoded_vocab_len	int32_t	encoded_vocab 的总字节数
n_stop_tokens	int32_t	停止（EOS）token ID 的数量
stop_tokens	const int32_t*	长度为 n_stop_tokens 的停止 token ID 数组

RKNN3_GRAMMAR

不透明的会话级 grammar 句柄类型。持有语法规则字符串/类型及按请求的 GrammarMatcher 状态，共享上下文级的 TokenizerInfo / GrammarCompiler。通过 `rknn3_grammar_create` 创建，`rknn3_grammar_destroy` 销毁。

RKNN3_GRAMMAR_SET_OPTIONS

`rknn3_grammar_set` / `rknn3_session_set_grammar` 的编译选项。值为 -1（int类型字段）或 NULL（字符串字段）表示使用 xgrammar 默认值；options 指针传 NULL 等价于全部使用默认值。选项在设置时

生效（每次设置都会重新编译语法）。

成员变量	数据类型	含义
any_whitespace	int	JSON Schema：-1=默认(true)，0=false，1=true
indent	int	JSON Schema：-1=默认(None)，>=0=缩进空格数
strict_mode	int	JSON Schema：-1=默认(true)，0=false，1=true
max_whitespace_cnt	int	JSON Schema：-1=默认(None)，>=0=元素间最大空白数
any_order	int	JSON Schema：-1=默认(false)，0=false，1=true
item_separator	const char*	JSON Schema：NULL=默认，否则为元素分隔符（逗号）
key_value_separator	const char*	JSON Schema：NULL=默认，否则为键值分隔符（冒号）
root_rule_name	const char*	EBNF：NULL=默认("root")

6.11. MatMul类型

以下类型定义于 `rknn3_matmul_api.h`。输入结构体及其 `reserved` 字段必须清零。

rknn3_matmul_context

不透明MatMul句柄，0为无效值；通过 `rknn3_matmul_init` 创建、`rknn3_matmul_destroy` 释放，与父 `rknn3_context` 分别管理。

rknn3_matmul_type

W表示B（权重）的位宽，A表示输入A的位宽。实际类型如下，以查询返回的 `dtype` 为准。

枚举	A	B	C	set_quant要求
RKNN3_MATMUL_W16A16	FP16	FP16	FP16	无
RKNN3_MATMUL_W4A16	FP16	INT4	FP16	仅B，分组量化
RKNN3_MATMUL_W6A16	FP16	INT6	FP16	仅B，分组量化
RKNN3_MATMUL_W8A16	FP16	INT8	FP16	仅B，分组量化
RKNN3_MATMUL_W8A8_PER_CHANNEL	INT8	INT8	INT8	A/C按tensor，B按输出通道
RKNN3_MATMUL_W8A8_PER_LAYER	INT8	INT8	INT8	A/C按tensor，B按层
RKNN3_MATMUL_W8A8O32	INT8	INT8	INT32	无，K不超过4096

`RKNN3_MATMUL_TYPE_MAX` 为枚举边界，不可作为运行类型。

rknn3_matmul_desc

字段	说明
M / K / N	矩阵乘法维度，均大于0。
A_batch / B_batch	A/B批次数，均大于0；C_batch=max(A_batch,B_batch)，必须能被两者整除。
type	本句柄的计算类型。

字段	说明
A_layout / C_layout	当前均设置为 `RKNN3_TENSOR_NCHW`，描述逻辑输入/输出。
B_layout	普通B设置NCHW，查询得到打包权重布局；单形状W16A16可设NC1HWC2以使用feature B。
group_size	W4A16/W6A16/W8A16设32、64或128；其他类型设0。
core_mask	单个NPU核心的掩码，不能组合多个核心；AUTO当前选择核心0。
reserved[64]	保留，全部置0。

一个句柄内只有M和A_batch可以不同；type、core_mask、B_batch、K、N、group_size、B_layout及查询得到的B大小和布局必须一致。feature B仅适用于单形状W16A16。

规范形状与逻辑存储： 查询属性中的 `n_dims=4`，shape始终采用下表的逻辑坐标，即使NPU物理布局为NC1HWC2。

Tensor	shape	连续NCHW逻辑索引
A	[A_batch,K,1,M]	`(a_batch*K+k)*M+m`
B	[B_batch,K,1,N]	`(b_batch*K+k)*N+n`
C	[C_batch,N,1,M]	`(c_batch*N+n)*M+m`

单批次下，`C[n*M+m] = sum_k A[k*M+m]*B[k*N+n]`（W8A8O32）；其他类型按相应浮点或量化规则计算。逻辑shape不能用物理填充维度替换。

rknn3_matmul_config

仅包含 `uint8_t reserved[256]`。默认构建可传NULL，或传全部清零的结构体。

rknn3_matmul_tensor_attr

字段	说明
n_dims / shape	规范逻辑形状，见上表。
n_strides / strides	物理存储步长，单位为存储元素而非字节。NC1HWC2有5项，按[N,C1,H,W,C2]排列；打包权重及连续逻辑端为0项。
size	字节数；NPU端包含填充，是最低存储容量要求；逻辑端为连续数据字节数。
alignment	NPU有效起始地址所需的字节对齐，逻辑端忽略。
quant_info	量化要求，查询结果不含实际scale/zero point数组。
layout / dtype	存储布局与计算数据类型；INT4/INT6逻辑端使用解包int8存储。
core_id	分配NPU内存时使用的核心编号。
reserved[32]	保留，全部置0。

NPU端属性应原样使用查询结果。`rknn3_tensor_mem.size` 表示整个缓冲区容量，不能与属性中的 `size` 混淆；必须确保 `offset <= size` 且剩余容量满足属性要求。分配时按查询的 `core_id` 选择核心，并保证有效起始地址 `phys_addr + offset` 满足 `alignment`；需要额外对齐空间时可增加分配大小。CPU访问与NPU访问必须使用同一offset。

rknn3_matmul_info

字段	说明
shape_id	注册形状的索引。
type / group_size	该形状的计算类型与分组大小。
A / B / C	三个用户NPU缓冲区的 `rknn3_matmul_tensor_attr`。
reserved[32]	保留字段。

A/C查询布局为NC1HWC2。描述中的B_layout为NCHW时，查询B布局为O1I1HWI2O2；为NC1HWC2时，查询B布局为NC1HWC2。只需按查询结果分配A/B/C，其他运行资源由运行时管理。

rknn3_matmul_quant_type_t

枚举	说明
RKNN3_MATMUL_QUANT_NONE	无量化参数。
RKNN3_MATMUL_QUANT_PER_TENSOR	每tensor一组；B按批次分别提供。
RKNN3_MATMUL_QUANT_PER_CHANNEL	B每个输出通道一组。
RKNN3_MATMUL_QUANT_GROUP_PER_TOKEN	B沿K方向分组，每批次、每组、每输出通道一组。

rknn3_matmul_quantization_t

字段	说明
type / group_size	量化方式和分组大小，匹配查询结果。
scale_dtype / zero_point_dtype	scale数据类型及算子的量化零点类型。
scales / scales_size	scale数组指针及字节数，元素数为num_quant_params。
zero_points / zero_points_size	零点数组指针及字节数；数组存储始终为解包 `int32_t`。
num_quant_params	参数组数，按查询结果准备数组。
reserved[3]	保留，全部置0。

查询时 `scales` / `zero_points` 为NULL。提交时可复制对应的 `quant_info`，然后填写数组指针和实际字节数。`scales_size = num_quant_params * scale元素字节数`，`zero_points_size = num_quant_params * sizeof(int32_t)`；零点不能按 `zero_point_dtype` 位宽进行打包。

类型	A参数数	B参数数	C参数数	scale存储
W4A16/W6A16/W8A16	无	B_batch×ceil(K/group_size)×N	无	FP16位模式，每个2字节
W8A8 per-channel	1	B_batch×N	1	FP32，每个4字节
W8A8 per-layer	1	B_batch	1	FP32，每个4字节

量化数组应覆盖全部批次及对应的分组或输出通道；所需参数数以查询返回的 `num_quant_params` 为准。W16A16/W8A8O32没有量化数组。

SCALES与ZERO_POINTS的存储约定

`scales` 保存缩放因子，`zero_points`（以下简称zp）保存零点。对于一个量化值q，其对应的实数值为 `scale * (q - zp)`。每个量化单元由一对scale/zp描述；两个数组的元素个数和逻辑shape相同，`scales[i]` 与 `zero_points[i]` 始终对应同一单元。

两个数组分别连续存储，不交错排列。下面的shape描述用户提交数组的逻辑维度，最右侧维度连续；无需按NPU的NC1HWC2或O1I1HWI2O2布局补齐、重排，也不应提交设备Param的打包结果。

计算类型	Tensor	scales与zp的逻辑shape	每个数组的元素数	scale元素存储	zp元素存储
W4A16/W6A16/W8A16	B	`[B_batch,G,N]`，`G=ceil(K/group_size)`	`B_batch*G*N`	FP16位模式，2字节	`int32_t`，4字节
W8A8 per-channel	A、C各自	`[1]`	各1个	`float`（FP32），4字节	`int32_t`，4字节
W8A8 per-channel	B	`[B_batch,N]`	`B_batch*N`	`float`（FP32），4字节	`int32_t`，4字节
W8A8 per-layer	A、C各自	`[1]`	各1个	`float`（FP32），4字节	`int32_t`，4字节
W8A8 per-layer	B	`[B_batch]`	`B_batch`	`float`（FP32），4字节	`int32_t`，4字节
W16A16/W8A8O32	A/B/C	无	0	无	无

`scales_size`、`zero_points_size` 是字节数，不是元素数。两个指针虽然声明为 `const uint8_t*`，实际元素存储仍按上表准备。尤其注意：FP16 scale必须先转换成FP16位模式，不能把FP32数组指针强制转换后当成FP16数组；所有zp均用 `int32_t` 数组，不能使用 `int8_t` 数组或INT4/INT6位打包。

所有scale必须为有限正数。即使采用零点为0的量化，也要提供包含0的zp数组，不能用NULL代替；需要量化的Tensor必须同时提供scale和zp。实际数值由调用者提供，查询只返回格式、数量和大小要求。

W4A16/W6A16/W8A16：B沿K方向分组

A、C为FP16，无需提交它们的scale/zp；调用 `set_quant(matmul, NULL, &quant_B, NULL)`。B的逻辑shape为 `[B_batch,K,1,N]`，每个批次、每个K分组、每个输出通道n分别使用一对参数。

令 `G=ceil(K/group_size)`。对于B中的逻辑坐标 `(batch,k,n)`：

```
group = k / group_size                  // 整数除法
index = (batch * G + group) * N + n
scale = scales[index]
zp    = zero_points[index]
B_real(batch,k,n) = scale * (B_quant(batch,k,n) - zp)
```

同一组中，相同输出通道的所有k共享scale/zp。数组先排列batch，再排列group，每个group内按 `n=0..N-1` 连续排列。`group_size` 为32/64/128；K不能整除group_size时，最后不足一组的有效k仍使用完整的一组N个参数，不按物理填充后的K增加参数。

例如 `B_batch=2,K=65,N=4,group_size=32`，则 `G=3`，两个数组的shape均为 `[2,3,4]`，各24个元素：

数组索引	B的batch	覆盖的有效k	n顺序
0..3	0	0..31	0..3
4..7	0	32..63	0..3
8..11	0	64	0..3
12..15	1	0..31	0..3
16..19	1	32..63	0..3
20..23	1	64	0..3

坐标 `(batch=1,k=64,n=3)` 使用 `scales[23]` 和 `zero_points[23]`。本例 `scales_size=24*2=48` 字节，`zero_points_size=24*4=96` 字节。

三个类型的数组shape和索引相同，仅B权重位宽及查询返回的零点类型不同。当前接口中W4A16的 `zero_point_dtype=INT4`，zp范围为 `[-8,7]`；W6A16/W8A16的 `zero_point_dtype=INT8`，zp范围为 `[-128,127]`。这些范围是值的约束，数组存储始终是 `int32_t`。

整个句柄成功提交一次B参数即可；切换M/A_batch不改变B的量化数组。

W8A8 PER-CHANNEL：B每个输出通道一对参数

A、B、C均为INT8，调用 `set_quant(matmul, &quant_A, &quant_B, &quant_C)`。A和C各使用一对标量参数，覆盖各自全部元素及全部批次；A/C的参数个数不会随M、A_batch或C_batch增加。

B的两个数组shape均为 `[B_batch,N]`。对于 `(batch,k,n)`，使用 `index=batch*N+n`，该输出通道的所有k共享 `scales[index]` 和 `zero_points[index]`。这里的channel是B的输出通道n，不是K，也不是A的行m。

例如 `B_batch=2,N=3`，B数组索引依次为 `(batch,n)=(0,0),(0,1),(0,2),(1,0),(1,1),(1,2)`，各6个元素；B的两个数组各24字节。A和C各自的scale数组和zp数组均为1个元素、各4字节。

量化映射为：

```
A_real = A_scale[0] * (A_quant - A_zp[0])
B_real(batch,k,n) = B_scale[batch*N+n] * (B_quant(batch,k,n) - B_zp[batch*N+n])
C_real = C_scale[0] * (C_quant - C_zp[0])
```

其中C参数描述输出INT8的量化映射。A/B/C的scale均为FP32，zp均以 `int32_t` 存储且取值范围为 `[-128,127]`。每个shape分别提交一次A/B/C参数；不同shape的B数组值须一致，A/C的标量值可以不同。

W8A8 PER-LAYER：B每个批次一对参数

A/C的参数shape、存储类型和提交方式与per-channel相同。B的两个数组shape为 `[B_batch]`，每个批次的全部K×N个权重共享一对参数：

```
index = batch
B_real(batch,k,n) = B_scale[batch] * (B_quant(batch,k,n) - B_zp[batch])
```

例如 `B_batch=2` 时，B各有2个scale/zp元素，两个数组各8字节；单批次时各有1个元素，两个数组各4字节。参数个数不随K或N增加。scale为FP32，zp用 `int32_t` 存储且范围为 `[-128,127]`；多shape提交规则与per-channel相同。

W16A16/W8A8O32：不提交SCALE/ZP

这两个类型不调用 `set_quant()`。W16A16直接使用FP16输入；W8A8O32使用INT8输入并输出INT32累加结果，不接受额外的A/B/C scale/zp数组。

7. API变更、废弃和迁移

v0.3.0 -> v0.4.0 Session API 主要改动 (2025-11-03)

> 参考 demo1: rknn3-runtime/examples/rknn3_session_test_demo
>
> 参考 demo2: rknn3_model_zoo/examples/Qwen2_5_VL

1. 新增多模态 `音频/视频` 输入类型，并支持多种模态数据以任意顺序和组合方式输入。
2. 在输入 prompt 时，应明确插入各模态的标记标签，例如：`你需要做3件事：1.<audio>将这段语音转为文本； 2.<image>描述一下这张图片的内容； 3.<video>描述一下这段视频的内容`。
3. rknn3_llm_multimodal_tensor 结构体优化：例如，将原有的 input_tensor.n_image 等成员调整为 input_tensor.image.n_image 等子结构体表达。
4. 新增 `思考（thinking）模式` 切换功能。
5. 通过设置 enable_thinking = true 开启思考模式 (默认是 false)。
6. 新增支持配置多个 `eos token` 来控制推理结束。
7. params.vocab_info.special_eos_id 的初始化方式更新为以列表形式进行赋值。
8. 新增 `rknn3_dump_features` 接口，用于按层运行模型并将中间Tensor以 `.npy` 形式导出。支持未提供输出时自动分配输出Tensor。

v0.4.0b0 -> v0.5.0 主要改动 (2025-11-29)

1. 新增后处理相关查询命令与自定义算子插件机制。
2. 新增 `RKNN3_QUERY_POSTPROCESS_*` 查询项。
3. 新增 `rknn3_register_custom_ops_plugins` 以及相关结构体与枚举。
4. LLM回调与配置增强。
5. 新增 `rknn3_session_set_function_tools` 接口。
6. 以 `LLMOutputCallback` + `LLMOutputCallbackState` 取代“最后隐藏层回调”。
7. `rknn3_llm_config` 新增 `model_type` 和 `task_type` 字段。
8. `rknn3_kvcache_policy_param` 新增 recurrent 参数。
9. 图像/YUV相关接口完善。
10. `rknn3_im_mem_create` 参数扩展，支持 size/core_id/flags。
11. 新增 `rknn3_im_proc_flag` 枚举，用于描述图像处理流程。

v0.5.0 -> v1.0.0 主要改动 (2025-12-22)

1. 新增性能与内存分析接口。
2. 新增 `rknn3_profile_ops`、`rknn3_profile_mem`。

v1.0.0 -> v1.0.4 主要改动 (2026-05-09)

1. 新增流式权重加载接口。
2. 新增 `rknn3_load_weight_chunk` 接口，支持在流式模式下分段上传权重数据到NPU设备。
3. `rknn3_load_model_from_path` 和 `rknn3_load_model_from_data` 支持传入NULL权重参数进入流式权重加载模式。
4. `rknn3_model_init` 在流式权重加载模式下仅完成模型配置，权重数据需通过 `rknn3_load_weight_chunk` 分段上传。
5. 新增 `rknn3_create_mem_from_fd` 接口，支持从文件描述符创建tensor内存对象。
6. API头文件优化。
7. `rknn3_api.h` 移除对非必要数据类型的依赖。

v1.0.4 -> v1.1.0 主要改动 (2026-08-03)

1. 新增KV缓存分组机制：`rknn3_attention_type` 枚举、`rknn3_attention_kvcache_lens` / `rknn3_kvcache_len_group_info` 结构体、`RKNN3_QUERY_KVCACHE_LEN_GROUP_INFO` 查询命令、`rknn3_llm_config` 新增注意力KV缓存长度配置字段。
2. 新增接口：`rknn3_set_kvcache_group`、`rknn3_set_weight_mem`、`rknn3_session_pause`、`rknn3_session_resume`、`rknn3_mem_sync_range`。
3. 补全自定义算子相关定义：
4. 新增 `rknn3_custom_op_attr` 结构体文档。
5. `rknn3_custom_op_context` 结构体补全 `get_param` 函数指针字段说明。
6. 补全结构体字段：`RKLLMCallback`（input_callback等）、`RKLLMRunState`（n_input_tokens等）、`rknn3_kvcache_policy_param`（checkpoint_tail_overwrite）、`rknn3_llm_infer_param`（prefill_only/disable_sampling）；`rknn3_sdk_version` 字段长度 64 修正为 256；`RKNN3_MAX_LORA_NUM` 由 128 修正为 32。
7. 补全 `RKNN3_QUERY_CUSTOM_STRING` 查询命令枚举值。
8. 修正 `rknn3_session_save_kvcache` 接口参数类型：`char*` 修正为 `const char*`。
9. `rknn3_mem_sync` 接口补充 `MEM_SYNC_CHUNK_SIZE` 环境变量说明（FROM_DEVICE模式分块传输，默认2MiB）。
10. 与 `rknn3_api.h` 头文件同步，修正以下接口与数据结构定义不一致项：
11. `rknn3_tensor_attr` 结构体补全 `n_orig_dims`、`orig_shape[RKNN3_MAX_DIMS]`、`orig_layout` 三个字段。
12. `rknn3_im_mem_create` 接口参数 `width` / `height` / `size` / `core_id` 类型由 `int` 修正为 `int32_t`。
13. 多个结构体的数组类型字段补全数组维度：`rknn3_core_mem_size` / `rknn3_config` / `rknn3_shape_info` / `rknn3_allocation_info` / `rknn3_init_extend` / `rknn3_llm_config` / `rknn3_vocab_info` / `rknn3_llm_extend_param` / `rknn3_llm_infer_param` 的 `reserved` 字段补全为 `uint8_t[N]`；`rknn3_dev_mem_info` 的 `node_mem_info` 补全为 `rknn3_node_mem_info[RKNN3_MAX_NPU_NODE_NUM]`；`rknn3_devices` 的 `devices` 补全为 `rknn3_device[RKNN3_MAX_DEVS]`。

v1.1.0 -> v1.2.0 主要改动 (2026-10-22)

1. 新增PagedAttention（分页KV cache）支持：
2. 新增页表绑定接口 `rknn3_set_kvcache_pages`，支持按 `(core_id, cache_type, request_index)` 建立页绑定，页内存由调用方通过 `rknn3_create_mem` / `rknn3_destroy_mem` 管理。
3. 新增查询命令 `RKNN3_QUERY_KVCACHE_PAGE_INFO` 及 `rknn3_kvcache_page_info`、`rknn3_kvcache_page_binding`、`rknn3_cache_page_attr`、`rknn3_page_cache_type` 枚举等数据结构。
4. 使用流程见 4.4.4 节（Page KV cache内存）。
5. 新增结构化输出(grammar)接口，基于xgrammar实现JSON Schema/EBNF/正则等输出约束：
6. 上下文级引擎初始化：`rknn3_init_grammar`。
7. grammar句柄管理：`rknn3_grammar_create`、`rknn3_grammar_destroy`。
8. 语法设置/清除/重置：`rknn3_grammar_set`、`rknn3_grammar_clear`、`rknn3_grammar_reset`。
9. 采样约束：`rknn3_grammar_apply_mask`、`rknn3_grammar_accept_token`。
10. 会话级便捷接口：`rknn3_session_set_grammar`、`rknn3_session_clear_grammar`。
11. 新增 `rknn3_grammar_type` 枚举、`rknn3_grammar_config` / `rknn3_grammar` / `rknn3_grammar_set_options` 结构体及 `RKNN3_GRAMMAR_SET_OPTIONS_INIT` 初始化宏。
12. 补全枚举值：`rknn3_tensor_type` 新增 `RKNN3_TENSOR_INT6`；`rknn3_kvcache_clear_policy` 补全 `RKNN3_KVCACHE_KEEP_N_TOKENS`。
13. `rknn3_llm_infer_param` 新增 `keep_grammar` 字段（`reserved` 数组由 128 字节缩减为 127 字节）：启用后每次 `rknn3_session_run` 开始/结束时不再重新初始化 per-request grammar matcher，在多次运行之间保留 grammar 状态以支持连续生成。
14. LoRA接口更新：`rknn3_lora_init` / `rknn3_lora_init_from_data` 支持重复调用以注册多个权重文件/缓冲区（所有已注册文件中的适配器名称必须唯一）；`rknn3_lora_enable` 与 `rknn3_session_enable_lora` 明确映射到同一pattern槽位的不同适配器不能同时启用（不同会话可复用该槽位，运行时会在推理前重新应用每个会话的LoRA绑定）。
15. 新增 `rknn3_load_model_from_device` 接口，支持从设备端文件系统（emmc）加载RKNN3模型。`model_path` / `weight_path` 指设备（如协处理器）文件系统上的路径，由设备端本地读取解析，而非Host端路径。